



National University of Computer and Emerging Sciences, Lahore



FYP Report Title

Name of Student 1 Roll No. 1 BS(CS/DS/SE)

Name of Student 2 Roll No. 2 BS(CS/DS/SE)

Name of Student 3 Roll No. 3 BS(CS/DS/SE)

Supervisor: SupervisorName

Final Year Project

October 10, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: FYP Report Title

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date:

Name:

Signature:

Name:

Signature:

Name:

Signature:

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

An Abstract is a short summary of the work being reported. Three to five sentences describing the essence of the work. An abstract is a short, 50-125 words summary of a work. An Abstract should state your work's purpose, findings, and conclusion without commenting on or evaluating the work itself. It should be only one paragraph. Put the abstract on a separate page that follows the title page.

Executive Summary

The executive summary should be one to two pages' of an overview of the information contained in the FYP report. It should give the reader an easy reference, in a very brief form, to the important information contained in the report and explained in more detail in the body of the report. People reading the report will use this section as a reference during presentations.

Table of Contents

List of Figures	x
List of Tables	xi
1 Introduction	1
1.1 Purpose of this Document	1
1.2 Intended Audience	2
1.3 Definitions, Acronyms, and Abbreviations	2
1.4 Conclusion	2
2 Project Vision	3
2.1 Problem Domain Overview	3
2.2 Problem Statement	3
2.3 Problem Elaboration	3
2.4 Goals and Objectives	3
2.5 Project Scope	3
2.6 Sustainable Development Goal (SDG)	4
2.7 Constraints	5
2.8 Business Opportunity	5
2.9 Stakeholders Description/ User Characteristics	5
2.9.1 Stakeholders Summary	5
2.9.2 Key High-Level Goals and Problems of Stakeholders	5
3 Literature Review / Related Work	6
3.1 Definitions, Acronyms, and Abbreviations	6
3.2 Detailed Literature Review	7
3.2.1 General-Purpose Audio Self-Supervised Learning by Masked Prediction	8
3.2.2 The Masked Modeling Duo Family	9
3.2.3 Other Latent-Prediction Models: JEPA and Classification-Based Variants	14

3.2.4	Music-Specific Self-Supervised Models	20
3.2.5	Further Pre-training, Forgetting and Adaptation	25
3.2.6	Public Datasets	31
3.2.7	Evaluation Benchmarks and Protocols	37
3.2.8	Related Applications and Frameworks	39
3.3	Literature Review Summary Table	40
3.4	Conclusion	43
4	Software Requirement Specifications	45
4.1	List of Features	45
4.2	Functional Requirements	46
4.3	Quality Attributes	47
4.3.1	Usability	47
4.3.2	Correctness and Consistency	47
4.3.3	Reliability	47
4.3.4	Maintainability and Extensibility	48
4.3.5	Portability	48
4.4	Non-Functional Requirements	48
4.4.1	Performance	48
4.4.2	Capacity and Concurrency	48
4.4.3	Availability	48
4.4.4	Security and Privacy	49
4.4.5	Reusability	49
4.4.6	Compatibility	49
4.5	Assumptions	49
4.6	Use Cases	50
4.7	Hardware and Software Requirements	55
4.7.1	Hardware Requirements	55
4.7.2	Software Requirements	55
4.8	Graphical User Interface	56
4.9	Database Design	59
4.9.1	ER Diagram	59
4.9.2	Data Dictionary	61
4.10	Risk Analysis	62
4.10.1	Technical Risks	62

4.10.2	Schedule Risks	63
4.10.3	Legal and Licensing Risks	63
4.10.4	Misinterpretation Risks	63
4.11	Conclusion	64
5	Proposed Approach and Methodology	65
5.1	Methodological Foundations	65
5.1.1	Masked Latent Prediction and M2D	65
5.1.2	Specializing a General Model: M2D-X	66
5.1.3	Music-Specific Self-Supervised Models	67
5.1.4	Further Pre-training, Forgetting and Regularization	68
5.1.5	Design Decisions Derived from the Literature	69
5.2	Research Questions and Hypotheses	69
5.3	Dataset Acquisition	70
5.3.1	Choice of Corpus	70
5.3.2	Archives and Metadata	71
5.3.3	Eligibility Rules and Subset Selection	71
5.3.4	Background-Noise Corpus	72
5.3.5	Licensing and Ethics	72
5.4	Data Preprocessing Pipeline	73
5.4.1	Audit and Decoding	73
5.4.2	Front-End	74
5.4.3	Cropping and Normalization	74
5.4.4	Musical-Teacher Targets	75
5.4.5	Storage, Manifest and Integrity Checks	75
5.5	Further Pre-training Framework	76
5.5.1	Overview, Initialization and Alternatives Considered	76
5.5.2	Masked Latent Prediction on Noisy Music	77
5.5.3	Background Noise and Denoising	77
5.5.4	Frozen-Teacher Regularization	78
5.5.5	Musical Teacher	78
5.5.6	Joint Objective and Optimization	79
5.5.7	Compute and Memory Budget	79
5.6	Experimental Design	80
5.6.1	Reference Models	81

5.6.2	Pre-training Arms	81
5.6.3	Controls and Reproducibility	82
5.7	Evaluation Protocol	83
5.7.1	Downstream Tasks	83
5.7.2	Frozen-Feature Linear Evaluation	83
5.7.3	Metrics, Aggregates and Statistics	84
5.7.4	Threats to Validity	85
5.8	Conclusion	85
6	High-Level and Low-Level Design	87
6.1	System Overview	87
6.1.1	Data Preparation Pipeline	87
6.1.2	Further Pre-training Framework	87
6.1.3	Evaluation and Analysis	88
6.1.4	Demonstration Dashboard	88
6.2	Design Considerations	88
6.2.1	Assumptions and Dependencies	88
6.2.2	General Constraints	89
6.2.3	Goals and Guidelines	89
6.2.4	Development Methods	90
6.3	System Architecture	90
6.3.1	Subsystem Architecture	91
6.3.2	Sequence Diagrams	93
6.4	Architectural Strategies	93
6.4.1	Extending the M2D Code Base	93
6.4.2	Offline Precomputation of Inputs and Targets	94
6.4.3	Experiments as Command Lines and Run Folders	95
6.4.4	One Evaluation Interface for All Models	95
6.4.5	Files as Interfaces and Single-Worker Serving	95
6.5	Domain Model/Class Diagram	96
6.6	Policies and Tactics	96
6.6.1	Choice of Tools	96
6.6.2	Coding Guidelines and Conventions	96
6.6.3	Testing	97
6.6.4	Requirements Traceability	98

6.6.5	Source Organization and Build	98
6.6.6	Maintenance	98
6.7	Conclusion	99
7	Implementation and Test Cases	100
7.1	Implementation	100
7.1.1	Implementation of First Component/Algorithm	100
7.2	Test case Design and description	100
7.3	Test Metrics	101
7.3.1	Sample Test case Metric.No.2	101
8	User Manual	102
9	Experimental Results and Discussion	103
10	Conclusions	104
10.1	aa	104
10.1.1	dfs	104
A	First Appendix if Required	112
A.1	References	112
A.2	Equations	112
A.3	Figures	112
A.4	Tables	113
A.5	Pseudo Code	113
A.6	Code of Programming Languages	113
A.7	Recommendations for L ^A T _E X	114

List of Figures

2.1 This figure represents all the SDGs that can be targets of an FYP; Adding this sub-caption is mandatory; make sure you describe the picture in a way that if a visually impaired person is reading using a text reader, they can understand whats in this picture.	4
4.1 Use-case diagram of the dashboard	50
4.2 Navigation flow of the dashboard	56
4.3 Wireframe of the Results Overview	56
4.4 Wireframe of the Demo screen: task and input	57
4.5 Wireframe of the Demo screen: output	57
4.6 Wireframe of the comparison view	58
4.7 Wireframe of the About page	58
4.8 ER diagram of the dashboard database	60
5.1 Overview of the proposed methodology	65
5.2 Data acquisition and preprocessing pipeline	73
5.3 Architecture of the further pre-training framework	76
6.1 Layered system architecture	91
6.2 Modules of the further pre-training subsystem	92
6.3 Sequence diagram of one further pre-training step	93
6.4 Sequence diagram of a dashboard prediction request	94
6.5 Class diagram	97
A.1 Fast Logo is the Caption.	112

List of Tables

3.1	Summary of the reviewed research studies	40
3.2	Summary of the reviewed public datasets	42
3.3	Summary of related applications and frameworks	43
4.1	Functional requirements of the dashboard	46
4.2	UC-01 View Research Results	51
4.3	UC-02 Select a Downstream Task	51
4.4	UC-03 Provide Audio Input	52
4.5	UC-04 Run Inference and View Output	53
4.6	UC-05 Compare Two Models on a Clip	53
4.7	UC-06 Import Evaluation Results	54
4.8	Data dictionary of the dashboard database	61
4.9	Summary of risks of the dashboard	63
5.1	Literature findings and the design decisions they lead to	69
5.2	FMA audio archives and their use in this study	71
5.3	Front-end and target parameters	74
5.4	Fixed settings of the further pre-training runs	79
5.5	Reference models	81
5.6	Pre-training arms	82
5.7	Downstream tasks	83
7.1	Sample Test case No.1	100
7.2	Sample Test case Matric.No.1	101
A.1	Give a Caption to the Table.	113

Chapter 1 Introduction

This chapter is mandatory. The introduction should contain a brief overview of the problem being addressed and the background information needed for the reader to understand the work being done and the reasoning behind it. The last paragraph of the introduction chapter should contain an outline of the entire report. Summarize each chapter in one line to make the last paragraph.

1.1 Purpose of this Document

Specify the purpose of this Project. The purpose of the document in an FYP report is to clearly and concisely state the main objective and goals of the project, as well as the research question(s) or hypothesis(es) being investigated. It should provide a brief overview of the background and context of the project, and explain why the research is important and relevant. The purpose of the document should be written in a way that is easy for the reader to understand, and should be included at the beginning of the report.

In addition to providing an overview of the project, the purpose of the document should also outline the specific objectives and goals of the research, such as what the project aims to achieve and what the research questions or hypotheses are. This will help the reader understand the scope of the project and what to expect from the report. It is also important to include a brief description of the methods and techniques used in the research, as well as any limitations or assumptions of the project. This will provide the reader with a clear understanding of how the research was conducted and what the results and conclusions are based on. Finally, it should be clear what the report is going to cover, what kind of information will be presented, and in what order.

For Example, "The purpose of this document is to present the design, implementation, and evaluation of our Final Year Project (FYP) which aims to develop a web-based application for small businesses to manage their inventory and customer orders. The project will focus on creating a user-friendly interface and providing a seamless user experience. The goal of this research is to provide a Minimum Viable Product (MVP) that can be used by small businesses to improve their inventory management and customer order processing. The research question we aim to answer is: Can we develop a web-based application that can improve inventory management and customer order processing for small businesses? This report will present the methodology, design, implementation, testing, and evaluation of our project, as well as the limitations and future work."

1.2 Intended Audience

Identifying and writing the intended audience for a Final Year Project (FYP) report is an important step in the writing process. The intended audience is the group of people for whom the report is written and who will primarily read and use the report. It is crucial to identify the intended audience early in the writing process as this will guide the report's style, tone, and level of detail.

To identify the intended audience, consider the report's purpose, who will be using the report and for what purpose. For example, the intended audience for a FYP report in Computer Science may be a panel of professors who will be evaluating the project. In contrast, the intended audience for a FYP report in Business may be potential investors or clients.

1.3 Definitions, Acronyms, and Abbreviations

List all important definitions, acronyms, and abbreviations used in this document. For example: **SDG**: Sustainable Development Goal

FYP: Final Year Project

MVP: Minimum Viable Product

UI: User Interface

UX: User Experience

Agile: Agile Development

SCRUM: Scrum Development

REST: Representational State Transfer

ORM: Object-Relational Mapping

CRUD: Create, Read, Update, Delete

API: Application Programming Interface

1.4 Conclusion

The last paragraph of the introduction chapter should contain an outline of the entire report. Summarize each chapter in one line to make the last paragraph.

Chapter 2 Project Vision

For each related work provide a paragraph of introduction and at the end, a paragraph of conclusions. Give a page break after the chapter ends. This chapter is mandatory. Clearly specify the goals and objectives of the project along with the scope of the project. (You can make sub-heading of goals and objectives and the scope of the project).

2.1 Problem Domain Overview

Describe in full detail what your system does. This section can be further divided into sub-sections, but basically, a paragraph description of the system is required.

2.2 Problem Statement

Provide the **problem** which you are trying to solve.

2.3 Problem Elaboration

Describe the problem in detail and the various sub-problems you would work on.

2.4 Goals and Objectives

Write goals and objectives here.

2.5 Project Scope

Specify the scope of your project here.

For example, The scope of this project is to design and develop a web-based application for small businesses to manage their inventory and customer orders. The application will allow small businesses to:

- Create, read, update, and delete inventory items
- Create, read, update, and delete customer orders
- Generate reports on inventory levels and customer orders
- Send notifications to customers about their orders
- Provide an easy-to-use user interface and a seamless user experience

The project will be developed using a combination of HTML, CSS, JavaScript, and a web-development framework such as React.js or Angular.js. The application will use a RESTful API to communicate with a back-end database, which will be implemented using a ORM library. The project will be developed using Agile development principles, and project management will be done using SCRUM.

The project deliverables will include:

- A functional web-based application for small businesses to manage their inventory and customer orders
- A user manual for the application
- A documentation of the design, implementation, and testing process

The project scope does not include the following:

- Providing hosting or server infrastructure for the application
- Developing mobile applications (iOS or Android)
- Integrating the application with third-party services or APIs

This project scope will be used as a guide throughout the development

2.6 Sustainable Development Goal (SDG)

Identify at least one SDG area that you are targeting in this project. This has to be documented as a paragraph. The complete SDG list is shown in Figure ??.



Figure 2.1: This figure represents all the SDGs that can be targets of an FYP; Adding this sub-caption is mandatory; make sure you describe the picture in a way that if a visually impaired person is reading using a text reader, they can understand what's in this picture.

2.7 Constraints

2.8 Business Opportunity

2.9 Stakeholders Description/ User Characteristics

Identify and briefly describe who the users of this system will be, and their roles.

2.9.1 Stakeholders Summary

2.9.2 Key High-Level Goals and Problems of Stakeholders

Notice how a section's heading is changed to "heading 1". It will show up in the table of contents as a section under a chapter. Notice its numbering also. Similarly, subsections can have heading 2 and heading 3 styles.

NOTE: Subsections from 2.7 to 2.9 are optional for Research projects but compulsory for Development projects.

Chapter 3 Literature Review / Related Work

This chapter reviews the work on which the project builds. It is organized into eight categories: general-purpose audio self-supervised learning by masked prediction; the Masked Modeling Duo (M2D) family; other latent-prediction models; music-specific self-supervised models; further pre-training, forgetting and adaptation; public datasets; evaluation benchmarks; and related applications and frameworks. Each research item is summarized, analyzed critically and related to the proposed work. The chapter ends with summary tables and the research gap that this project addresses.

3.1 Definitions, Acronyms, and Abbreviations

Audio foundation model: A large model pre-trained with self-supervised learning on broad audio data and reused for many downstream tasks.

Catastrophic forgetting: The loss of previously learned ability when a model is trained further on new data.

CLAP: Contrastive language-audio pre-training, which aligns audio and text embeddings.

CQT: Constant-Q transform, a spectral transform with logarithmically spaced frequency bins, so that every octave has the same number of bins.

DAPT: Domain-adaptive pre-training, that is, continuing self-supervised pre-training on unlabeled data from the target domain.

Denoising distillation: Training a student on noisy input to match the features that a teacher computes from the clean input.

EMA: Exponential moving average, used to update the target network from the online network.

Encodec / RVQ: A neural audio codec whose residual vector quantization (RVQ) turns audio into discrete codes.

Further pre-training: Resuming self-supervised pre-training of an already pre-trained model on a new, usually smaller, dataset; also called continued or continual pre-training.

JEPA: Joint-embedding predictive architecture, which predicts the latent representation of hidden regions from a visible context.

Linear probing: Evaluating a frozen encoder by training only a linear layer on its features.

Log-mel spectrogram: A time-frequency representation with mel-spaced frequency bands and logarithmic magnitude.

MaLaP: Masked latent prediction, that is, predicting the latent representation of masked patches instead of their raw values.

Masking ratio: The fraction of input patches that are hidden from the online network.

MCL: Multiple choice learning, where several prediction hypotheses are produced and the best one is trained.

mAP / ROC-AUC: Mean average precision and area under the receiver operating characteristic curve, used for multi-label tagging.

Mel-RVQ: The residual vector quantizer applied to mel spectra that provides the targets of MuQ.

MIR: Music information retrieval.

MLM: Masked language modeling, the BERT-style objective of predicting hidden tokens.

Offline network: In M2D-X, the additional network that receives clean input and provides an extra training signal.

Patch: A small rectangle of a spectrogram (for example 16×16 bins) that forms one token of a vision transformer.

PETL: Parameter-efficient transfer learning, which trains only a small number of added or selected parameters.

SSL: Self-supervised learning, learning from unlabeled data through a pretext task.

Target network: The slowly updated network whose outputs are the prediction targets in M2D and related methods.

ViT: Vision transformer, the transformer encoder applied to patch tokens.

AudioSet, FSD50K: General sound-event datasets; AudioSet is the pre-training corpus of M2D.

FMA, MSD: Free Music Archive and Million Song Dataset, two large music collections.

GTZAN, MTT, NSynth, OpenMIC-2018, MTG-Jamendo, Music4All: Music datasets for genre, tagging, instrument and pitch tasks (MTT is MagnaTagATune).

Benchmarks and tools: **MARBLE** and **HEAR** (evaluation benchmarks), **EVAR** and **X-ARES** (evaluation toolkits), **nnAudio** (GPU spectrogram library).

Model names: **M2D** (Masked Modeling Duo), **M2D-X** (M2D for application X), **M2D-S** (M2D for Speech), **MATPAC** (masked latent prediction and classification), **MERT** (acoustic music understanding model with large-scale self-supervised training), **PupuJEPA / PupuM2D** (a 2-D spectrogram music model, renamed from the first to the second name).

3.2 Detailed Literature Review

The review covers 36 research studies, datasets and benchmarks in full and five related applications and frameworks. Items were selected for their relevance to at least one of four questions: how masked-latent-prediction models such as M2D learn general audio representations and how well these transfer to music; what music-specific foundation models do differently in terms of targets, data and scale, and what they gain; what is known about further pre-training a pre-trained model on a small domain dataset,

including forgetting and regularization; and which public datasets and evaluation protocols allow such a study to be run and compared on a single GPU. Preprints that have not been peer reviewed are marked as such in the analysis.

3.2.1 General-Purpose Audio Self-Supervised Learning by Masked Prediction

Before latent prediction, general-purpose audio self-supervision was dominated by reconstruction of masked spectrogram patches and by prediction of discrete labels. The two studies below define what M2D improves on.

3.2.1.1 Masked Autoencoders that Listen

Summary

Huang et al. [1] extended masked autoencoders to audio spectrograms. About 80 % of the patches are masked, a ViT-Base encoder processes only the visible patches, and a decoder with local-window attention reconstructs the whole spectrogram. The model was pre-trained on AudioSet-2M for 32 epochs with a batch size of 512 on 64 V100 GPUs, which took about 36 hours. After fine-tuning the encoder, it reached an AudioSet-2M mAP of 47.3, 94.1 % accuracy on ESC-50 and 98.3 % on Speech Commands V2.

Critical analysis

Strengths:

- Simple design that scales well and reached state-of-the-art fine-tuning results without external supervised pre-training.
- The local attention in the decoder exploits the strong correlation of neighboring time-frequency bins.
- Code and weights are public, so the model is widely used as a baseline.

Weaknesses:

- The target is the spectrogram itself, so the learning signal depends partly on low-level detail; BEATs was proposed precisely because reconstruction loss is a weak driver of semantic abstraction [2].
- The headline results are obtained by fine-tuning the whole encoder on each target dataset, which says little about the quality of frozen features.

Relationship to the proposed work

Audio-MAE is the reconstruction baseline that M2D replaces by latent prediction, and the MAE-style masked spectrogram model MSM-MAE [3] from which M2D grew belongs to the same family. The code base of M2D is itself derived from the MAE implementation, and the AudioSet-based pre-training recipe is shared, which makes the M2D checkpoint a direct descendant of this design.

3.2.1.2 BEATs: Audio Pre-Training with Acoustic Tokenizers

Summary

Chen et al. [2] train an audio encoder to predict discrete labels of masked patches, where the labels come from an acoustic tokenizer. In the first iteration the tokenizer is a random projection; in later iterations a tokenizer is distilled from the previously trained (or fine-tuned) audio model, and the two are improved alternately. About 75 % of the input is masked. The authors report an AudioSet-2M mAP of 48.6 for a single model and 50.6 for an ensemble without external data, and 98.1 % accuracy on ESC-50.

Critical analysis

Strengths:

- Discrete, semantically rich targets give strong results with fewer parameters and less data than earlier models.
- Public checkpoints and a clear iterative recipe.

Weaknesses:

- Training is multi-stage: a tokenizer must be trained, and the strongest variants distill the tokenizer from a model that was fine-tuned on AudioSet labels, so they are not purely self-supervised.
- Tokenizer training and several pre-training rounds are far beyond a single-GPU budget.

Relationship to the proposed work

BEATs is the base model that SONAR continues to pre-train on FMA data (Section 3.2.5), and its discrete-target principle is shared by MERT, MusicFM and MuQ. This project keeps the continuous latent targets of M2D and therefore avoids training or distilling a tokenizer.

3.2.2 The Masked Modeling Duo Family

The M2D family is the direct basis of the project. The four studies below cover the base method, its specialization to speech by denoising distillation, its generalization into an application-agnostic framework, and its combination with text.

3.2.2.1 Masked Modeling Duo: Learning Representations by Encouraging Both Networks to Model the Input

Summary

Niizumi et al. [4] proposed Masked Modeling Duo (M2D). An online network encodes the visible patches and a predictor, given shared mask tokens, estimates the representations of the masked patches. The target network, an EMA of the online encoder, encodes *only the masked patches*, so that the target carries no information about the visible patches and both networks are pushed to model the input. The loss is the mean squared error between ℓ_2 -normalized prediction and standardized target. The released model uses a ViT-Base encoder on 80×608 log-mel inputs (6 s at 16 kHz) with 16×16 patches and a masking ratio of 0.7. It was pre-trained for 300 epochs on AudioSet (2,005,132 clips, 5,569 h) with an effective batch size of 2,048. The conference paper reports state-of-the-art linear-evaluation results on four of nine tasks (UrbanSound8K, VoxCeleb1, CREMA-D and GTZAN), and its comparison with the MAE-based predecessor MSM-MAE shows that predicting latents helps most tasks; in the journal extension [5], the model with masking ratio 0.7 reached an average of 80.2 over nine linear-evaluation tasks, against 79.1 for MSM-MAE, with 84.1 % on GTZAN.

Critical analysis

Strengths:

- Simple method with ablations of the masking ratio, target input and patch size; public code and weights.
- Encoding only the masked patches in the target network prevents shortcuts that make prediction too easy.
- The ViT-Base encoder (about 86 M parameters) is small enough for a single 16 GB GPU.

Weaknesses:

- Pre-training from scratch needs a large dataset and a multi-GPU budget, so the recipe cannot be repeated on a new domain by a small group.
- The learned features are abstract: compared with the MAE, the authors observe worse results on pitch classification (Surge) in linear evaluation and on VoxCeleb1 in fine-tuning, and conclude that M2D representations are less sensitive to pitch.
- Fixed 6-s input and random masking only.

Relationship to the proposed work

M2D is the pre-trained model of this study. The checkpoint M2D/0.7, its front-end, its loss and its random-masking setting are reused unchanged as the starting point and the base objective of the further pre-training (Chapter 5). The reported insensitivity to pitch is one reason for adding a pitch-oriented musical teacher.

3.2.2.2 Masked Modeling Duo for Speech: Specializing General-Purpose Audio Representation to Speech Using Denoising Distillation

Summary

Niizumi et al. [6] asked whether a general-purpose model can be specialized to a demanding domain, using speech as the example. They added an offline network to M2D and proposed the task of *denoising distillation*: the M2D networks receive speech mixed with AudioSet background noise, while the offline network, distilled from a HuBERT-style speech model that predicts clustered features, receives the clean speech. M2D-S learns this task jointly with the M2D masked prediction task. On the SUPERB benchmark it performed comparably to state-of-the-art speech models (HuBERT, WavLM) and outperformed them on keyword spotting and emotion recognition. Ablations showed that distillation alone is better than M2D alone, that adding denoising helps further, and that learning all tasks together works best. The study also showed a trade-off: speech tasks improved with more speech data and fewer general sounds, while the non-speech tasks degraded.

Critical analysis

Strengths:

- First demonstration that a general-audio model can be specialized to a competitive domain by adding a domain teacher and a denoising task.
- Reports the trade-off between specialization and generality, which is the question that matters for retention.

Weaknesses:

- Requires a strong domain model trained with clustering (HuBERT) as teacher, and changes the patch size and input duration to speech settings.
- Remains behind WavLM on speaker identification (80.69 % against 84.51 % in the SUPERB comparison) and behind HuBERT and WavLM on phoneme recognition and intent classification.
- Pre-training uses a large speech corpus (960 h of LibriSpeech), not a small one.

Relationship to the proposed work

This is where denoising distillation originates. The proposed framework applies the same pattern to music: noisy input to the online and target networks, clean input to the offline branch. The domain teacher is the frozen original M2D for regularization and the CQT, a deterministic music-specific teacher, for the musical task; distilling a frozen music model such as MERT is left as an optional extension.

3.2.2.3 Masked Modeling Duo: Towards a Universal Audio Pre-training Framework (M2D-X)

Summary

In the journal extension, Niizumi et al. [5] redefine M2D as a framework for learning representations specialized to an application X , including cases with little data. M2D-X adds background noise to the input, which acts as augmentation and creates a denoising task, and an offline network with a configurable additional task, namely supervised learning of labels, distillation of a domain model, or regularization with the original pre-trained model. The total loss is the weighted sum $\lambda_{\text{m2d}}\mathcal{L}_{\text{m2d}} + \lambda_{\text{off}}\mathcal{L}_{\text{off}}$. The framework was validated on AudioSet with supervised labels (M2D-AS, average 80.6 in linear evaluation), on speech, and on respiratory sounds. In the last case, the AudioSet model was further pre-trained on 539 ICBHI 2017 recordings: continuing with the M2D loss alone and no noise lowered the score from 61.93 to 61.17, background noise at a ratio of 0.3 raised it to 62.73, and the full M2D-X with the frozen original model as offline teacher reached 63.29. The base model for this experiment was the variant with 16×4 patches and 2-s inputs. The run used 600 epochs of 10,000 virtual samples with an effective batch size of 128 on a single 24 GB GPU (an RTX 3090 Ti), with the stated aim of completing pre-training within a day. For the AudioSet setting, the authors state that M2D-X can be pre-trained within 2.5 days on four A100 GPUs.

Critical analysis

Strengths:

- The main published recipe for further pre-training of a masked-latent-prediction audio model on a very small dataset, with code, weights and a practical guide.
- Careful ablations that separate the effect of noise from that of the offline regularizer.
- Runs on a single consumer GPU.

Weaknesses:

- The further pre-training result rests on one small medical dataset (with a similar trend on a second respiratory dataset, SPRSound), and its gains of one to two points are of the size of the variation between runs.

- No music data and no measurement of what happens to general-audio ability after further pre-training.
- The noise ratio and loss weights are chosen per application and not studied systematically.

Relationship to the proposed work

M2D-X is the methodological foundation of Chapter 5. The proposed framework keeps its noise injection, offline network and frozen-teacher regularization, adds a music-specific teacher, and tests whether the recipe carries over from a respiratory database of 5.5 h to about 83 h of music while measuring retention explicitly.

3.2.2.4 M2D-CLAP: Masked Modeling Duo Meets CLAP for Learning General-Purpose Audio-Language Representation

Summary

Niizumi et al. [7] combine M2D with contrastive language-audio pre-training so that one representation serves both transfer learning and zero-shot classification. The 2024 version reached a zero-shot GTZAN accuracy of 75.17 %. The extended version [8] uses LLM-based sentence embeddings and a multi-stage procedure in which the first stage learns M2D and CLAP objectives jointly and later stages refine the CLAP features; it reports an AudioSet mAP of 49.0 and state-of-the-art results on music tasks. The official repository recommends the 2025 M2D-CLAP checkpoint (80×1001 input, i.e., 10 s).

Critical analysis

Strengths:

- Retains the strong transfer quality of M2D while adding language alignment.
- Strong music results from a general-audio model.

Weaknesses:

- Needs audio-caption pairs and an LLM-based text encoder in addition to audio.
- Longer 10-s input and a more complex training pipeline; the extended 2025 version is an arXiv preprint.

Relationship to the proposed work

M2D-CLAP shows that general-audio models of the M2D family are already state of the art on music tasks, which supports the premise of this study. It is a candidate alternative starting checkpoint, but it was not chosen because the M2D-X recipe is defined for the plain M2D loss, and the CLAP head would be discarded.

3.2.3 Other Latent-Prediction Models: JEPA and Classification-Based Variants

Other models also predict latent representations of masked audio, but differ from M2D in which network sees which patches, in the masking strategy, and in whether a classification objective is added. They show which design choices are shared and which are open.

3.2.3.1 Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA)

Summary

Assran et al. [9] proposed I-JEPA for images: from a single context block, the network predicts the representations of several target blocks of the same image, with a slowly updated target encoder providing the targets. The authors find that the masking strategy is decisive, namely sampling target blocks of sufficiently large scale and using an informative, spatially distributed context block. No hand-crafted view augmentations are used. A ViT-Huge/14 was trained on ImageNet with 16 A100 GPUs in under 72 hours and transferred to classification, counting and depth prediction.

Critical analysis

Strengths:

- Predicting in latent space avoids pixel-level detail without needing image-specific augmentations.
- Scalable and computationally cheaper than earlier invariance-based methods.

Weaknesses:

- Vision only; the block masking and the target encoder that sees the full image do not transfer directly to spectrograms, as the audio studies below show.

Relationship to the proposed work

I-JEPA is the conceptual root of M2D, MATPAC, Audio-JEPA and PupuM2D. The project uses the M2D variant of the idea, in which the target encoder sees only the masked patches, because the audio studies below find that exposing the target encoder to the full input leads to shortcuts or collapse.

3.2.3.2 Investigating Design Choices in Joint-Embedding Predictive Architectures for General Audio Representation Learning

Summary

Riou et al. [10] studied JEPA-style pre-training of a ViT-Base on AudioSet spectrograms and evaluated on environmental, speech and music tasks. They compared unstructured, multi-block and time masking, and compared masking the target in the audio domain with masking it only in the latent domain. Unstructured random masking outperformed the alternatives on nearly all tasks, in contrast to images, and masking the target spectrogram before the target encoder was, in most tasks, better than feeding it the whole input. The duration of the training audio mattered: with 2.1 s, 3.2 s and 6.4 s inputs, GTZAN accuracy rose from 82.1 to 84.3 and 86.9 while NSynth accuracy fell from 76.8 to 75.6 and 74.9.

Critical analysis

Strengths:

- Systematic ablation of the design choices that matter for audio, evaluated with a common toolkit on eight tasks.
- Open code and pre-trained models.

Weaknesses:

- Only AudioSet pre-training and one backbone size; no music-specific data or objective.
- Exploration limited to a few masking strategies and three durations.

Relationship to the proposed work

The findings justify keeping random masking and masked-only target encoding in the framework, and they show that a longer input favors genre-like (global) tasks while shorter inputs favor note-level tasks such as instrument and pitch. The 6.08-s window of M2D therefore suits genre tasks, and its effect on pitch tasks is one of the questions that the NSynth pitch evaluation will probe.

3.2.3.3 Audio-JEPA: Joint-Embedding Predictive Architecture for Audio Representation Learning

Summary

Tuncay et al. [11] translated I-JEPA to audio with a ViT-Base (16×16 patches) and random patch masking of 40–60 %. They pre-trained on 1,921,982 AudioSet clips at 32 kHz (5,338 h) for 100,000 steps, about 13 epochs, which took 14 hours on four V100 GPUs, and evaluated with the 21-dataset X-ARES suite. With an MLP probe the model was first or second on some tasks and last on about half, with particular weakness on speech commands; with a k-nearest-neighbor probe it ranked first on ESC-50, FMA-small and GTZAN among three models, ahead of wav2vec 2.0 and data2vec despite using under a fifth of their training data.

Critical analysis

Strengths:

- Very small training cost and open code and checkpoints.
- Includes FMA-small and GTZAN in a standardized suite.

Weaknesses:

- Short training (13 epochs), no hyper-parameter tuning, and only two baselines (wav2vec 2.0 and data2vec); it is an arXiv preprint written for a challenge.
- The target encoder sees the whole spectrogram, and the authors note that the embedding space is not guaranteed to be linearly separable, which hurts linear probing.

Relationship to the proposed work

The study illustrates the cost of not following the M2D rule of masking the target input and of weak linear separability of JEPA features. It is also an example of the X-ARES protocol that includes FMA-small, which this project uses as its in-domain genre task.

3.2.3.4 Masked Latent Prediction and Classification for Self-Supervised Audio Representation Learning (MATPAC)

Summary

Queleennec et al. [12] combine masked latent prediction with an unsupervised classification task. The predicted and the target latents of the masked patches are each projected by a DINO-style head [13] (three fully connected layers with a 256-dimensional bottleneck and a weight-normalized layer to $K = 2048$ prototypes) into probability distributions that are matched by cross-entropy, with centering and sharpening to prevent collapse. The total loss is $(1 - \alpha)\mathcal{L}_{\text{cls}} + \alpha\mathcal{L}_{\text{pred}}$ with $\alpha = 0.5$. Pre-training follows M2D (ViT-Base, 6-s inputs of AudioSet, 300 epochs, batch size 2,048). In linear probing over seven tasks the average was 74.9, against 74.0 without the classification task and 73.7 for M2D, and the model reached 41.1 mAP on MagnaTagATune, above the supervised and self-supervised methods compared.

Critical analysis

Strengths:

- Improves the latent space at no inference cost, with an ablation of the loss weight and the number of prototypes.
- Public code and checkpoints.

Weaknesses:

- The authors conclude that the classification task mainly acts as a regularizer, and without the prediction task the model collapses.
- Differences from M2D are small and depend on the AudioSet version available to the authors, because part of AudioSet is no longer downloadable.

Relationship to the proposed work

MATPAC is the closest competitor to M2D within masked latent prediction. Its design shows that an extra objective on top of the prediction loss can improve the representation, which parallels the extra offline objectives of this project, but the classification head and prototypes are not adopted because of their added hyper-parameters.

3.2.3.5 MATPAC++: Enhanced Masked Latent Prediction for Self-Supervised Audio Representation Learning

Summary

Queleennec et al. [14] address the ambiguity of predicting masked content with multiple choice learning: the predictor produces r hypotheses for each masked patch and the best one, selected with an annealed temperature, is trained and passed to the classification task. With a unified, statistically controlled protocol (same data splits, 95 % confidence intervals, Wilcoxon tests), MATPAC++ obtained an average linear-probing score of 75.7 over seven tasks, against 75.2 for MATPAC and 74.3 for M2D. When trained only on music (about 0.9 M Million Song Dataset samples and 0.18 M private web-radio samples), its average over eight music tasks rose from 47.2 to 47.9, a gain of only 0.7 points over the AudioSet-trained model. In the same table the AudioSet-trained M2D averaged 46.3 against 44.2 for MERT, whose scores were taken from its paper. Pre-training on 6-s segments took about 30 hours on four H100 GPUs, and the encoder has 86 M parameters.

Critical analysis

Strengths:

- The most careful comparison available of general-audio against music-only pre-training under one protocol.
- Checkpoints for both a general and a music-trained encoder are public.

Weaknesses:

- An arXiv preprint under review; the web-radio data are private.
- Additional hypotheses increase the predictor size, and the benefit is smaller on structured music than on diverse audio; the MERT numbers come from a different protocol.

Relationship to the proposed work

This is the key motivating observation of the project: general-audio models are already strong on music, and large-scale music-only training adds little. The question this project asks is whether a small, cheap, music-specific step can recover a comparable increment. The MATPAC++ checkpoints, general and music, serve as reference models in the experimental design.

3.2.3.6 PupuJEPA / PupuM2D: Frequency-Aware Self-Supervised Music Representation Learning

Summary

Gu et al. [15] propose a masked-latent-prediction model trained directly on 2-D music spectrograms. It was first called PupuJEPA and renamed PupuM2D in the third version of the preprint, because the target encoder, as in M2D, receives only the masked patches. Domain-specific changes include 4×16 patches that give a 25 Hz frame rate, rotary position embeddings, SwiGLU feed-forward layers, QK-normalization, a smoothed-L1 loss, a mixture of random, blockwise and time-frequency masking (overall ratio 0.8) whose proportions follow a curriculum, and the removal of DropPath, LayerScale and batch normalization, which caused collapse. It was trained on about 100,000 hours of in-house music (24 kHz, 128 mel bands, 10.24-s crops) for 500,000 steps with a batch size of 2,048 on 32 B200 GPUs, in five sizes from 5 M to 632 M parameters. On the MARBLEv2 benchmark with linear probing, PupuM2D-Large reached 66.1 on GiantSteps key detection, 86.9 on GTZAN genre and 40.8 AP on MagnaTagATune. In the same tables, AudioSet-trained checkpoints evaluated by the authors were at or above the music-specific MERT, MuQ and MusicFM on tagging and instrument classification: for example, MagnaTagATune AP was 40.9 for M2D, 40.9 for an M2D-X checkpoint and 41.7 for MATPAC++, against 37.5 to 38.5 for MERT, MuQ and MusicFM, and the MTG-Jamendo instrument AP was 21.1 for M2D and MATPAC++ against 18.5 to 19.7 for the music models. MATPAC++ also matched or exceeded PupuM2D-Large on genre and tagging.

Critical analysis

Strengths:

- Evaluates M2D, M2D-X and MATPAC++ checkpoints next to music-specific models in one environment, and re-trains three audio baselines on the authors' music data under identical settings.
- Thorough ablations of architecture, masking and inference paradigm, and attention maps that the authors present as showing musically meaningful patterns.

Weaknesses:

- A preprint, not yet peer reviewed, trained on a private corpus of 100,000 hours with a 32-GPU budget, so it cannot be reproduced or compared at equal cost.
- The scores of the 2-D models are the best over several swept inference paradigms, and MARBLEv2 scores are not strictly comparable with those of MARBLE v1.
- The advantage over strong general-audio models is clear mainly for valence in emotion recognition; for tagging, genre and instrument classification it is small or absent, and key detection and beat tracking could not be compared because the frame rate of the general models is too low.

Relationship to the proposed work

This is the most recent evidence for the central question. It shows that the M2D design also works for music-specific pre-training, and it quantifies how little a 100,000-hour in-domain model adds over AudioSet-trained models on tagging, genre and instrument tasks, while frame-level tasks such as key detection and beat tracking, where the general models could not be compared, are where a music-specific model is most useful. These are the tasks that the musical teacher of MERT targets. The study also warns about the cost of fine temporal resolution: the proposed work keeps 16×16 patches (160 ms per step) and therefore excludes frame-level tasks from its evaluation.

3.2.4 Music-Specific Self-Supervised Models

Music-specific models differ from general-audio models in the data they use, in their targets and, usually, in scale. The first three studies below represent the early generative, contrastive and mixed approaches; the last three are the masked-prediction foundation models used as references.

3.2.4.1 Codified Audio Language Modeling Learns Useful Representations for Music Information Retrieval (JukeMIR)

Summary

Castellon et al. [16] used the hidden states of Jukebox, a language model trained on codified (discretely encoded) audio from about one million songs, as input features for shallow probes. Averaged over four tasks (tagging, genre classification, key detection and emotion recognition), the representations were 30 % stronger than those of models pre-trained on tag prediction, with a particularly large advantage for key detection, and competitive with purpose-built systems on several tasks.

Critical analysis

Strengths:

- First clear demonstration that self-supervised pre-training on music audio gives better representations than tag-based supervised pre-training.
- Shows that tag-pretrained models have blind spots, for example for key.

Weaknesses:

- Extracting features is expensive because Jukebox has billions of parameters; the MERT authors report that inference on a large dataset takes weeks on a consumer GPU [17].
- Only four tasks and a generative model that was not designed as an encoder.

Relationship to the proposed work

The result that key detection benefits from modeling music audio, not tags, is the reason for adding a pitch-oriented teacher in this project. The cost of huge models is also the reason for starting from an 86 M-parameter encoder.

3.2.4.2 Contrastive Learning of Musical Representations (CLMR)**Summary**

Spijkervet and Burgoyne [18] applied SimCLR to raw-waveform music with a chain of audio augmentations. A linear classifier on the learned representations reached a higher average precision than supervised models on MagnaTagATune and comparable performance on the Million Song Dataset, transferred across datasets, and, with only 259 labeled songs (1 % of MagnaTagATune), still obtained an average precision of 33.1 %.

Critical analysis

Strengths:

- Label-efficient and based on small models, with released code.
- An ablation of music-specific augmentations.

Weaknesses:

- Contrastive learning depends on a chain of augmentations, which the authors of PupuM2D consider less natural for complete songs [15].
- Evaluated on music tagging only.

Relationship to the proposed work

CLMR is an early example of data-efficient music self-supervision. The proposed work avoids augmentation design altogether: its only data augmentation is background noise, which the M2D-X recipe already provides.

3.2.4.3 Supervised and Unsupervised Learning of Audio Representations for Music Understanding (MULE)**Summary**

McCallum et al. [19] compared pre-training on music with pre-training on general audio (a version of AudioSet), and supervised with unsupervised (SimCLR-style) training, on tasks such as genre, era, mood, instrumentation, key and tempo. Supervised models on a large expert-annotated music catalog (about 1.8 M pieces, 117,497 hours) reached state-of-the-art results with under 100 M parameters but specialized to their labels. Among the unsupervised models, restricting the domain of the pre-training data to music allowed training with smaller batch sizes while achieving the best unsupervised results; the unsupervised models were trained on 16 A100 GPUs in about 80 hours.

Critical analysis

Strengths:

- A controlled comparison of the effect of the pre-training domain under one method.
- Pays attention to compute: models of fewer than 100 M parameters that need no fine-tuning.

Weaknesses:

- The large music dataset is not publicly available, so the central result cannot be reproduced.
- Contrastive training with large batches; no masked latent prediction.

Relationship to the proposed work

MULE gives direct evidence that the domain of the pre-training data matters, which the present study tests for a masked-latent-prediction model by adding a small amount of in-domain data rather than by replacing the data.

3.2.4.4 MERT: Acoustic Music Understanding Model with Large-Scale Self-Supervised Training

Summary

Li et al. [17] train a transformer with HuBERT-style masked prediction on raw audio and two kinds of targets. The *acoustic teacher* gives discrete labels, either from K-means on log-mel and chroma features or from the residual quantization codes of EnCodec. The *musical teacher* is the constant-Q transform: a linear head regresses the CQT (84 bins, 12 per octave, from 32.7 Hz) of the audio with a mean-squared-error loss, and the total loss is $\alpha \mathcal{L}_H + \mathcal{L}_{\text{CQT}}$. Short excerpts from the same batch are mixed into each clip (in-batch noise mixup, probability 0.5). Models of 95 M parameters were trained on about 1,000 hours, and a 330 M-parameter model on 160,000 hours, with 5-s contexts. On 14 tasks evaluated by probing, MERT achieved the best overall score. Adding the CQT loss raised key-detection accuracy from 55.1 to 65.0 and GTZAN genre accuracy from 75.2 to 78.6 for the K-means variant, and from 60.5 to 63.2 and from 72.8 to 77.2 for the RVQ variant.

Critical analysis

Strengths:

- A clear mechanism, the CQT musical teacher, that injects pitch and harmonic bias and is validated by ablation.
- Competitive models of only 95 M parameters, and public checkpoints; the version trained on open data (MERT-v0-public, about 900 hours) shows that the recipe works with limited data.

Weaknesses:

- Training at 330 M parameters was unstable and needed special normalization and attention relaxation, and the contexts are limited to 5 s.
- The strongest models rely on a very large collection of music, and the released weights are licensed for non-commercial use only [20].

Relationship to the proposed work

MERT supplies the musical teacher: the framework adds a CQT-regression task with the same bin layout and a unit weight. MERT-v1-95M and MERT-v0-public are reference models, one with 20,000 hours and one with about 900 hours of training data. Its in-batch noise mixup corresponds to the background-noise mixing of M2D-X.

3.2.4.5 A Foundation Model for Music Informatics (MusicFM)

Summary

Won et al. [21] adapt BEST-RQ [22] to music: a frozen random projection and a random codebook turn the normalized log-mel features into discrete tokens, and a 12-layer Conformer is trained to predict the tokens of masked segments (400-ms windows with probability 0.6). They compared model sizes, token rates, input durations and data, and evaluated on token-level and sequence-level tasks such as beat tracking, chord and structure analysis, key detection and tagging. Longer (30-s) inputs improved tasks that need long-term context. A model trained on the whole Free Music Archive (about 8,000 hours) scored below the model trained on 160,000 hours of in-house data on most tasks, which the authors attribute to scale and to the crowd-sourced nature of FMA.

Critical analysis

Strengths:

- No external tokenizer is needed, and both an MSD and an FMA checkpoint are public.
- Evaluation covers local (token-level) as well as global tasks.

Weaknesses:

- Random targets carry no musical prior, and the model has about 330 M parameters and needed about two weeks of GPU training.
- Training on FMA alone gave clearly weaker results than training on a larger, curated corpus.

Relationship to the proposed work

MusicFM shows what training from scratch on the whole FMA achieves, and thereby sets a ceiling for the data source used here. Its FMA checkpoint is a reference model, and the observation that FMA is a weaker pre-training source than a larger private corpus is a caution that applies to this project.

3.2.4.6 MuQ: Self-Supervised Music Representation Learning with Mel Residual Vector Quantization

Summary

Zhu et al. [23] train a 12-layer Conformer to predict tokens from a Mel residual vector quantizer. The quantizer, called Mel-RVQ, has a single linear layer as encoder and decoder and a residual codebook structure, is trained on the mel spectrum, and replaces both the random projection of BEST-RQ and the large codec of MERT. Pre-trained on only 0.9K hours of the open Music4All dataset, MuQ already outperformed MERT and MusicFM on the MARBLE benchmark (average 75.8 against 74.4 and 75.4); with 160K hours of in-house data it reached 76.7, and with iterative refinement 77.0. A music-text variant, MuQ-MuLan, achieved a state-of-the-art zero-shot tagging score on MagnaTagATune.

Critical analysis

Strengths:

- Shows that a good target definition matters more than data volume: 900 hours were enough to beat larger models.
- The quantizer is tiny and trainable on one GPU, and the code and checkpoints are open.

Weaknesses:

- The best models still use 160,000 hours of private data and 32 A100 GPUs.
- Evaluation is by linear probing on a limited set of tasks, and the tokenizer needs a separate training stage.

Relationship to the proposed work

MuQ provides further evidence that a small amount of music can suffice if the learning signal is well chosen, which is the premise of the musical teacher. It is cited as an alternative to CQT targets, and its checkpoint is a candidate reference model.

3.2.5 Further Pre-training, Forgetting and Adaptation

These studies examine what happens when a model that is already pre-trained sees more data from a domain, how this can fail, and which cheaper alternatives exist. They come from language, speech, general audio, respiratory sound and music, because no study applies further pre-training of a masked-latent-prediction model to music.

3.2.5.1 Don't Stop Pretraining: Adapt Language Models to Domains and Tasks

Summary

Gururangan et al. [24] asked whether it is still useful to tailor a broadly pre-trained language model (RoBERTa-base) to the domain of a target task. Across four domains (biomedical and computer science publications, news and reviews) and eight classification tasks, a second phase of in-domain pre-training (domain-adaptive pre-training) improved results in both high- and low-resource settings, and a further phase on the task's own unlabeled data (task-adaptive pre-training) added a gain even after the domain phase. Each domain phase was a single pass of 12.5K steps on one TPU, and adapting to a task corpus augmented by simple data selection was an effective alternative.

Critical analysis

Strengths:

- Broad evidence that continuing self-supervised training is cheap and effective when the new data are relevant.
- Shows that adaptation to an irrelevant domain does not help and can lower performance, which establishes relevance as the condition for success.

Weaknesses:

- Text only, with a masked-language-model objective and fine-tuning for evaluation.
- Focuses on gains on the target tasks; the effect on the broad abilities of the original model is not systematically measured.

Relationship to the proposed work

This study is the conceptual precedent for further pre-training on music. The finding that relevance of the extra data decides success is why a music corpus is used for a general-audio model, and the missing measurement of lost general ability is why this project adds retention tasks.

3.2.5.2 Robust wav2vec 2.0: Analyzing Domain Shift in Self-Supervised Pre-Training

Summary

Hsu et al. [25] studied what happens when the unlabeled pre-training data, the labeled fine-tuning data and the test data come from different domains. Using target-domain data during pre-training led to large improvements, and in a large-scale setup pre-training on unlabeled in-domain data reduced the gap between models trained on in-domain and on out-of-domain labeled data by 66 % to 73 %. Pre-training on several domains also improved generalization to domains that were not seen, and continual pre-training (first one domain, then another) performed similarly to joint training on the combined data when more in-domain data were added.

Critical analysis

Strengths:

- Large-scale, systematic analysis of data-domain mismatch in self-supervised speech models.
- Practical message: unlabeled target-domain data are much easier to obtain than labeled ones.

Weaknesses:

- Speech recognition with a contrastive objective and corpora of hundreds to thousands of hours, far from the 83 hours used here.
- Measures accuracy on the target and unseen domains but not the loss of ability on the original domain.

Relationship to the proposed work

It supports the premise that a small amount of unlabeled in-domain audio can close part of a domain gap, and the benefit of multi-domain pre-training agrees with the choice to keep general-audio data (the background noise) in the training mix.

3.2.5.3 Self-Distillation for Further Pre-training of Transformers

Summary

Lee et al. [26] observed that further pre-training of vision transformers on small target data overfits. Their remedy first further pre-trains the initial model on the target unlabeled data to obtain a teacher, then takes the same initial model as a student and trains it with the masked-autoencoding objective while pulling its hidden representations towards those of the teacher. The method outperformed the baselines on several image and text classification benchmarks.

Critical analysis

Strengths:

- Directly targets overfitting during further pre-training by regularizing in representation space.
- Applies to both vision and language transformers.

Weaknesses:

- Requires two rounds of further pre-training (teacher, then student), which doubles the cost.
- Image and text benchmarks only; the teacher is itself a further pre-trained model and may therefore carry the overfitting that the method is meant to remove.

Relationship to the proposed work

M2D-X takes the opposite choice of teacher: the frozen *original* model, which costs no extra training round and cannot overfit the new data. The two approaches share the idea that representation-level regularization is the right remedy.

3.2.5.4 SONAR: Self-Distilled Continual Pre-training for Domain Adaptive Audio Representation**Summary**

Zhang et al. [27] continue pre-training BEATs on unlabeled data from four domains (speech emotion, music from FMA-Large, bioacoustics and environmental sound), with 30,000 to 50,000 audio segments per domain. The framework combines task-relevant stratified sampling from a general pool and the domain data, dual-source self-distillation that constrains both the tokenizer and the encoder to stay close to the frozen original, and an online-clustered codebook that adapts the tokenizer to new acoustic patterns. For music, direct continual pre-training that resumes the original objective lowered the frozen-feature GTZAN F1 score from 86.0 to 76.0 and the AudioSet tagging mAP from 34.8 to 14.7, a forgetting rate of 57.8 %; SONAR raised GTZAN to 88.0 and kept the mAP at 34.7.

Critical analysis

Strengths:

- The first audio study to quantify forgetting after domain adaptation, with a music (FMA) case and a regularized method.
- Distillation towards the frozen original model gives near-zero forgetting.

Weaknesses:

- An arXiv preprint; built on the discrete-token model BEATs, with tokenizer-specific components that do not apply to M2D.
- The regularization weight is very large (10^6) and tuned once; one downstream task per domain, so music is judged by GTZAN alone.

Relationship to the proposed work

SONAR is the closest prior study: it uses FMA, unlabeled data, a small adaptation set and a frozen-original regularizer. This project differs in the base model (M2D, continuous targets), in adding a musical teacher, in the single-GPU budget, and in a larger and more varied set of music and retention tasks. Its forgetting numbers also confirm that naive continuation must be treated as a control that may fail.

3.2.5.5 Towards Pre-training an Effective Respiratory Audio Foundation Model

Summary

Niizumi et al. [28] compared 21 audio foundation models on seven tasks of the OPERA respiratory benchmark with linear evaluation. Models pre-trained on AudioSet, a general and diverse corpus, were more effective than models pre-trained only on respiratory sounds: eight of 19 general models outperformed the respiratory model OPERA-CT. Further pre-training M2D on a combination of AudioSet and respiratory data (14,127 respiratory samples) improved the average AUROC from 0.782 for plain M2D to 0.814, above the 0.733 of OPERA-CT. The authors also found that preserving frequency-wise information when aggregating features is important.

Critical analysis

Strengths:

- Uses M2D itself and the same linear-evaluation toolkit as this project.
- Shows that diverse general data and narrow domain data complement each other.

Weaknesses:

- The best configuration mixes AudioSet with the domain data, and the domain (coughs and breath sounds) is narrow and monotonous.
- Seven small tasks (six binary and one five-class) with wide variability; no music.

Relationship to the proposed work

The study supports the central premise, that a general-audio model is a strong starting point that often beats domain-only training, and suggests that keeping some general data in the mix helps. Because AudioSet cannot be re-downloaded reliably, this project keeps general sounds in the mix through the FSD50K background noise and preserves frequency-wise information by using the 3,840-dimensional frame feature layout of M2D.

3.2.5.6 Parameter-Efficient Transfer Learning for Music Foundation Models**Summary**

Ding and Lerch [29] study how music foundation models are adapted to downstream tasks. Probing leaves the weights frozen and may be suboptimal, whereas full fine-tuning is expensive and prone to overfitting on small data. They compare adapter-based, prompt-based and reparameterization-based parameter-efficient methods, which train only a small number of parameters. These methods outperformed both probing and fine-tuning on music auto-tagging and matched fine-tuning on key detection and tempo estimation at a much lower cost. The authors also note that a small model trained from scratch achieves similar key and tempo results, which questions the usefulness of the current foundation models for these tasks.

Critical analysis**Strengths:**

- Evaluates several adaptation methods on music foundation models with attention to training cost.
- Honest negative finding about key and tempo.

Weaknesses:

- An arXiv preprint; adaptation is supervised, so it needs labels for each task.
- It adapts models to tasks, not to a data domain, and does not measure forgetting.

Relationship to the proposed work

This is the main alternative to further pre-training: instead of updating the whole encoder with unlabeled music, add a few trainable parameters per task. The proposed work keeps the features frozen at evaluation time and updates them once, with unlabeled data, which makes the result reusable for every downstream task. Parameter-efficient adaptation of the further pre-trained encoder is a natural extension.

3.2.6 Public Datasets

The datasets below determine what can be pre-trained and evaluated without proprietary data. The first two are general-audio corpora, the next seven are music corpora used for pre-training or evaluation, and the last is a music dataset derived from FMA.

3.2.6.1 AudioSet**Summary**

AudioSet [30] is an ontology of audio events and a collection of human-labeled 10-s segments from YouTube videos. Its project page lists 2,084,320 labeled clips, about 5.8 thousand hours, and an ontology of 632 event classes, of which 527 are annotated with clips. M2D was pre-trained on the 2,005,132 clips that were still downloadable, which amount to 5,569 hours [5].

Critical analysis**Strengths:**

- The largest labeled general-audio collection and the standard pre-training corpus for audio self-supervision.
- Contains a substantial amount of music.

Weaknesses:

- Only links to online videos are distributed, so clips disappear and every group trains on a different subset, which makes published numbers hard to compare [14].
- Audio quality and labels are web-video quality, and the music portion is not curated.

Relationship to the proposed work

AudioSet is the source of the general-audio knowledge in the M2D checkpoint. It is not downloaded in this project, which relies on the released weights.

3.2.6.2 FSD50K: An Open Dataset of Human-Labeled Sound Events

Summary

Fonseca et al. [31] released FSD50K, 51,197 Freesound clips (108.3 hours) labeled with 200 classes of the AudioSet ontology. The clips are divided into a development and an evaluation split without common uploaders, the evaluation set is exhaustively labeled, and all clips are under Creative Commons licenses so that the waveforms can be freely redistributed.

Critical analysis

Strengths:

- Openly licensed audio with ontology-based labels.
- Already used as the background-noise corpus in the M2D-X recipe.

Weaknesses:

- Moderate size and short, variable-length clips.
- Contains musical-instrument classes, so it is not purely non-musical noise.

Relationship to the proposed work

The development portion is the noise corpus of the denoising task. An optional ablation removes music-related classes to test whether purely non-musical noise is a better distractor.

3.2.6.3 FMA: A Dataset for Music Analysis

Summary

Defferrard et al. [32] released the Free Music Archive dataset: 106,574 tracks by 16,341 artists in a hierarchy of 161 genres, with full-length audio of the order of 900 GiB (343 days) under Creative Commons licenses, plus metadata and pre-computed features. Subsets are provided as 30-s clips cut from the middle of the tracks: small (8,000 clips, 8 balanced genres), medium (25,000 clips, 16 genres, selected for metadata completeness and popularity) and large (all tracks). An 80/10/10 split into training, validation and test sets is stratified by genre and filtered by artist so that no artist appears in two splits. The tracks are MP3 files, mostly stereo at 44.1 kHz and 320 kbit/s.

Critical analysis

Strengths:

- Redistributable full-quality audio with artist-level metadata and leakage-controlled splits, which is rare among music datasets.
- Several sizes, from a GTZAN-like subset to the whole archive.

Weaknesses:

- Genre labels are provided by the artists and are noisy; some genres, such as experimental music, are under-represented in the medium subset.
- Documented issues include corrupted or truncated tracks, tracks shorter than 30 s and duplicates [33], and models trained only on FMA train models trained on a larger curated corpus [21].

Relationship to the proposed work

FMA is the further pre-training corpus: 10,000 tracks of the medium subset, taken from the training split only. Its FMA-small test split is the in-domain genre evaluation. The sampling, audit and conversion are specified in Chapter 5.

3.2.6.4 The Million Song Dataset**Summary**

Bertin-Mahieux et al. [34] released the Million Song Dataset (MSD), a collection of one million contemporary popular music tracks by 44,745 artists, described by metadata and pre-computed audio features [32]. It does not distribute audio.

Critical analysis

Strengths:

- Scale and a long record of use, with rich metadata and tag annotations.

Weaknesses:

- No audio, so models that train on MSD audio must obtain 30-s excerpts from third parties [14, 21], and the source and quality of those excerpts vary.
- Mostly contemporary popular music.

Relationship to the proposed work

MSD is the pre-training source of the MusicFM checkpoint and of part of the MATPAC++ music model, which are used as references. It is not used for pre-training here because it does not provide audio.

3.2.6.5 Music4All: A New Music Database and Its Applications**Summary**

Santana et al. [35] built Music4All, a database of 109,269 pieces with 30-s audio excerpts, about 910 hours in total, together with song metadata. An extension (Music4All-Onion) adds 26 audio, video and metadata features and listening records of 119,140 users. Music4All is the corpus on which MERT-v0-public (together with part of FMA) and the small-data version of MuQ were trained.

Critical analysis

Strengths:

- A moderately sized, quality-controlled corpus of music excerpts, which is the main open alternative to FMA.
- Rich side information.

Weaknesses:

- Mostly popular music with limited stylistic diversity, as noted by the MERT authors.
- Limited to 30-s excerpts, so the structure of complete tracks cannot be learned.

Relationship to the proposed work

Music4All is the data behind the small-data music-specific references. Its scale of about 910 hours is the natural comparison for the 83 hours used here. It is not used for pre-training because FMA offers artist-filtered splits and open licenses.

3.2.6.6 The MTG-Jamendo Dataset for Automatic Music Tagging**Summary**

Bogdanov et al. [36] released MTG-Jamendo, over 55,000 full-length audio tracks from the Jamendo platform with 195 tags: 87 genre, 40 instrument and 56 mood and theme tags. Audio is available as 320 kbit/s MP3 (508 GB) or as a lower-bitrate mono version (156 GB), the audio is under Creative Commons licenses and the annotations under CC BY-NC-SA 4.0, for non-commercial research. Official training, validation and test splits are provided.

Critical analysis

Strengths:

- Large, multi-label and well-defined tasks (genre, instrument, mood, top-50) that are part of MAR-BLE and of recent music benchmarks.

Weaknesses:

- A very large download (508 GB at full quality) and a license restricted to non-commercial research.

Relationship to the proposed work

MTG-Jamendo is used by MATPAC++ and PupuM2D for evaluation. The download and probing cost exceed the budget of this project, so it is listed as an optional extension to broaden the evaluation.

3.2.6.7 Evaluation of Algorithms Using Games: The Case of Music Tagging (MagnaTagATune)**Summary**

Law et al. [37] introduced the MagnaTagATune data, in which tags were collected from players of the TagATune game. The data consist of 25,863 clips cut from over 5,000 songs, distributed as 16 kHz, 32 kbit/s mono MP3 files [32], and the standard task is tagging with the 50 most frequent tags.

Critical analysis

Strengths:

- Widely used, so results can be compared with MERT, MuQ and MATPAC.

Weaknesses:

- Low audio quality (16 kHz mono), and several clips per song (about five on average), so a split must keep songs together to avoid leakage.

Relationship to the proposed work

MagnaTagATune is the tagging task of the evaluation suite. A fixed published split is used and recorded before any evaluation.

3.2.6.8 GTZAN and Its Faults

Summary

GTZAN [38] consists of 1,000 excerpts of 30 s in ten genres. Sturm [39] catalogued its faults, namely 50 exact repetitions, 21 recording repetitions and 13 version repetitions, repeated artists (for example, more than one third of the reggae excerpts are by one artist), mislabeled excerpts and distortions, and showed that these faults affect the evaluation of genre systems. Kereliuk et al. [40] proposed a fault-filtered split, used in the M2D evaluation with 443 training, 197 validation and 290 test clips.

Critical analysis

Strengths:

- The most widely used genre benchmark, which allows comparison with almost all earlier work.

Weaknesses:

- Faults and artist repetition make results optimistic, and the test split is very small, so confidence intervals are several points wide.

Relationship to the proposed work

GTZAN is an evaluation task with the fault-filtered split. Because of its size, the project does not draw conclusions from it alone and reports bootstrap intervals.

3.2.6.9 Neural Audio Synthesis of Musical Notes with WaveNet Autoencoders (NSynth)

Summary

Engel et al. [41] introduced the NSynth dataset of 305,979 monophonic musical notes of 4 s at 16 kHz from 1,006 instruments, annotated with pitch, velocity and instrument family and under a Creative Commons Attribution 4.0 license. The 11 instrument families are the usual classification target, and pitch classification is a second task.

Critical analysis

Strengths:

- Large, clean and controlled, with exact pitch labels.

Weaknesses:

- Isolated single notes from sample libraries, far from polyphonic music.

Relationship to the proposed work

The instrument and pitch tasks probe timbre and pitch information, the content that the CQT musical teacher targets. They are part of the evaluation suite.

3.2.6.10 OpenMIC-2018: An Open Dataset for Multiple Instrument Recognition**Summary**

Humphrey et al. [42] released OpenMIC-2018, a collection of 20,000 10-s audio excerpts drawn from the Free Music Archive with crowd-sourced labels for the presence of 20 instrument classes. It is distributed on Zenodo with the audio, pre-computed VGGish features, individual and aggregated labels and train and test partitions.

Critical analysis

Strengths:

- Real polyphonic music with instrument labels and an open license.

Weaknesses:

- Labels are weak and incomplete (crowd-sourced presence judgments), and ten source files are affected by an issue of the source FMA data.

Relationship to the proposed work

OpenMIC is used by MATPAC and MATPAC++ for evaluation. Because its audio comes from FMA, it could overlap with the pre-training pool, and it is therefore kept as an optional task.

3.2.7 Evaluation Benchmarks and Protocols

Comparisons between audio and music representations are only meaningful under a shared protocol. The two benchmarks below define the frozen-feature protocols that most of the reviewed models were evaluated with.

3.2.7.1 MARBLE: Music Audio Representation Benchmark for Universal Evaluation

Summary

Yuan et al. [43] proposed MARBLE, a benchmark of music representations built on a taxonomy of four levels (acoustic, performance, score and high-level description). It defines a unified protocol based on 18 tasks on 12 public datasets, with frozen pre-trained models and simple downstream heads, and it provides baselines for all open-source pre-trained models trained on music recordings. The results suggested that recently proposed large-scale pre-trained musical language models performed best on most tasks, with room for improvement.

Critical analysis

Strengths:

- Broad coverage of tagging, key, genre, emotion, instrument, pitch, beat and structure tasks, with an open toolkit and a leaderboard.
- Clear copyright statement, which makes the benchmark reusable.

Weaknesses:

- Many tasks need large downloads and several downstream heads, which is costly for a single-GPU project.
- The implementation has been revised, and later papers state that scores of the revised version are not strictly comparable with those of the original [15].

Relationship to the proposed work

MARBLE guided the choice of music tasks (genre, tagging, instrument, pitch and key). The project evaluates a smaller subset with the toolkit of the M2D authors for budget reasons and does not mix its numbers with MARBLE scores.

3.2.7.2 HEAR: Holistic Evaluation of Audio Representations

Summary

Turian et al. [44] report the HEAR 2021 challenge, which asked which audio embedding approach generalizes best to a wide range of downstream tasks across everyday domains without fine-tuning. Twenty-nine models submitted by thirteen external teams were evaluated through a common application programming interface on nineteen tasks derived from sixteen datasets covering speech, environmental sound and music. The code, the submitted models and the datasets were released.

Critical analysis

Strengths:

- The first large, open comparison of general-purpose audio embeddings under one protocol.
- The M2D line of work began as a submission to this challenge [3].

Weaknesses:

- Models date from 2021 and do not include current masked-latent-prediction models, and only some tasks involve music.

Relationship to the proposed work

HEAR establishes the principle of judging representations by frozen features on many tasks, which this project applies to a music-focused task set and to retention tasks.

3.2.8 Related Applications and Frameworks

Several open-source tools and released models determine how a study of this kind can be carried out on one machine. They are summarized below.

3.2.8.1 EVAR: Evaluation of Audio Representations

EVAR [45] is the evaluation toolkit of the M2D authors. It supports linear evaluation, fine-tuning and zero-shot evaluation on more than twelve tasks, among them GTZAN, NSynth (instrument, pitch and velocity), ESC-50, UrbanSound8K and FSD50K, and on more than ten models, and it has a plugin folder for extending to the MARBLE and OPERA benchmarks. It is the protocol of the M2D papers and is therefore the main evaluation tool of this project, but FMA-small and MagnaTagATune are not included and require task metadata to be written.

3.2.8.2 M2D Official Repository and Checkpoints

The official repository [46] provides the PyTorch code, the pre-trained weights (AudioSet models with several input sizes, speech models, M2D-CLAP models and an M2D-X model for respiratory sounds), the pre-training and further pre-training scripts with options for the background-noise list, the noise ratio, the offline teacher and the offline loss weight, and a guide for further pre-training. It is built by patching the MAE code and relies on pre-converted log-mel spectrograms. It is the code base of this project, but it has no support for a CQT teacher, which has to be added.

3.2.8.3 nnAudio

nnAudio [47] converts audio to spectrograms on the GPU with one-dimensional convolutional layers and provides mel and constant-Q transforms. It computes the log-mel front-end of M2D and the CQT of the official MERT implementation, so the same library produces both the inputs and the musical-teacher targets in this project.

3.2.8.4 X-ARES

X-ARES [48] is the evaluation suite of the ICME 2025 Audio Encoder Capability Challenge. It covers 21 public datasets across speech, music and environmental sound, with linear and k-nearest-neighbor probing, and it was used by Audio-JEPA, including the FMA-small and GTZAN tasks. It is not used in this project, but it shows how FMA-small is used as a standard evaluation task.

3.2.8.5 Public Checkpoints of Music and Latent-Prediction Models

Pre-trained models are available on public hubs: MERT in several versions (for example MERT-v0-public and MERT-v1-95M, released under a non-commercial CC-BY-NC license) [20], the general-audio and the music MATPAC++ encoders [49], and MusicFM models trained on the Million Song Dataset and on FMA [50]. They differ in sampling rate, context length and size, which limits comparability, but they make the reference experiments of this project possible without any pre-training.

3.3 Literature Review Summary Table

Table 3.1 summarizes the research studies in the research-project format, Table 3.2 the public datasets, and Table 3.3 the related applications and frameworks.

Table 3.1: Summary of the reviewed research studies: the author and reference, the method and data, the main reported result, and the main limitation with respect to this project.

Author	Method	Results	Limitations
Huang et al. [1]	Masked autoencoder on spectrograms; 80% masking; local-window decoder; AudioSet-2M	AudioSet-2M mAP 47.3; ESC-50 94.1% (fine-tuned)	Reconstruction target; needs fine-tuning
Chen et al. [2]	Iterative acoustic tokenizer and discrete-label prediction	AudioSet-2M mAP 48.6 (single), 50.6 (ensemble); ESC-50 98.1%	Multi-stage; best variants use a supervised teacher
Niizumi et al. [4]	Online and target networks; target sees only masked patches; ViT-Base on AudioSet	Linear-evaluation average 80.2 on nine tasks; best on four	Costly pre-training; weaker on speaker identification

Author	Method	Results	Limitations
Niizumi et al. [6]	Denoising distillation of speech-model features; noisy input to M2D	Comparable to HuBERT and WavLM on SUPERB; better on keyword spotting and emotion	Speech only; needs a domain teacher
Niizumi et al. [5]	Background noise and an offline network (labels, distillation or regularization)	ICBHI score 61.93, 61.17 (naive), 62.73 (noise), 63.29 (M2D-X); one 24-GB GPU	One small medical task; no music; forgetting not measured
Niizumi et al. [7]	M2D combined with CLAP, LLM sentence embeddings in the extension	Zero-shot GTZAN 75.17%; AudioSet mAP 49.0 (extension)	Needs captions; 10-s input; extension is a preprint
Assran et al. [9]	Predict target-block latents from a context block (images)	ViT-Huge/14 trained in under 72 h on 16 A100 GPUs	Vision only
Riou et al. [10]	Study of masking and duration for audio JEPA	Random masking best; GTZAN 82.1 to 86.9 as duration grows from 2.1 to 6.4 s	AudioSet only; few designs tested
Tuncay et al. [11]	Audio JEPA, ViT-Base, AudioSet, 13 epochs	kNN: first on ESC-50, FMA-small and GTZAN among three models	Under-trained; weak linear probing; preprint
Quelennec et al. [12]	Latent prediction plus DINO-style classification	Average 74.9 against 74.0 without classification; MTT 41.1 mAP	Classification acts as regularizer; AudioSet version drift
Quelennec et al. [14]	Multiple-hypothesis predictor with annealed MCL	Average 75.7; music-only 47.9 against 47.2 for AudioSet training	Preprint; private data
Gu et al. [15]	2-D music model, 100,000 h, mixed masking, SwiGLU, RoPE	Key 66.1; GTZAN 86.9; MTT AP 40.8; general models match on tagging and instruments	Preprint; private data; 32 GPUs
Castellon et al. [16]	Probes on Jukebox language-model features	30% better than tag-pretrained on four tasks; strong key detection	Billions of parameters; four tasks
Spijkervet and Burgoyne [18]	SimCLR on raw-waveform music	AP 33.1 with 1% of labels; above supervised on MTT	Tagging only; augmentation design
McCallum et al. [19]	Music against generic data; supervised against contrastive	Music-only pre-training best unsupervised with smaller batches	Private 117,497-h dataset
Li et al. [17]	Masked prediction with acoustic and CQT teachers; 95M and 330M	Best overall on 14 tasks; key 55.1 to 65.0 with CQT	5-s context; instability; large data
Won et al. [21]	Random-projection targets; Conformer, 330M	FMA-only (8,000 h) below the 160,000-h model	Random targets; large model
Zhu et al. [23]	Mel-RVQ tokens; Conformer	0.9K h beats MERT and MusicFM; average 75.8, 76.7, 77.0	160K-h data private; separate tokenizer stage
Gururangan et al. [24]	Domain- and task-adaptive pre-training of RoBERTa	Gains in four domains and eight tasks, also in low-resource settings	Text only; forgetting not measured
Hsu et al. [25]	Pre-training on in-domain unlabeled speech	Closes 66–73% of the domain gap	Speech; large corpora
Lee et al. [26]	Self-distillation for further pre-training of transformers	Beats baselines on image and text benchmarks	Two rounds; teacher may overfit

Author	Method	Results	Limitations
Zhang et al. [27]	Continual pre-training of BEATs with distillation and sampling	GTZAN F1 86.0 (original), 76.0 (naive), 88.0 (SONAR); AudioSet mAP 34.7 against 14.7	Preprint; tokenizer-specific
Niizumi et al. [28]	Compare 21 models on OPERA; further pre-train M2D on AudioSet and respiratory data	AUROC 0.814 for M2D with respiratory data, 0.782 for M2D, 0.733 for OPERA-CT	Narrow domain; no music
Ding and Lerch [29]	Adapters, prompts and reparameterization on music foundation models	Beat probing and fine-tuning on tagging; match fine-tuning on key and tempo	Preprint; supervised, per task
Yuan et al. [43]	Benchmark: 18 tasks on 12 datasets	Large musical language models best on most tasks	Heavy; versions not comparable
Turian et al. [44]	Benchmark: 29 models, 19 tasks, 16 datasets	First broad frozen-embedding comparison	2021 models; few music tasks

Table 3.2: Summary of the reviewed public datasets: the name and reference, the content and size, the license and access, and the main limitation with respect to this project.

Dataset	Content and size	License and access	Limitations
AudioSet [30]	2,084,320 labeled 10-s clips; 632-class ontology	Links to YouTube videos; audio not hosted	Clips disappear; versions differ
FSD50K [31]	51,197 clips; 108.3 h; 200 classes	Creative Commons; waveforms redistributable	Short clips; includes instrument classes
FMA [32]	106,574 tracks; 161 genres; subsets of 8,000 and 25,000 clips	Creative Commons audio; open download	Noisy genre labels; some corrupted files
MSD [34]	1,000,000 tracks; 44,745 artists	Metadata and features; no audio	Audio must be obtained elsewhere
Music4All [35]	109,269 excerpts of 30 s; about 910 h	Released by the authors for research	Mostly pop; 30-s excerpts
MTG-Jamendo [36]	Over 55,000 full tracks; 195 tags	Creative Commons audio; annotations CC BY-NC-SA 4.0	508 GB; non-commercial use
MagnaTagATune [37]	25,863 clips from over 5,000 songs; 50 common tags	16 kHz mono MP3 for research	Low quality; several clips per song
GTZAN [38, 39]	1,000 excerpts of 30 s; 10 genres	Publicly downloadable	Faults; 290-clip test split
NSynth [41]	305,979 notes; 1,006 instruments; 11 families	CC BY 4.0	Isolated single notes
OpenMIC-2018 [42]	20,000 clips of 10 s; 20 instruments	Open on Zenodo with audio and features	Weak crowd labels; derived from FMA

Table 3.3: Summary of related applications and frameworks: their main features, their relevance to this project, and their limitations.

Application	Features	Relevance	Limitations
EVAR [45]	Linear, fine-tuning and zero-shot evaluation; over 12 tasks and 10 models; plugins	Protocol of the M2D papers; GTZAN, NSynth, ESC-50 and UrbanSound8K ready	FMA-small and MagnaTagATune need new metadata
M2D repository [46]	Code, weights, further pre-training scripts and guide	Starting point and training code	Patch-based setup; no CQT teacher
nnAudio [47]	GPU mel and constant-Q transforms	Inputs and CQT targets from one library	CQT settings must match MERT
X-ARES [48]	21 datasets; linear and kNN probing	Includes FMA-small and GTZAN	Not used in this project
Public checkpoints [20, 49, 50]	MERT, MATPAC++ and MusicFM weights	Reference models without pre-training	Licenses and input rates differ

3.4 Conclusion

The literature shows that masked latent prediction on spectrograms yields strong general-purpose audio representations, with M2D and MATPAC++ at the top of the AudioSet-trained models. These general-audio models are already strong on music: in the unified comparison of MATPAC++, the AudioSet-trained models were on a par with or above MERT, and music-only training added about 0.7 points on average [14]; in the evaluation of PupuM2D, AudioSet-trained M2D, M2D-X and MATPAC++ checkpoints matched or exceeded MERT, MuQ and MusicFM on tagging and instrument classification, and a 100,000-hour in-domain model was clearly ahead mainly on valence in emotion recognition, while the frame-level tasks (key, beat) could not be compared with the general models [15]. Music-specific models gain from music-oriented targets, such as the CQT of MERT and the Mel-RVQ tokens of MuQ, but they rely on 900 to 160,000 hours of music, often private, and on multi-GPU training, and a model trained on FMA alone was weaker than one trained on a larger curated corpus [21].

Further pre-training on domain data is established in language, speech and respiratory sound [24, 25, 28], and the M2D-X recipe makes it practical on one GPU through background noise and a frozen-teacher regularizer [5]. The risks are also documented: naive continuation can overfit and forget, as the FMA-based experiments of SONAR on a different model family show [27]. Public resources suffice to run such a study: FMA is the only open, artist-filtered and redistributable full-quality music corpus of suitable size, FSD50K supplies noise, and EVAR provides a protocol for the M2D family together with the public MERT, MATPAC++ and MusicFM checkpoints for reference. Protocols differ between benchmarks, so numbers from different papers cannot be compared directly.

No reviewed work applies the M2D-X recipe to music, combines noise, a frozen-teacher regularizer

and a music-specific CQT teacher in one framework, does so on a small open subset of about 10,000 tracks with one GPU, or measures the retention of general-audio ability next to the music gain under one protocol. This project addresses that gap: it further pre-trains the AudioSet M2D model on FMA with the M2D-X-Music objective, compares each component in a controlled set of arms, and compares the result with music-specific references (Chapter 5).

Chapter 4 Software Requirement Specifications

This chapter specifies the software that accompanies the research: a small web dashboard, called the *M2D Music Dashboard* in this report, that is built for demonstration only. The dashboard has two purposes. First, it presents the results of the further pre-training study of Chapter 5 in a form that a supervisor, an evaluator or another student can explore without reading the result files. Second, it demonstrates four music downstream tasks that belong to the MARBLE benchmark [43], namely genre classification (GTZAN [38]), music auto-tagging (MagnaTagATune [37]), instrument classification and pitch classification (both NSynth [41]). A visitor chooses a task, gives an audio clip and sees the prediction of a chosen model. The training and evaluation pipeline itself, the datasets and the models are specified in Chapter 5 and are not repeated here; this chapter only covers the web application and its development. The dashboard is deliberately simple and minimal: it has no user accounts, no login and no sign-up, and it runs on the same workstation as the research. The chapter lists the features, the functional requirements, the quality attributes and the non-functional requirements, the assumptions, the use cases, the hardware and software requirements, the planned user interface, the database design and the risks.

4.1 List of Features

The dashboard is a single-page web application with three views: *Results*, *Demo* and *About*. The following features will be available.

- **Results overview.** The aggregate music score S_{mus} and the retention score S_{gen} (Section 5.7) of every model, a table of the per-task scores with their spread over probe seeds, and a bar chart, with a filter by model group (original model, further pre-trained arms, data-size runs and reference models).
- **Task selection.** Four task cards (genre, tagging, instrument and pitch) that show the dataset, the label set and the metric of each task.
- **Audio input.** Upload of an audio file (WAV, MP3, FLAC or OGG) or selection of a bundled, openly licensed sample clip, with in-browser playback and a log-mel spectrogram preview.
- **Model selection.** A choice between the unmodified M2D model (arm A0) and the best further pre-trained model (A*); reference models such as MERT-v1-95M [17] can be added when their probes are available.
- **Task-specific output.** The top-5 genres with probabilities; the tags above the decision threshold together with the ten highest tag scores; the top-3 instrument families; and the top-3 pitches as

MIDI number, note name and frequency. The inference time is shown with every prediction, and the label given by the source dataset is shown for sample clips.

- **Model comparison.** The same clip can be run through two models and the two outputs are shown side by side, with the labels predicted by only one model highlighted.
- **About page.** A short description of the project, the models, the datasets and their licenses, and a statement that single-clip predictions are a demonstration, not benchmark results.

4.2 Functional Requirements

The functional requirements describe the external behavior of the dashboard. Three kinds of users are involved: the *Visitor*, who uses the web interface (supervisor, evaluator, researcher or student); the *Developer*, who maintains the dashboard and loads results through a command-line script, since there is no administration page; and the *System* itself for automatic behavior. Table 4.1 lists the requirements.

Table 4.1: Functional requirements of the dashboard. Each requirement has an identifier, a statement and the user to whom it applies.

ID	Requirement	User
FR-01	The system shall open on the Results Overview and show S_{mus} and S_{gen} for every model stored in the database.	Visitor
FR-02	The system shall show a table of per-task test scores (mean and standard deviation over probe seeds) with the metric of each task.	Visitor
FR-03	The visitor shall be able to filter the results by model group: original model, further pre-trained arms, data-size runs and reference models.	Visitor
FR-04	The system shall draw a bar chart of the mean music score of the selected models.	Visitor
FR-05	The system shall list the four demonstration tasks with their dataset, number of labels and metric, and the visitor shall be able to select one.	Visitor
FR-06	The visitor shall be able to upload one audio file in WAV, MP3, FLAC or OGG format of at most 10 MB.	Visitor
FR-07	The visitor shall be able to select a bundled sample clip instead of uploading a file, and to play the chosen clip in the browser.	Visitor
FR-08	The system shall check the type, size and readability of an uploaded file and reject an invalid file with an error message that states the reason.	System
FR-09	The system shall convert the input to mono at the sampling rate of the selected model (16 kHz for M2D), keep at most the first 30 s, compute the log-mel spectrogram with the normalization used in evaluation, and show a preview of it.	System
FR-10	The visitor shall be able to select the model used for the prediction; A^* shall be selected by default.	Visitor
FR-11	The system shall compute the clip embedding with the frozen encoder (6.08-s windows, padded if shorter, averaged over time) and apply the linear probe of the selected model and task.	System
FR-12	The system shall display the task-specific output: top-5 genres; tags whose probability exceeds the threshold of the probe and the ten highest tag scores; top-3 instrument families; top-3 pitches with MIDI number m , note name and frequency $440 \cdot 2^{(m-69)/12}$ Hz. It shall also display the inference time, the device used and, for sample clips, the source label.	Visitor

ID	Requirement	User
FR-13	The visitor shall be able to run two models on the same clip and see both outputs side by side, with the labels predicted by only one model highlighted.	Visitor
FR-14	The system shall queue inference requests when the model is busy and show the position of the request in the queue.	System
FR-15	The system shall delete an uploaded file as soon as its prediction has been returned.	System
FR-16	The system shall record anonymous metadata of every prediction (task, model, input source, duration, device, latency and status) in the inference log, without audio or network addresses.	System
FR-17	The developer shall be able to import evaluation results and register models, probes and sample clips with a command-line script that validates every record before writing it.	Developer
FR-18	The system shall provide an About page with the project description, the models, the datasets and licenses, and a demonstration disclaimer.	Visitor

4.3 Quality Attributes

The following quality attributes guide the design and will be used to judge whether the dashboard serves its demonstrative purpose. Because the dashboard is a companion to a research study, the attribute that matters most is that it shows the research faithfully.

4.3.1 Usability

The audience includes evaluators who are not specialists in music information retrieval. The interface therefore uses one page with three tabs and a three-step flow on the Demo tab (choose a task, give audio, run), plain labels such as “Genre” rather than dataset codes, and short help texts beside each input. A first-time visitor should be able to obtain a prediction without instructions.

4.3.2 Correctness and Consistency

A demonstration is useful only if it reflects the models that were evaluated. The dashboard shall reuse the preprocessing, windowing and embedding code of the evaluation pipeline (Section 5.7) and load the same linear probes, rather than re-implementing them. The scores on the Results tab shall come from the same result files as the tables in the report, imported unchanged, so that the dashboard can never show a number that the report does not.

4.3.3 Reliability

The dashboard will be shown during evaluations and presentations, where a failure is costly. Invalid inputs shall be rejected with clear messages instead of errors, an inference failure shall not stop the server, and the Results tab shall remain usable even if the model cannot be loaded.

4.3.4 Maintainability and Extensibility

New checkpoints are produced during the project and the best arm A^* is only known at the end. Adding a model or a task shall therefore require only a probe file and new rows in the database, imported with the developer script, without any change to the web code. The backend separates the inference module, the database layer and the web interface so that each can be changed alone.

4.3.5 Portability

The dashboard shall run on the research workstation and on any machine with a recent Python environment, with or without an NVIDIA GPU, and the frontend shall need nothing more than a web browser.

4.4 Non-Functional Requirements

The non-functional requirements state measurable targets for the dashboard. The figures below are design targets for a demonstration on one workstation, not measured results.

4.4.1 Performance

- For a clip of up to 30 s, the system shall return a prediction within 5 s on the GPU host and within 30 s on CPU, excluding upload time.
- The Results Overview shall load within 2 s on the local network.
- The models and probes shall be loaded once when the server starts, not for every request.

4.4.2 Capacity and Concurrency

- The system shall serve at least five visitors at the same time, which is enough for a presentation or a lab demonstration.
- Inference requests shall be processed one at a time by a single worker on the GPU, and further requests shall wait in a queue, so that a burst of requests cannot exhaust the GPU memory.

4.4.3 Availability

- The dashboard shall be available on the local network during the scheduled evaluation and presentation sessions; it is not intended as a permanent public service.
- The server shall start with one command and be ready within two minutes.

4.4.4 Security and Privacy

- The system shall not create user accounts and shall not store personal data or tracking cookies.
- Uploaded files shall be stored under random names in a temporary directory and deleted after the prediction, or at the latest after ten minutes.
- The server shall enforce the file type and the size limit itself and shall not rely on checks in the browser.
- The web interface shall have read-only access to the result tables; results can be written only through the developer script.

4.4.5 Reusability

- The inference module (audio loading, embedding and probe prediction) shall be a Python package that can be used without the web server, for example from a notebook.
- The frontend shall obtain all data through a small JSON API, so that another interface could use the same backend.

4.4.6 Compatibility

- The frontend shall work in current versions of Google Chrome, Mozilla Firefox and Microsoft Edge at a desktop or laptop screen width.

4.5 Assumptions

The specification is based on the following assumptions.

- The further pre-trained checkpoints and the linear probes of the four demonstration tasks are produced by the evaluation of Chapter 5 before the dashboard is finalized.
- The dashboard is hosted on the research workstation with the RTX A4000 GPU, or on a CPU-only machine with slower predictions.
- The inputs are music recordings. The instrument and pitch probes are trained on single isolated notes from NSynth, so their outputs are meaningful only for similar inputs; for other music they are shown for illustration.
- Visitors have basic computer skills and a current web browser, and, to play clips, speakers or headphones.

- The result files imported into the dashboard are the final values produced by the evaluation pipeline.
- Visitors understand that a prediction on one clip illustrates a model and is not a measure of its quality; the benchmark scores are those on the Results tab.

4.6 Use Cases

The use cases below cover the central behavior of the dashboard. Figure 4.1 shows the use-case diagram: the Visitor takes part in UC-01 to UC-05 through the web interface, and the Developer takes part in UC-06 through the command line. UC-05 extends UC-04 by running a second model on the same clip. Tables 4.2 to 4.7 describe each use case in the standard format, with step numbers that alternate between the actor and the system; an alternative step such as 4-A replaces the system response with the same number.

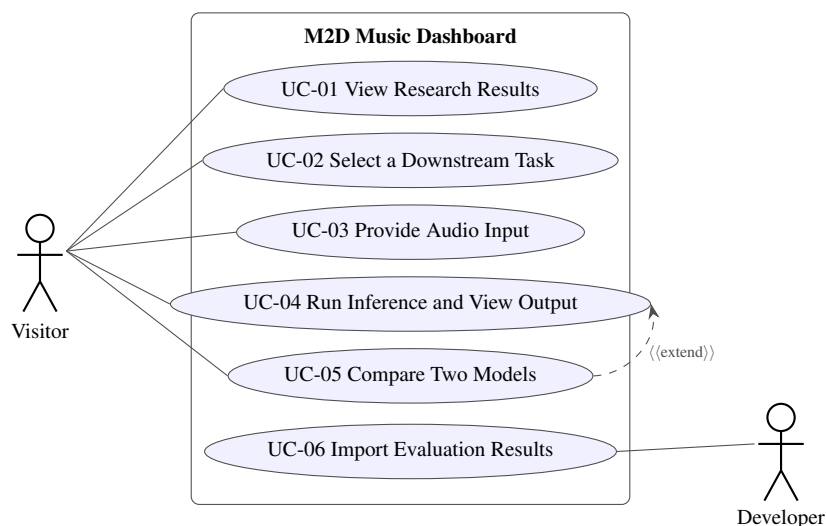


Figure 4.1: Use-case diagram of the dashboard. A stick figure labeled Visitor on the left is connected to five use cases inside the system boundary: view research results, select a downstream task, provide audio input, run inference and view output, and compare two models. A dashed arrow marked “extend” goes from the comparison use case to the inference use case. A stick figure labeled Developer on the right is connected to the sixth use case, import evaluation results.

Table 4.2: Use case UC-01: View Research Results.

Name		View Research Results	
Actors		Visitor	
Summary		The visitor opens the dashboard and reviews the results of the further pre-training study: the aggregate scores, the per-task scores and a chart, filtered by model group.	
Pre-Conditions		The server is running. The developer has imported the evaluation results (UC-06).	
Post-Conditions		The results of the selected model group are displayed. The database is not changed.	
Special Requirements		The scores shall be identical to those reported in the results chapter of this report.	
Basic Flow			
Actor Action		System Response	
1	The visitor opens the dashboard address in a browser.	2	The system loads the results from the database and displays the Results Overview with S_{mus} , S_{gen} , the per-task table and the bar chart.
3	The visitor selects a model group in the filter.	4	The system updates the table and the chart to show only the models of that group.
Alternative Flow			
1	The visitor opens the dashboard before any result has been imported.	2-A	The system displays the message “No results have been imported yet” and keeps the Demo and About tabs available.

Table 4.3: Use case UC-02: Select a Downstream Task.

Name		Select a Downstream Task	
Actors		Visitor	
Summary		The visitor chooses one of the four demonstration tasks, and the system shows the labels and the input expected by that task.	
Pre-Conditions		The server is running and probes of at least one model exist for the task.	
Post-Conditions		The task is selected and the audio input panel is enabled for it.	
Special Requirements		For the instrument and pitch tasks the system notes that a single isolated note is expected, as in NSynth.	
Basic Flow			
Actor Action		System Response	
1	The visitor opens the Demo tab.	2	The system displays four task cards with the dataset, the number of labels and the metric of each task.
3	The visitor clicks a task card.	4	The system highlights the card, shows the expected input and enables the audio input panel and a model list that contains only the models with a probe for the task.
Alternative Flow			
3	The visitor clicks a task for which no probe has been registered.	4-A	The system shows the card as unavailable with the message “This task is not available yet”.

Table 4.4: Use case UC-03: Provide Audio Input.

Name		Provide Audio Input	
Actors		Visitor	
Summary		The visitor uploads an audio file or selects a bundled sample clip, which can be played before the prediction.	
Pre-Conditions		A task has been selected (UC-02).	
Post-Conditions		A valid clip is held by the server for the current request, and a player and a log-mel preview of it are displayed.	
Special Requirements		Accepted formats are WAV, MP3, FLAC and OGG, with at most 10 MB per file. Uploaded files are deleted after the prediction.	
Basic Flow			
Actor Action		System Response	
1	The visitor drags an audio file into the upload area, or clicks <i>Browse</i> and selects a file.	2	The system checks the type and size, decodes the audio, and displays its duration, a player and a log-mel preview.
3	Alternatively, the visitor selects a sample clip from the list.	4	The system loads the clip and displays its source dataset, license and source label, a player and a log-mel preview.
Alternative Flow			
1	The visitor uploads a file in an unsupported format.	2-A	The system displays “Unsupported file format. Please use WAV, MP3, FLAC or OGG.”
1	The visitor uploads a file larger than 10 MB.	2-B	The system displays “File too large (maximum 10 MB).”
1	The visitor uploads a clip longer than 30 s.	2-C	The system accepts the file and displays “Only the first 30 s will be analyzed.”
1	The visitor uploads a corrupted file.	2-D	The system displays “The file could not be read.”

Table 4.5: Use case UC-04: Run Inference and View Output.

Name		Run Inference and View Output	
Actors		Visitor	
Summary		The visitor runs the selected model on the clip and sees the task-specific prediction.	
Pre-Conditions		A task has been selected (UC-02), a valid clip is available (UC-03) and a model has been selected.	
Post-Conditions		The prediction is displayed, an anonymous record is added to the inference log and an uploaded file is deleted.	
Special Requirements		The preprocessing and embedding are those of the evaluation pipeline. The response time follows the targets of Section 4.4.	
Basic Flow			
Actor Action		System Response	
1	The visitor selects a model (A* by default) and clicks <i>Run</i> .	2	The system resamples the clip, computes the log-mel spectrogram and the clip embedding, and applies the linear probe of the model and task.
3	The visitor reads the result.	4	The system displays the task-specific output, the inference time, the device and, for a sample clip, the source label.
Alternative Flow			
1	The visitor clicks <i>Run</i> while another request is being processed.	2-A	The system queues the request, shows its position and starts it when the model is free.
1	The visitor clicks <i>Run</i> on a host without a usable GPU.	2-B	The system runs the model on the CPU and notes that the response is slower.
1	The visitor clicks <i>Run</i> and the inference fails.	2-C	The system displays “The prediction failed. Please try again.” and records the failure in the log.

Table 4.6: Use case UC-05: Compare Two Models on a Clip.

Name		Compare Two Models on a Clip	
Actors		Visitor	
Summary		The visitor runs two models on the same clip and sees their outputs side by side, for example the original M2D model A0 and the further pre-trained model A*.	
Pre-Conditions		As in UC-04, and probes of both models exist for the selected task.	
Post-Conditions		Both predictions are displayed side by side and two records are added to the inference log.	
Special Requirements		Both models receive the same decoded clip.	
Basic Flow			
Actor Action		System Response	
1	The visitor ticks <i>Compare with</i> and chooses a second model.	2	The system lists the models with a probe for the task, excluding the model already selected.
3	The visitor clicks <i>Run</i> .	4	The system runs both models one after the other and displays the outputs in two columns, highlighting the labels predicted by only one model.
Alternative Flow			
3	The visitor clicks <i>Compare with another model</i> on an output page.	4-A	The system reuses the clip, runs only the second model and opens the comparison view.

Table 4.7: Use case UC-06: Import Evaluation Results.

Name		Import Evaluation Results	
Actors		Developer	
Summary		The developer runs a command-line script that reads the result files of the evaluation pipeline and registers models, probes, results and sample clips in the database.	
Pre-Conditions		The evaluation has finished and its result files exist in the agreed CSV or JSON format. The developer has access to the server.	
Post-Conditions		The database contains the new rows, and the Results Overview shows them after a refresh.	
Special Requirements		The import is transactional: either all rows of a file are written or none.	
Basic Flow			
Actor Action		System Response	
1	The developer runs the import script with the path of a result file.	2	The script checks every record (known model and task codes, the metric of the task, a score between 0 and 100, at least one seed) and writes all rows in one transaction.
3	The developer opens the dashboard.	4	The system displays the updated results.
Alternative Flow			
1	The file contains an unknown model or task code or a missing field.	2-A	The script rejects the whole file, reports the line and the field, and writes nothing.
1	Results for the same model, task and split already exist.	2-B	The script stops, unless the replace option is given, in which case the old rows are replaced.

4.7 Hardware and Software Requirements

The dashboard is small and reuses the environment of the research, so its requirements are modest. The following hardware and software are needed to develop and deploy it.

4.7.1 Hardware Requirements

- A development and hosting workstation; the research workstation with one NVIDIA RTX A4000 GPU (16 GB) will be used. A GPU is recommended for fast predictions but is not required.
- At least 16 GB of system memory and about 10 GB of free disk space for the checkpoints, probes, sample clips and database.
- A desktop or laptop computer with a web browser for each visitor, with speakers or headphones to play clips.
- A local network connection between the host and the visitors' computers, or the host itself for a single-screen demonstration.

4.7.2 Software Requirements

- Linux operating system with the NVIDIA driver and CUDA runtime on the host.
- Python 3.10 or above, the language of the backend and of the research code.
- PyTorch and torchaudio for model inference, with the M2D code base and [timm \[46\]](#) to build the encoder.
- [nnAudio \[47\]](#) for the log-mel front-end, as in the evaluation pipeline.
- [soundfile](#) and [FFmpeg](#) to decode the accepted audio formats.
- [FastAPI](#) and the [Uvicorn](#) server for the JSON API and for serving the frontend files.
- [SQLite](#) with the [SQLAlchemy](#) library for the database.
- [HTML5](#), [CSS3](#) and [JavaScript](#) for the frontend, with the [Chart.js](#) library for charts.
- [Git](#) and [GitHub](#) for version control, and [Visual Studio Code](#) as editor.
- A current web browser such as [Google Chrome](#), [Mozilla Firefox](#) or [Microsoft Edge](#).

4.8 Graphical User Interface

This section presents the planned screens of the dashboard as wireframes; screenshots of the implemented dashboard will replace them in the chapter on implementation. All screens share a top bar with the dashboard name and three tabs, Results, Demo and About, so that every view can be reached from every screen. Values shown in the wireframes are placeholders. Figure 4.2 shows the navigation flow, and Figures 4.3 to 4.7 show the screens with the use cases that each one covers.

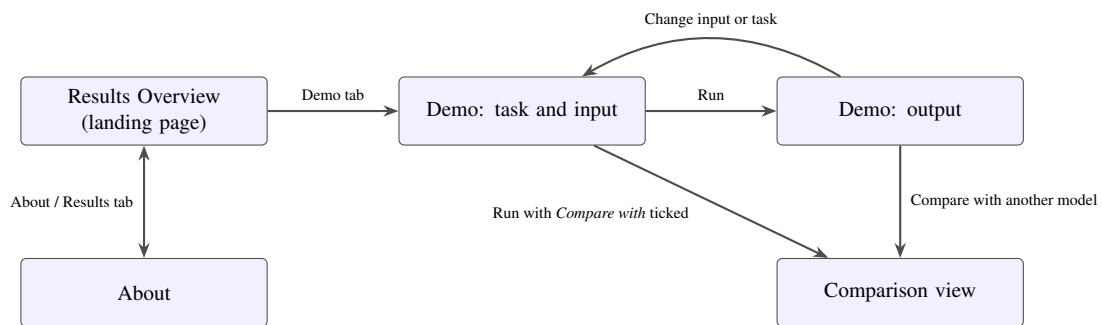


Figure 4.2: Navigation flow of the dashboard. The visitor starts on the Results Overview. The Demo tab leads to the task and input screen; clicking Run leads to the output screen, or to the comparison view when a second model is selected. From the output screen the visitor can go back to change the input or task, or open the comparison view. The About page is reachable through the tabs from every screen.

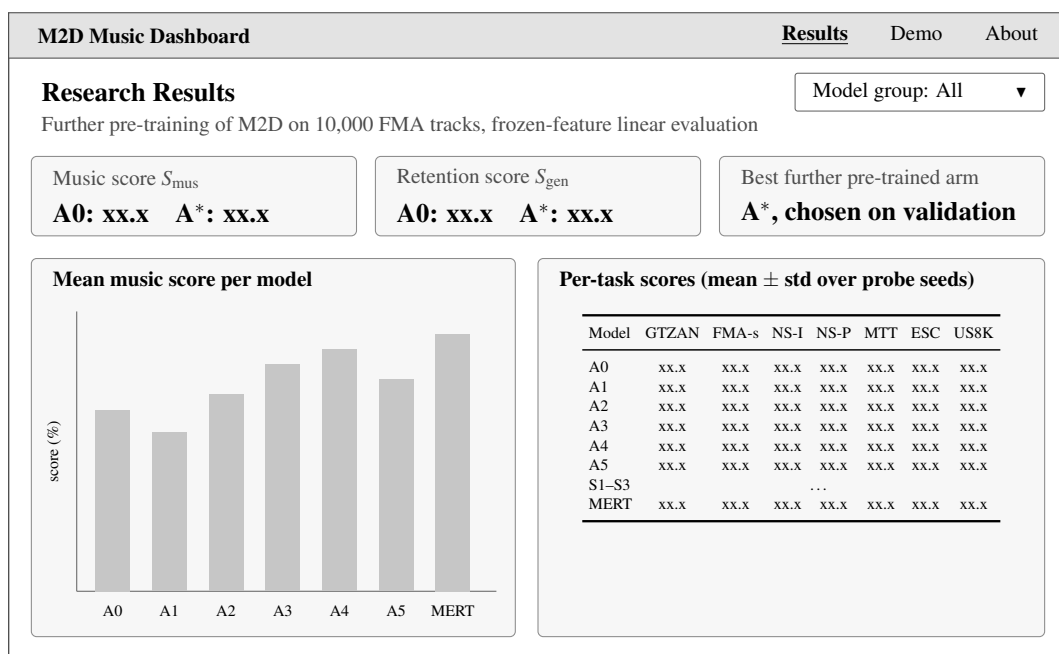


Figure 4.3: Wireframe of the Results Overview, the landing page (UC-01). Below the top bar with the Results, Demo and About tabs, the page shows a model-group filter at the top right; three summary cards with the music score and the retention score of the original model A0 and of the best further pre-trained model A*, and the identity of A*; a bar chart of the mean music score per model on the left; and a table of per-task scores on the right. All numbers are placeholders.

M2D Music Dashboard
Results
Demo
About

1. Choose a task

Genre ✓
GTZAN, 10 genres

Tagging
MagnaTagATune, 50 tags

Instrument
NSynth, 11 families

Pitch
NSynth, MIDI pitch

2. Provide audio

Drag and drop an audio file here

Browse

WAV, MP3, FLAC or OGG; at most 10 MB; only the first 30 s are used

or choose a sample clip:

Sample 3: FMA clip (CC license), source label "Rock"

0:00 / 0:30

log-mel spectrogram preview

3. Choose a model and run

Model: A* (further pre-trained)

☐ Compare with

A0 (original M2D)

Run

Demonstration only: predictions on single clips are not benchmark results. Uploaded audio is deleted after processing.

Figure 4.4: Wireframe of the Demo screen for task selection and audio input (UC-02, UC-03, and the start of UC-04 and UC-05). Step 1 shows four task cards (genre, tagging, instrument and pitch) with the genre card selected. Step 2 shows a dashed drag-and-drop upload area with a Browse button and the accepted formats, and on the right a sample-clip list, an audio player and a log-mel spectrogram preview. Step 3 shows a model list, a Compare with check box with a second model list, and a Run button. A disclaimer is shown at the bottom.

M2D Music Dashboard
Results
Demo
About

Result: genre classification (GTZAN labels)

Input: sample clip (FMA)
Duration: 30.0 s (5 windows)
Source label: Rock

Model: A* (further pre-trained)
Device: GPU
Inference time: x.xx s

0:00 / 0:30

Top-5 genres

rock	xx %
metal	xx %
blues	xx %
country	xx %
pop	xx %

Change input

Change task

Compare with another model

Figure 4.5: Wireframe of the Demo output screen for the genre task (UC-04). The left panel lists the input (a sample clip from FMA, its duration and source label), the model, the device and the inference time, with an audio player. The right panel shows horizontal bars for the five most probable genres, the top one highlighted. Buttons at the bottom change the input or task or open the comparison view. For tagging, the panel lists the tags above the threshold and the ten highest tag scores; for instrument, the three most probable instrument families; and for pitch, the three most probable pitches with MIDI number, note name and frequency. Values are placeholders.

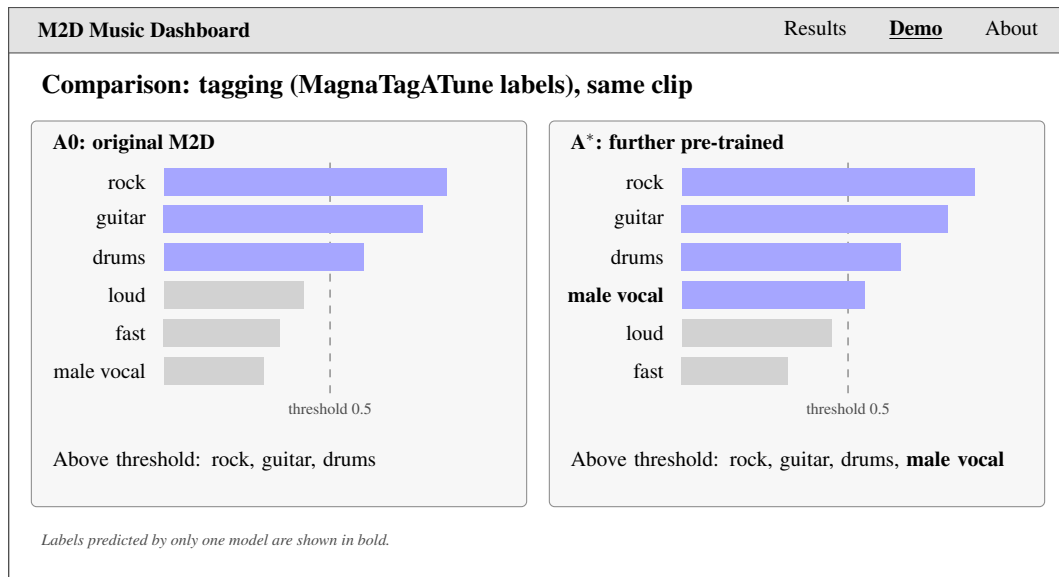


Figure 4.6: Wireframe of the comparison view (UC-05), here for the tagging task. Two panels side by side show the outputs of the original model A0 on the left and of the further pre-trained model A* on the right for the same clip. Each panel has horizontal bars for the highest tag scores, a dashed line at the decision threshold of 0.5, and the list of tags above the threshold; a tag predicted by only one model is shown in bold. The tags and scores are illustrative placeholders.

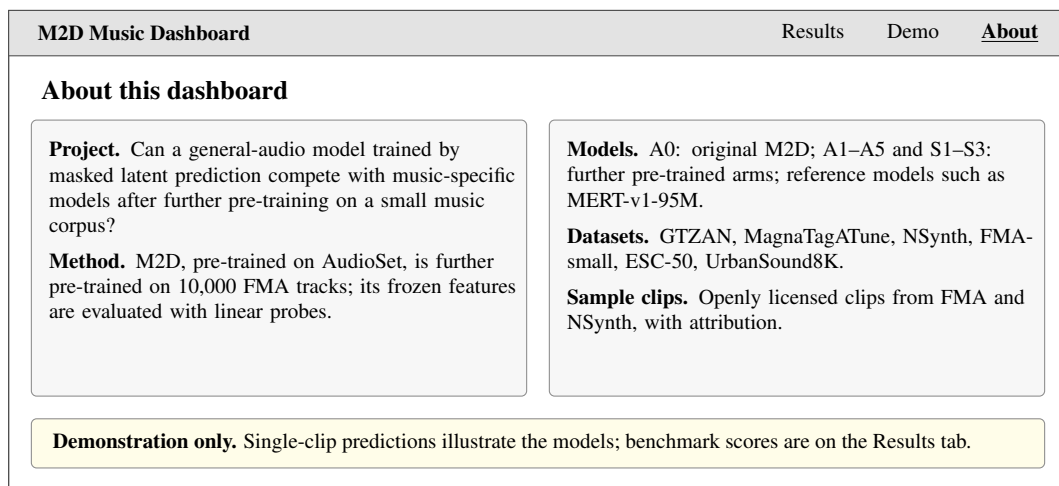


Figure 4.7: Wireframe of the About page (FR-18). Two text panels describe the project question and method on the left, and the models, datasets and sample-clip licenses on the right. A highlighted box at the bottom states that single-clip predictions are a demonstration and that the benchmark scores are those on the Results tab.

4.9 Database Design

The dashboard needs a small database for two kinds of data: the research data that it displays (models, tasks, labels, probes and evaluation results) and the operational data of the demonstration (sample clips and an anonymous inference log). It has no user data, because there are no accounts. SQLite is used: the dashboard runs on a single host, the data are small and mostly read, and a file-based database needs no separate server and can be copied together with the code. The tables are accessed through SQLAlchemy, so that a client–server database could replace SQLite later without changes to the application code. The retention tasks ESC-50 [51] and UrbanSound8K [52] are stored as tasks for the Results tab but are marked as not available in the demonstration.

4.9.1 ER Diagram

Figure 4.8 shows the entities and their relationships. Model and Task are the central entities. A probe, an evaluation result and an inference-log record each belong to one model and one task, so each of the three entities is in a one-to-many relationship with both. A task has many labels and many sample clips. Audio files and probe weights are stored on disk, and the database keeps only their paths.

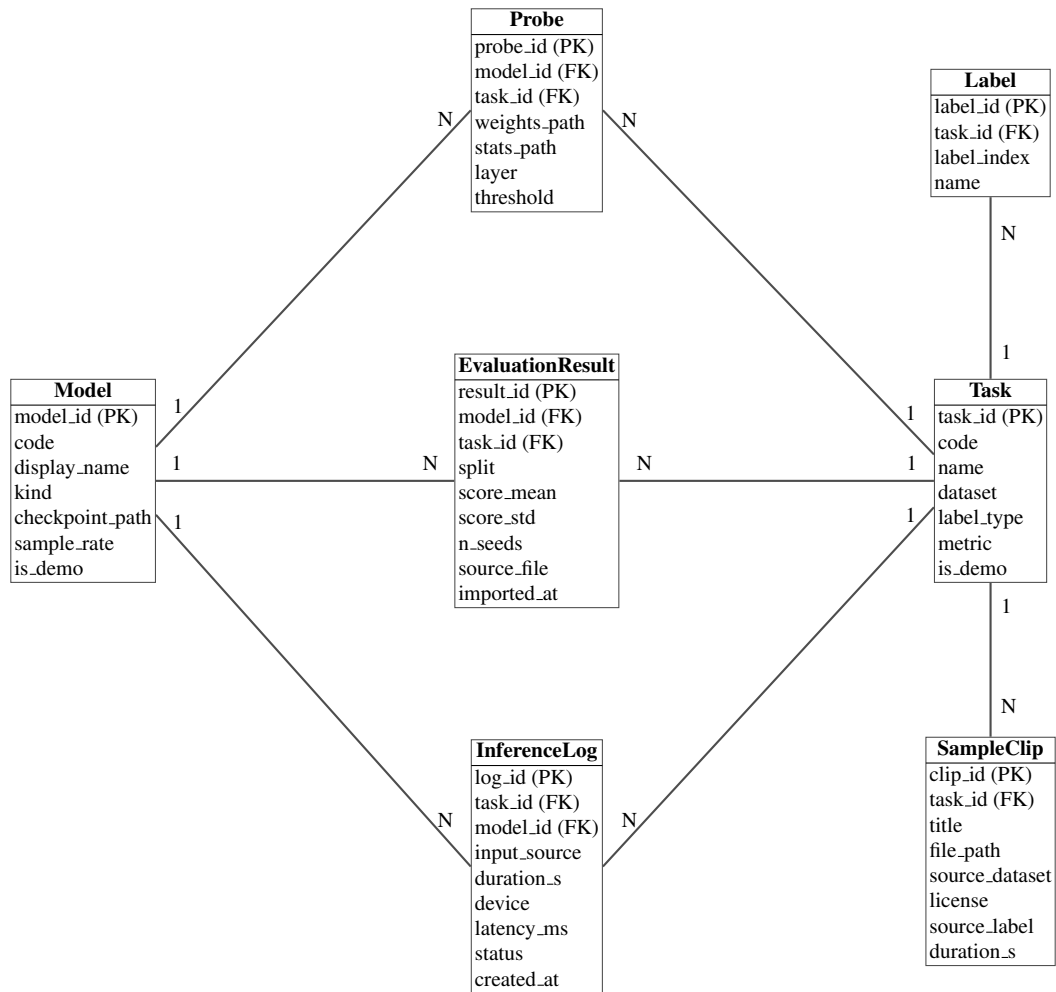


Figure 4.8: ER diagram of the dashboard database. Seven entities are drawn as boxes listing their attributes, with primary keys (PK) and foreign keys (FK) marked. Model on the left and Task on the right are each connected by one-to-many lines to the three entities in the middle: Probe, EvaluationResult and InferenceLog. Task is also connected by one-to-many lines to Label above it and SampleClip below it. The numbers 1 and N mark the cardinality at each end.

4.9.2 Data Dictionary

Table 4.8 lists the attributes of every entity. SQLite has no separate Boolean and date types; Boolean values are stored as integers 0 and 1, and time stamps as ISO 8601 text.

Table 4.8: Data dictionary of the dashboard database. For each entity the table gives the attributes, their data types, whether they may be empty, and a description.

Entity	Attribute	Data Type	Nullable	Description
Model	model_id	Integer	No	Primary key.
	code	Text	No	Unique short code, for example A0, A3, S2 or MERT-v1-95M.
	display_name	Text	No	Name shown in the interface.
	kind	Text	No	One of <i>original</i> , <i>further-pretrained</i> or <i>reference</i> .
	checkpoint_path	Text	Yes	Path of the checkpoint on disk; empty for models that are shown only on the Results tab.
	sample_rate	Integer	No	Input sampling rate in Hz (16,000 for M2D).
	is_demo	Boolean	No	Whether the model can be selected on the Demo tab.
Task	task_id	Integer	No	Primary key.
	code	Text	No	Unique short code, for example <i>genre</i> , <i>tagging</i> , <i>instrument</i> , <i>pitch</i> , <i>esc50</i> .
	name	Text	No	Name shown in the interface.
	dataset	Text	No	Evaluation dataset, for example GTZAN or NSynth.
	label_type	Text	No	<i>single</i> for single-label or <i>multi</i> for multi-label tasks.
	metric	Text	No	<i>accuracy</i> or <i>mAP</i> .
	is_demo	Boolean	No	Whether the task is offered on the Demo tab.
Label	label_id	Integer	No	Primary key.
	task_id	Integer	No	Foreign key to Task.
	label_index	Integer	No	Output index of the label in the probe.
	name	Text	No	Label name, for example <i>rock</i> , <i>guitar</i> , <i>brass</i> or a MIDI pitch.
Probe	probe_id	Integer	No	Primary key.
	model_id	Integer	No	Foreign key to Model.
	task_id	Integer	No	Foreign key to Task; the pair (model_id, task_id) is unique.
	weights_path	Text	No	Path of the linear-layer weights.
	stats_path	Text	No	Path of the mean and standard deviation used to standardize embeddings.
	layer	Integer	Yes	Encoder layer used for reference models; empty for the final layer of M2D.
	threshold	Real	Yes	Decision threshold for multi-label tasks (0.5 by default); empty for single-label tasks.

Entity	Attribute	Data Type	Nullable	Description
EvaluationResult	result_id	Integer	No	Primary key.
	model_id	Integer	No	Foreign key to Model.
	task_id	Integer	No	Foreign key to Task.
	split	Text	No	<i>validation</i> or <i>test</i> ; (model_id, task_id, split) is unique.
	score_mean	Real	No	Mean score over probe seeds, in percent.
	score_std	Real	Yes	Standard deviation over probe seeds.
	n_seeds	Integer	No	Number of probe seeds.
	source_file	Text	No	Result file from which the row was imported.
SampleClip	imported_at	Date-time	No	Time of import.
	clip_id	Integer	No	Primary key.
	task_id	Integer	No	Foreign key to Task for which the clip is offered.
	title	Text	No	Name shown in the sample list.
	file_path	Text	No	Path of the audio file on disk.
	source_dataset	Text	No	Dataset from which the clip comes, for example FMA or NSynth.
	license	Text	No	License of the clip and attribution text.
	source_label	Text	Yes	Label given by the source dataset, shown for reference.
InferenceLog	duration_s	Real	No	Duration in seconds.
	log_id	Integer	No	Primary key.
	task_id	Integer	No	Foreign key to Task.
	model_id	Integer	No	Foreign key to Model.
	input_source	Text	No	<i>upload</i> or <i>sample</i> .
	duration_s	Real	Yes	Duration of the analyzed audio; empty if decoding failed.
	device	Text	No	<i>gpu</i> or <i>cpu</i> .
	latency_ms	Integer	Yes	Processing time in milliseconds; empty if the request failed.
	status	Text	No	<i>ok</i> or an error code.
	created_at	Date-time	No	Time of the request.

4.10 Risk Analysis

The dashboard is a small part of a research project with a fixed three-month schedule and a single GPU, so its risks are mostly about timing, faithfulness and licensing rather than scale. The main risks are described below and summarized with their mitigation in Table 4.9.

4.10.1 Technical Risks

The dashboard depends on the outputs of the research: if the further pre-trained checkpoints or their probes are late, there is nothing to demonstrate for A^* . The dashboard is therefore developed first with the unmodified model A_0 , whose probes are available early, and A^* is added by import. A second risk is a silent mismatch between the preprocessing of the dashboard and that of the evaluation, which would

make the demonstration show a model that behaves differently from the one evaluated; it is reduced by importing the evaluation code instead of copying it and by a check that the dashboard reproduces the cached embeddings of a few test clips. Finally, the GPU may be busy with training or unavailable at a presentation, in which case the CPU path is used, and some uploaded files may fail to decode, which is handled by the input checks.

4.10.2 Schedule Risks

The research has priority over the dashboard in the three-month schedule. The dashboard is kept minimal, with no accounts and a single page, and it is built in small steps (results view, then one task, then the other tasks, then comparison) so that a usable version exists at every stage.

4.10.3 Legal and Licensing Risks

Bundled sample clips are redistributed to visitors, so they must have suitable licenses. Sample clips are taken only from openly licensed sources such as FMA [32] and NSynth [41] and shown with their license and attribution; GTZAN clips, which are excerpts of commercial recordings, are not bundled. The reference checkpoints have their own licenses, for example the non-commercial license of MERT [20], which is compatible with an academic demonstration but is stated on the About page.

4.10.4 Misinterpretation Risks

Visitors may read a single prediction as evidence that one model is better than another. The dashboard states on the Demo and About pages that single-clip predictions are illustrations, and the Results tab shows only the scores of the full evaluation, identical to those of the report.

Table 4.9: Summary of risks of the dashboard with their likelihood, impact and mitigation.

Risk	Likelihood	Impact	Mitigation
Checkpoints or probes of A* are late	Medium	High	Develop with A0 first; add A* by import
Preprocessing differs from evaluation	Medium	High	Import the evaluation code; check against cached embeddings
GPU unavailable at a presentation	Low	Medium	CPU fallback; models loaded at start-up
Uploaded files fail to decode	Medium	Low	Type, size and decoding checks with clear messages
Dashboard work delays the research	Medium	Medium	Minimal scope; incremental development
Unsuitable licenses of sample clips	Low	High	Only openly licensed clips with attribution; no GTZAN clips
Single predictions taken as results	Medium	Medium	Disclaimers; Results tab shows only evaluation scores

4.11 Conclusion

This chapter specified the M2D Music Dashboard, the small web application that accompanies the study. It presents the results of the further pre-training experiments and demonstrates four MARBLE tasks, namely genre, tagging, instrument and pitch, on clips that a visitor uploads or selects, with a side-by-side comparison of the original and the further pre-trained model. The requirements favor simplicity and faithfulness: there are no accounts, a single page with three tabs, a FastAPI backend that reuses the evaluation code and probes, a plain HTML and JavaScript frontend, and a small SQLite database whose results are imported unchanged from the evaluation. The use cases, wireframes, database design and risk analysis provide the basis for the design in the next chapters, while the research method that produces the models and results is described in Chapter 5.

Chapter 5 Proposed Approach and Methodology

This chapter describes the proposed approach at the level of algorithms and procedures. The study asks whether a general-purpose audio model trained by masked latent prediction can compete with music-specific foundation models after further pre-training on a small amount of music data. Figure 5.1 gives an overview. The approach has four parts: a data pipeline that acquires a 10,000-track subset of the Free Music Archive (FMA) and converts it into the exact input format of the pre-trained model; a further pre-training framework, called M2D-X-Music in this report, that combines the denoising and regularization recipe of M2D-X with a MERT-style musical teacher; an experimental design that isolates the contribution of each component and of the amount of data under a single-GPU budget; and an evaluation protocol that measures gains on music tasks, retention of general-audio ability, and the remaining gap to music-specific reference models. Section 5.1 first summarizes the literature that motivates each design choice. No results are reported in this chapter: it specifies what will be run and how the outcome will be judged.

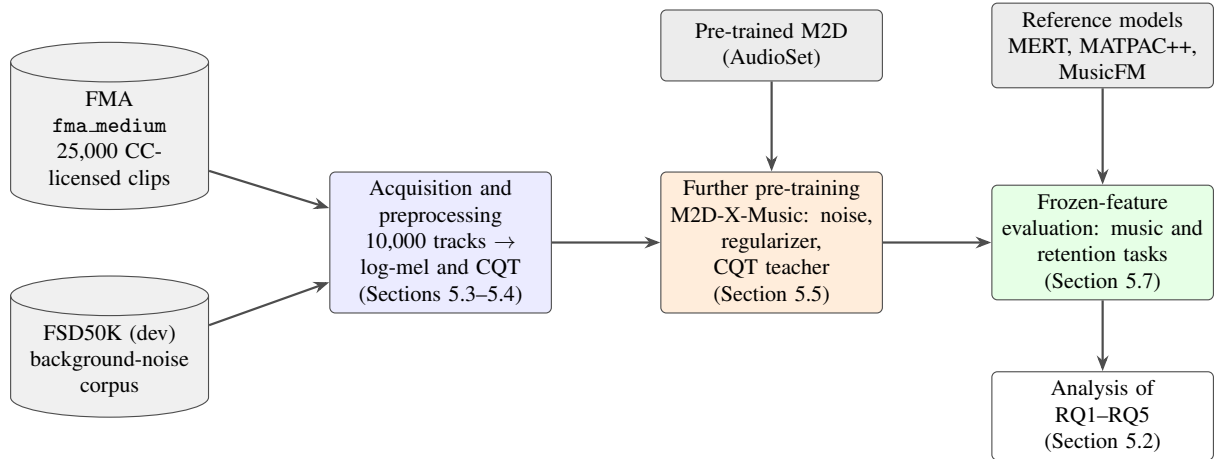


Figure 5.1: Overview of the proposed methodology. A 10,000-track subset of FMA and a background-noise corpus are converted into the input format of the pre-trained M2D model. The model is further pre-trained with the M2D-X-Music objective. The resulting checkpoints and the reference models are then compared on music and retention tasks with frozen features.

5.1 Methodological Foundations

This section reviews the work on which the methodology builds and states, for each source, what is adopted and what is deliberately left out. Table 5.1 at the end of the section collects these decisions.

5.1.1 Masked Latent Prediction and M2D

Masked prediction teaches an encoder to infer hidden parts of an input from the visible parts. Masked autoencoders reconstruct the hidden input values, first for images [53] and then for audio spectrograms [1],

so the learning signal depends partly on low-level detail. Methods that predict targets produced by a separate network instead avoid this dependence: BYOL [54], data2vec [55] and I-JEPA [9] regress the continuous latents of a slowly updated target network, and BEATs [2] predicts discrete labels from an acoustic tokenizer. Masked Modeling Duo (M2D) [4, 5] applies this idea to audio spectrograms with one distinguishing choice: the target network encodes *only the masked patches*, so the target does not carry information about the visible patches that the online network could exploit, and both networks are encouraged to model the input.

The released M2D model [46] takes a log-mel spectrogram of 6 s of audio at 16 kHz with 80 mel bins (80×608 frames) and splits it into 16×16 patches, giving $N_F = 5$ frequency positions and $N_T = 38$ time positions, that is, $N = 190$ patches. A vanilla ViT-Base encoder [56] with 768-dimensional tokens and about 86 M parameters encodes the visible 30 % of the patches. A shallow transformer predictor, taken from the MAE decoder, receives the encoded visible patches together with shared learnable mask tokens and predicts the representations of the masked patches. The target encoder is an exponential moving average (EMA) of the online encoder. The model was pre-trained on AudioSet [30] (2,005,132 clips, 5,569 h) for 300 epochs with an effective batch size of 2,048 [5], and it achieved top-level linear-evaluation results across environmental, speech and music tasks. Related members of the family add an unsupervised classification objective (MATPAC [12]) or multiple prediction hypotheses (MATPAC++ [14]), and M2D-CLAP aligns the representation with text [7]. M2D is chosen as the base model of this study for three practical reasons: the code and the AudioSet weights are public [46], an official recipe for further pre-training exists, and its ViT-Base encoder is small enough for a single 16 GB GPU.

5.1.2 Specializing a General Model: M2D-X

M2D-X extends M2D to learn representations specialized to an application X whose data are scarce and differ from the pre-training distribution [5]. It adds three ingredients. First, background noise from another corpus is mixed into the input of M2D. This acts as data augmentation, which matters because masked prediction normally needs large datasets [57], and, because an additional network sees the clean signal, it forms a denoising task. Second, an *offline network* receives the clean input and provides an additional training signal through a configurable loss, which can be supervised labels, the features of a domain model (distillation), or the features of the original pre-trained model (regularization). Third, the total loss is a weighted sum of the M2D loss and the offline loss. The earlier speech variant, M2D for Speech, distilled HuBERT features while denoising and reached results close to HuBERT on several speech tasks [6, 58].

The evidence most relevant to this project comes from the ICBHI 2017 respiratory-sound task, where

only 539 training recordings (about 5.5 h for the whole database) were available [5]. The AudioSet model (a variant with 16×4 patches and 2-s inputs) scored 61.93 on the task metric. Further pre-training with the M2D loss alone and no background noise *lowered* the score to 61.17, indicating overfitting to the small corpus. Adding background noise (mixing ratio 0.3) raised it to 62.73, and the full M2D-X, which adds the frozen-teacher regularizer, reached 63.29. The training list was replicated to 10,000 samples per epoch, 600 epochs were run with an effective batch size of 128 on a single 24 GB GPU, with the stated aim of completing pre-training within a day. The authors’ guide [46] states that this procedure suits application datasets of fewer than about 10,000 samples and that an effective batch size of 128 with 600 epochs worked for 10,000 samples. The 10,000-track budget of this project therefore sits at the upper end of the regime the recipe was designed for, which makes it a natural, low-risk starting point.

5.1.3 Music-Specific Self-Supervised Models

MERT. MERT [17] follows HuBERT-style masked prediction on raw audio with two teachers. The *acoustic teacher* supplies discrete targets, obtained either by K-means on log-mel and chroma features or from the residual vector quantization codes of EnCodec [59]. The *musical teacher* is a linear head on the encoder output that regresses the constant-Q transform (CQT) [60] of the audio with a mean-squared-error loss, and the total loss is $\alpha \mathcal{L}_H + \mathcal{L}_{\text{CQT}}$. The CQT has logarithmically spaced bins, so every octave has the same number of bins, which suits pitch and harmony. Adding the CQT term raised key-detection accuracy from 55.1 to 65.0 and GTZAN genre accuracy from 75.2 to 78.6 for the K-means teacher, and from 60.5 to 63.2 and from 72.8 to 77.2 for the RVQ teacher [17]. MERT also mixes short excerpts from the same batch into each clip (in-batch noise mixup) with probability 0.5 in its final setting. Its official implementation computes the CQT magnitude with nnAudio [47] using 84 bins at 12 bins per octave starting at 32.7 Hz [61], which is the configuration adopted here for the musical teacher. The base model was trained on about 1,000 h of music in the paper and on 160,000 h for the 330 M-parameter model [17]; the public MERT-v1-95M checkpoint used in this study was trained on 20,000 h [20].

MusicFM and MuQ. MusicFM [21] adapts the BEST-RQ idea [22] to music: targets are produced by a frozen random projection and a random codebook applied to normalized log-mel features, and a Conformer is trained to predict them. A variant trained on the whole FMA (about 8,000 h) scored below the variant trained on a 160,000-h in-house corpus on most downstream tasks, which the authors attribute to scale and to the crowd-sourced nature of FMA [21]. MuQ [23] predicts Mel-RVQ tokens and reports that it outperforms earlier self-supervised music models with only about 900 h of open-source pre-training data. These two models show that the FMA corpus is usable but not ideal, and that the

choice of target matters more than its sophistication; they are used as reference models and their target definitions are not adopted, because both require pre-training from scratch.

MATPAC and MATPAC++. MATPAC combines masked latent prediction with a DINO-style unsupervised classification objective [13], with total loss $(1 - \alpha)\mathcal{L}_{\text{cls}} + \alpha\mathcal{L}_{\text{pred}}$ and $\alpha = 0.5$ [12]. MATPAC++ replaces the single prediction by several hypotheses selected with annealed multiple choice learning [62, 14]. The finding that matters here is the effect of domain-specific training. When MATPAC++ was trained only on music (about 0.9 M Million Song Dataset samples plus about 0.18 M web-radio samples, roughly 30 h on four H100 GPUs for 6-s segments), its average over eight music tasks rose from 47.2 to 47.9 compared with training on AudioSet, a gain of 0.7 points [14]. In the same table the AudioSet-trained M2D averages 46.3 against 44.2 for MERT, although the MERT scores were taken from its paper and the protocols are not identical. In short, general-audio models are already strong on music tasks, and large-scale music-only training adds little. This is the observation that motivates studying whether a *small, cheap* music-specific step can recover a comparable increment.

5.1.4 Further Pre-training, Forgetting and Regularization

Continuing self-supervised pre-training on domain data is an established idea. In language modeling, domain-adaptive pre-training improves downstream tasks in both high- and low-resource settings [24]. In speech, adding unlabeled in-domain data to pre-training reduces most of the gap caused by domain shift [25]. The risks are equally well documented. Vision transformers further pre-trained on small target data overfit, which motivates distilling from a teacher that was itself further pre-trained [26]. For audio, SONAR continues pre-training BEATs on domain data and compares it with direct continual pre-training that resumes the original objective unchanged [27]. On FMA-derived data of 30,000 to 50,000 segments, direct continuation reduced the frozen-feature GTZAN F1 score from 86.0 to 76.0 and lowered the AudioSet tagging mAP from 34.8 to 14.7, whereas SONAR, which adds distillation-based regularization and data sampling, raised GTZAN to 88.0 and kept the mAP at 34.7 [27]. Another study with M2D found that, for narrow and small respiratory datasets, models pre-trained on diverse general audio beat models trained on the domain alone, and that further pre-training on a combination of general and domain data improved results [28]. The frozen-teacher term of M2D-X is closely related to learning without forgetting [63] and knowledge distillation [64], and it avoids the parameter-importance estimates required by elastic weight consolidation [65]. These studies jointly imply that naive continuation should be treated as a control that may fail, and that retention of general ability must be measured, not assumed.

Table 5.1: Literature findings and the design decisions they lead to.

Source	Finding used	Decision in this study
M2D, M2D-X [4, 5]	Latent prediction of masked patches; public AudioSet weights and further pre-training code; recipe for fewer than about 10,000 samples	Start from the AudioSet checkpoint with 80×608 input, 16 × 16 patches and masking ratio 0.7; keep its loss as the base objective
M2D-X noise and regularizer [5]	Naive further pre-training degraded the score; noise and a frozen original-model teacher recovered and exceeded it	Use background noise ($\eta = 0.3$) and the original M2D as frozen offline teacher; keep naive and partial variants as controls
MERT [17]	A CQT-regression musical teacher improves key detection and genre classification; in-batch mixup helps the RVQ variant	Add a CQT teacher (84 bins) through the offline branch with weight 1; do not train an acoustic tokenizer
MATPAC, MATPAC++ [12, 14]	Music-only pre-training at scale adds about 0.7 points over AudioSet training; public checkpoints	Use their checkpoints as references; do not add the classification head or multiple hypotheses
MusicFM, MuQ [21, 23]	FMA-only training trails a larger corpus; about 900 h of open data is enough for competitive results	Use as references with matched or larger data; do not adopt quantizer targets
Domain-adaptive pre-training, SONAR [24, 27, 26]	Naive continuation can hurt and forget; regularization and replay of general data help	Measure retention on environmental-sound tasks and report a forgetting score
FMA [32]	Creative Commons audio; 30-s clips; artist-filtered train, validation and test splits; curated medium subset	Sample from the training split of the medium subset only; cap tracks per artist; use no genre labels for pre-training
EVAR, MARBLE, HEAR [46, 45, 43, 44]	Frozen-feature linear probing is the common protocol for comparing audio representations	Evaluate every model with the same linear-probe protocol

5.1.5 Design Decisions Derived from the Literature

Table 5.1 summarizes, source by source, which finding is used and which decision follows from it in this study.

5.2 Research Questions and Hypotheses

The study is organized around five questions. Each is paired with a hypothesis that can be contradicted by the planned measurements. The arms and metrics named below are defined in Sections 5.6 and 5.7.

- **RQ1.** Does further pre-training of the AudioSet-pretrained M2D on about 83 h of FMA music improve its frozen-feature performance on music tasks? *H1:* the full recipe (A4) improves the average music score over the unmodified model (A0), whereas naive continuation (A1) does not.
- **RQ2.** Which parts of M2D-X are needed on music data? *H2:* the order of the music score is $A1 < A2 < A3$, that is, background noise and the frozen-teacher regularizer each add a gain, as in the respiratory-sound study.

- **RQ3.** Does a CQT musical teacher add value beyond M2D-X? *H3:* A4 exceeds A3 on pitch-related tasks (NSynth pitch) by more than the probe-seed uncertainty, and is not worse elsewhere.
- **RQ4.** How much of the gap to music-specific models does the cheap step close, and is general-audio ability kept? *H4:* the best arm closes a clearly positive fraction of the gap to the public MERT checkpoints and loses less than 1 point of average environmental-sound accuracy (a tolerance fixed in advance), while a gap to the larger, longer-trained references may remain.
- **RQ5.** How does the benefit depend on the number of FMA tracks? *H5:* the gain increases from 1,000 to 10,000 tracks with diminishing returns, so that most of it is reached before 10,000 tracks.

An outcome contradicting a hypothesis, such as no benefit of the CQT teacher or a collapse of retention, is a valid finding and will be reported as such.

5.3 Dataset Acquisition

This section describes where the pre-training audio comes from, which part of it is eligible, and how the 10,000-sample subset is drawn. Throughout this report a *sample* is one 30-s FMA clip, that is, one track, so 10,000 samples correspond to 10,000 tracks or about 83.3 h of music. This is about 1.5 % of the 5,569 h of AudioSet on which M2D was trained.

5.3.1 Choice of Corpus

The pre-training corpus must satisfy five requirements: it can be obtained and used legally for research; it provides full-quality audio, not only derived features or links; its metadata identify artists and provide predefined splits, so that leakage can be controlled; it is small enough to download and process on one workstation; and it has been used in earlier music self-supervised work, so that results can be compared. The Free Music Archive (FMA) [32] meets all five requirements. Its audio is licensed under Creative Commons licenses chosen by the artists, the tracks are MP3 files, most of them stereo at 44.1 kHz and 320 kbit/s (263 kbit/s on average), and the metadata include artist identifiers and an official split with an artist filter. FMA has been used to pre-train MusicFM [21], it is part of the data of the public MERT-v0-public model according to its model card [20], and FMA-derived data were used for continual pre-training of BEATs [27]. The alternatives are less suitable: the Million Song Dataset does not distribute audio, AudioSet consists of links to online videos whose availability changes, which already makes published results hard to compare [14], and MagnaTagATune (32 kbit/s, 16 kHz mono) and GTZAN (about 1,000 clips with documented faults [39]) are too small or too low in quality for pre-training and are used only as evaluation sets.

5.3.2 Archives and Metadata

FMA is distributed as one metadata archive (`fma_metadata.zip`) and four audio archives that are nested subsets of one another [32, 33]. The metadata table `tracks.csv` describes all 106,574 tracks, with artist, album, genre, license, and the columns that give the subset and the official training, validation and test split of each track. In the medium, small and large subsets the audio of a track is a 30-s clip cut from the middle of the full track, or the whole track if it is shorter. The medium subset contains 25,000 clips with a single top-level genre each, selected according to the completeness of their metadata and their popularity in the hope of selecting tracks of higher quality; the small subset is the balanced selection of 1,000 clips from each of the eight most popular genres of the medium subset [32]. The splits are 80 %/10 %/10 % for training, validation and test, stratified by genre, with an artist filter so that every artist appears in only one split, and a track keeps the same split in every subset [32].

Table 5.2: FMA audio archives and their use in this study. Sizes are those listed by the dataset maintainers [33]; clip counts and genre counts are from the dataset paper [32].

Archive	Clips	Genres	Size (GiB)	Use in this study
<code>fma_small</code>	8,000	8 (balanced)	7.2	Audio contained in the medium archive; official test split used for in-domain genre evaluation
<code>fma_medium</code>	25,000	16 (unbalanced)	22	Source of the pre-training pool (training split only) and of the <code>fma_small</code> evaluation clips
<code>fma_large</code>	106,574	161	93	Not used (larger download, no additional split advantage)
<code>fma_full</code>	106,574	161	879	Not used (untrimmed tracks)

The study downloads the metadata archive and the medium audio archive only, verifies the SHA-1 checksums published with the dataset, and unpacks the files. Because the small subset is contained in the medium subset, the single audio archive also supplies the clips for the in-domain evaluation.

5.3.3 Eligibility Rules and Subset Selection

A track enters the candidate list only if it satisfies all of the following rules.

1. It belongs to the small or medium subset and to the *training* split. Since the official splits are artist-filtered, no artist from the validation or test split contributes to pre-training, which prevents in-domain leakage into the FMA evaluation task. About 80 % of the 25,000 clips, that is, about 20,000 tracks, are expected to qualify; the exact count is recorded when the metadata are read.
2. Its license field is present in the metadata.
3. It passes the audio audit described in Section 5.4.
4. At most $k = 5$ tracks of the same artist are admitted. This limits the influence of prolific artists in

a corpus with a long-tailed artist distribution.

No genre label is used for selection, so the subset keeps the genre distribution of the medium subset, in which experimental music is under-represented [32]; the genre histogram of the chosen subset is reported so that this property is visible. Pre-training itself uses no labels at all.

Algorithm 1 fixes the selection. A seeded random permutation of the eligible tracks defines the order in which candidates are examined. The first 10,000 accepted tracks form the main subset, and shorter prefixes of the same list form the nested subsets with 1,000, 2,500 and 5,000 tracks, plus a 20,000-track extension if time allows. Using prefixes of one list means that all arms and all sizes draw on the same data, so a difference between two results cannot come from a different selection of tracks. Only the ordered list of track identifiers and the seed need to be stored to reproduce the subset.

Data: metadata table T ; seed s ; artist cap k ; sizes $n_1 < n_2 < \dots < n_J$

Result: ordered list L and nested subsets $S_j = L[1..n_j]$; rejection log

$E \leftarrow \{t \in T : t.\text{subset} \in \{\text{small}, \text{medium}\}, t.\text{split} = \text{training}, t.\text{license} \neq \emptyset\}$;

$\pi \leftarrow$ random permutation of E generated with seed s ;

$L \leftarrow$ empty list; $\text{count}[a] \leftarrow 0$ for every artist a ;

foreach track t in π , **in order do**

if $\text{count}[t.\text{artist}] < k$ **and** $\text{Audit}(t)$ passes **then**

 append t to L ;

$\text{count}[t.\text{artist}] \leftarrow \text{count}[t.\text{artist}] + 1$;

end

if $|L| = n_J$ **then** leave the loop;

end

return L , the subsets S_j , and the log of rejected tracks with their reasons;

Algorithm 1: Seeded, artist-capped selection of nested pre-training subsets

5.3.4 Background-Noise Corpus

The denoising component of M2D-X needs a corpus of background sounds. The official recipe uses the FSD50K dataset [31, 5], which contains 51,197 Freesound clips (108.3 h) under Creative Commons licenses, labeled with 200 classes of the AudioSet ontology. Only the development portion is used as noise, and the evaluation portion is left untouched. Because the ontology includes musical instruments, an optional ablation restricts the pool to clips that carry no music-related label, to test whether purely non-musical noise is a better distractor for music (Section 5.6).

5.3.5 Licensing and Ethics

FMA tracks carry artist-chosen Creative Commons licenses that permit research use. Audio and the spectrograms derived from it are kept local and are not redistributed; only the list of track identifiers and the seed are published. The metadata contain only public artist information and no personal data.

The CC-BY-NC license of the MERT weights [20] is respected by using them for non-commercial comparison only.

5.4 Data Preprocessing Pipeline

Figure 5.2 shows the pipeline. Its guiding rule is that the audio must reach the model through exactly the same front-end as in the original pre-training, because a pre-trained encoder that is fed differently computed spectrograms suffers a distribution shift that would be confounded with the effect under study.

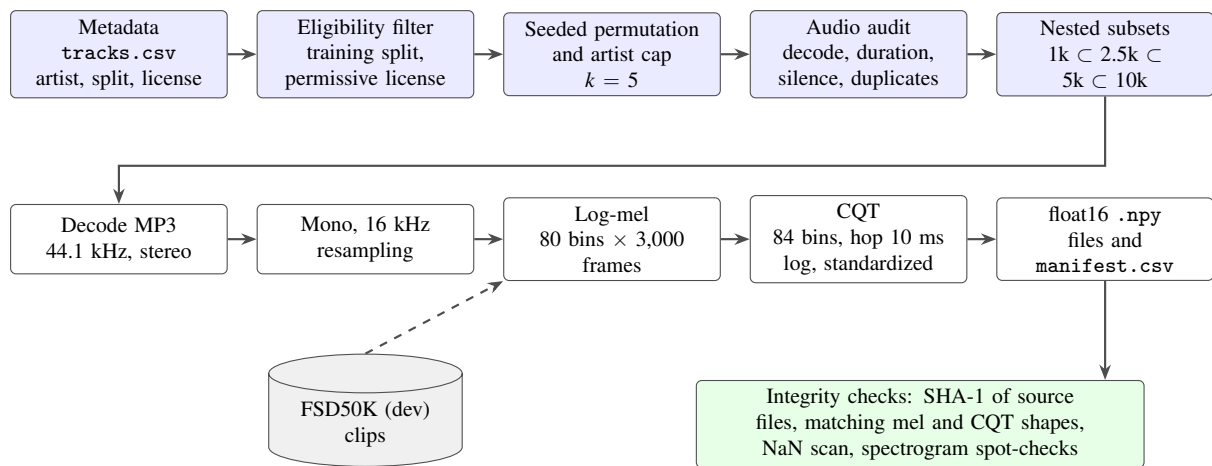


Figure 5.2: Data acquisition and preprocessing pipeline. The top row operates on metadata and audit results and fixes the ordered list of tracks. The bottom row converts the selected files with the same front-end that was used to pre-train M2D. The noise corpus passes through the log-mel stage only (dashed arrow).

5.4.1 Audit and Decoding

Each candidate MP3 file is decoded before it is accepted. The FMA maintainers document corrupted or truncated tracks, tracks shorter than 30 s, and duplicates [33], so the audit applies four rejection rules.

1. **Decoding failure:** the decoder reports an error or a truncated stream.
2. **Duration:** the decoded signal is shorter than 10 s, so that several different 6.08-s windows cannot be cut from it.
3. **Silence:** more than half of the frames have a power more than 60 dB below that of the loudest frame of the clip.
4. **Duplication:** the track has the same normalized pair of artist identifier and title as an accepted track, or the same hash of the decoded 16-kHz waveform. The track with the lowest identifier is kept.

Every rejection is logged with its reason, and the next candidate in the permutation takes the place of a

rejected track (Algorithm 1).

5.4.2 Front-End

The conversion script distributed with the M2D code (`wav_to_lms.py`) [46] is used without modification. It loads each file as a mono signal resampled to 16 kHz and computes a log-mel spectrogram with `nnAudio` [47]. With the mel filter bank H and the short-time Fourier transform X of the waveform, each spectrogram entry is

$$x_{f,t} = \log \left(\sum_k H_{f,k} |X_{k,t}|^2 + \varepsilon \right), \quad f = 1, \dots, 80, \quad (5.1)$$

where ε is a small constant that avoids the logarithm of zero. Table 5.3 lists all parameters. Spectrograms are stored in half precision, which halves the disk space (about 4.8 GB for 10,000 clips of 3,000 frames) at a rounding error far below the standard deviation of the data, and they are cast to single precision when loaded.

Table 5.3: Front-end and target parameters. Values of the log-mel front-end and of the input geometry follow the released M2D model [46, 5]; the CQT settings follow the official MERT implementation [61], with the hop size changed to match the log-mel frame grid.

Item	Value
Audio decoding	Channel mean to mono, resampled to 16 kHz
STFT	Window and FFT size 400 samples (25 ms), hop 160 samples (10 ms)
Mel bands	80 bands between 50 Hz and 8,000 Hz
Input window	608 frames (≈ 6.08 s); patches of 16×16 ; $N_F = 5$, $N_T = 38$, $N = 190$ patches
Normalization	Mean $\mu_{AS} = -7.1$ and standard deviation $\sigma_{AS} = 4.2$ of the AudioSet pre-training data
CQT target	Magnitude, 84 bins at 12 per octave from 32.7 Hz (C1) to about 3.95 kHz, hop 160 samples; logarithm, standardized per bin
Target token rate	Average over $P = 16$ frames (160 ms) to match one patch step

5.4.3 Cropping and Normalization

At every visit, a random window of 608 frames is cut from the stored 30-s spectrogram, as in the official loader. A 30-s track contains about five non-overlapping windows, and random start positions give many more overlapping ones, so the 10,000 tracks supply a large number of distinct inputs even though the track count is small. The window is standardized as

$$\bar{x} = \frac{x - \mu_{AS}}{\sigma_{AS}}, \quad (5.2)$$

with the AudioSet statistics of Table 5.3, so that the pre-trained encoder initially sees inputs of the same scale as during its original pre-training. The official M2D-X code instead recomputes these statistics

from the application data at run time. The deviation is deliberate: it removes one source of change at the start of training. The mean and standard deviation of the selected FMA subset are computed and reported so that the size of the shift is known, and training with the FMA statistics is kept as an optional ablation.

5.4.4 Musical-Teacher Targets

The musical teacher uses the constant-Q transform [60], whose center frequencies are spaced logarithmically,

$$f_k = f_{\min} 2^{(k-1)/12}, \quad k = 1, \dots, 84, \quad f_{\min} = 32.7 \text{ Hz}, \quad (5.3)$$

so that every semitone has its own bin and the top bin lies at about 3.95 kHz, below the Nyquist frequency of 8 kHz. Let $C_{k,t}$ be the CQT magnitude computed with nnAudio on the same 16-kHz mono waveform and with the same hop of 160 samples as the log-mel spectrogram. The target is the log magnitude standardized per bin with the mean m_k and standard deviation s_k over the selected subset, followed by average pooling over the $P = 16$ frames of each token time step τ :

$$\tilde{c}_{k,t} = \frac{\log(C_{k,t} + \epsilon) - m_k}{s_k}, \quad \bar{c}_{k,\tau} = \frac{1}{P} \sum_{t=(\tau-1)P+1}^{\tau P} \tilde{c}_{k,t}. \quad (5.4)$$

The choices differ from MERT in three respects, each for a reason. MERT regresses the linear magnitude on masked frames at its own frame rate; here the logarithm and the per-bin standardization keep the target on a scale comparable to the log-mel input and avoid a regression dominated by a few loud low-frequency bins, and the pooling matches the 160-ms time step of an M2D token. The CQT is deterministic and needs no additional network, and nnAudio is already a dependency of the M2D front-end. Because the two transforms use the same hop and centered frames, the log-mel and CQT arrays of a clip have the same number of frames; if they differ by a frame because of padding, both are trimmed to the shorter, and one random start index is used to crop both.

5.4.5 Storage, Manifest and Integrity Checks

For each accepted track the pipeline writes one half-precision array for the log-mel spectrogram (about 4.8 GB for 10,000 clips) and one for the CQT (about 5.0 GB), and a row in a manifest file with the track identifier, artist identifier, file paths, frame counts, SHA-1 of the source file, audit outcome and split. The noise corpus is converted to log-mel arrays only. The following checks are run and their outcome is saved: the SHA-1 of every source file matches the published checksum; log-mel and CQT frame counts agree; no array contains NaN or infinite values; value ranges lie within a plausible interval of the normalized scale; and a gallery of 20 randomly chosen spectrograms is inspected by eye against

the audio. The conversion is resumable and driven by one configuration file, so an interrupted run does not repeat finished tracks.

5.5 Further Pre-training Framework

This section defines M2D-X-Music, the objective and the training procedure applied to the AudioSet-pretrained M2D. Figure 5.3 shows the data flow.

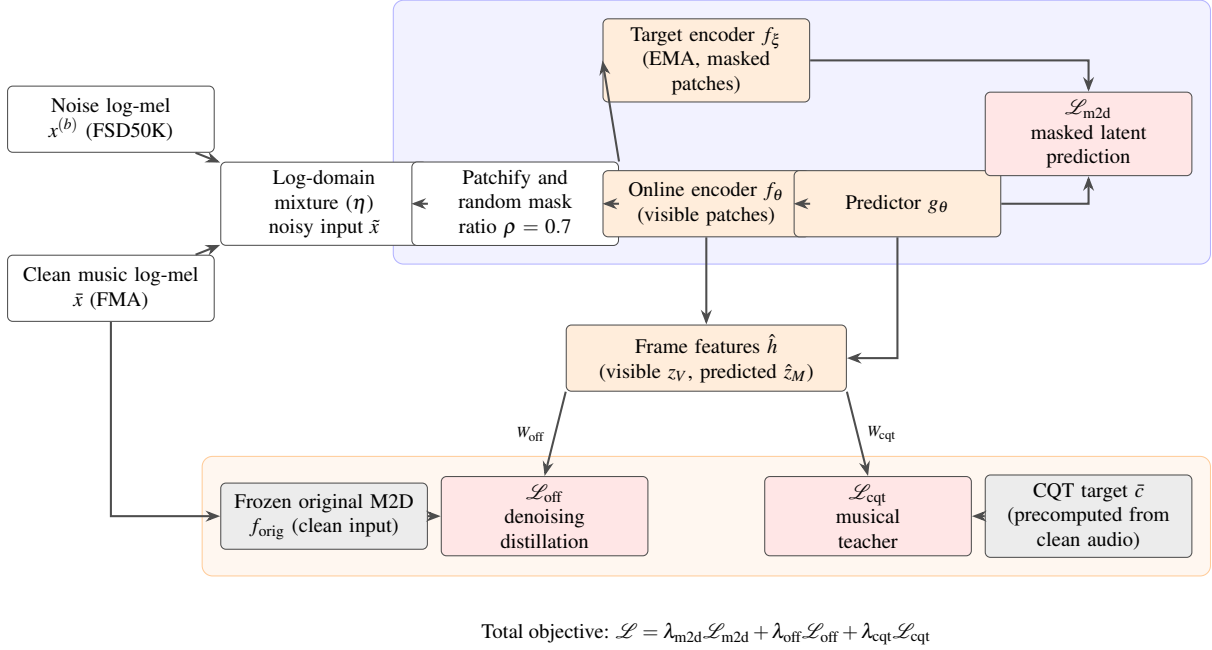


Figure 5.3: Architecture of the further pre-training framework. The online and target networks receive the noisy input (top region), whereas the offline branch receives the clean input and its precomputed targets (bottom region). The frame features $\hat{h}$ are shared by the two offline losses through separate linear mappers W_{off} and W_{cqt} . Only the online encoder f_{θ} is kept after pre-training.

5.5.1 Overview, Initialization and Alternatives Considered

Training starts from the released checkpoint M2D/0.7 (80×608 input, 16 × 16 patches, masking ratio 0.7) [46]. The online encoder f_{θ} , the predictor g_{θ} and the EMA target encoder f_{ξ} are all initialized from this checkpoint, and a frozen copy f_{orig} of the original encoder serves as offline teacher. Two linear mappers, W_{off} and W_{cqt} , are newly initialized. The online and target networks see the *noisy* input, whereas the offline branch sees the *clean* input and its precomputed targets. After pre-training only the online encoder is kept; its output features are what the evaluation of Section 5.7 uses.

Four alternatives were considered and not adopted. Pre-training on FMA from scratch would answer a different question and is far beyond the single-GPU budget. Restricting updates to adapter or low-rank parameters would reduce forgetting but changes the model the study sets out to compare, and it is better left to later work. Distilling the features of a frozen MERT model is the exact analogue of the

HuBERT distillation of M2D for Speech, but it needs a second, 24-kHz raw-audio branch and a second data format; it is kept as an optional extension. Replacing the objective by quantizer targets in the style of MusicFM or MuQ would require training the whole model again.

5.5.2 Masked Latent Prediction on Noisy Music

The standardized window is cut into $N = N_F N_T = 190$ patches. A random subset $\mathcal{M}$ of $|\mathcal{M}| = \text{round}(\rho N) = 133$ patches is masked with $\rho = 0.7$, and the remaining 57 patches form the visible set $\mathcal{V}$. The online encoder maps the visible patches to $z_{\mathcal{V}} = f_{\theta}(\tilde{x}_{\mathcal{V}})$, and the predictor, given mask tokens m and positional encodings p , predicts all token representations,

$$\hat{z} = g_{\theta}(\text{concat}(z_{\mathcal{V}}, m) + p), \quad (5.5)$$

of which only the masked ones, $\hat{z}_{\mathcal{M}}$, are used in the loss. The target encoder receives the masked patches only, $z_{\mathcal{M}} = f_{\xi}(\tilde{x}_{\mathcal{M}})$, and its output is standardized to zero mean and unit variance, giving $\tilde{z}_{\mathcal{M}}$. The loss is the mean squared error between the ℓ_2 -normalized prediction and target,

$$\mathcal{L}_{\text{m2d}} = \frac{1}{|\mathcal{M}|} \sum_{j \in \mathcal{M}} \left\| \frac{\hat{z}_j}{\|\hat{z}_j\|_2} - \frac{\tilde{z}_j}{\|\tilde{z}_j\|_2} \right\|_2^2 = \frac{1}{|\mathcal{M}|} \sum_{j \in \mathcal{M}} \left(2 - 2 \frac{\langle \hat{z}_j, \tilde{z}_j \rangle}{\|\hat{z}_j\|_2 \|\tilde{z}_j\|_2} \right). \quad (5.6)$$

No gradient flows into the target encoder. Its parameters follow the online encoder by an exponential moving average,

$$\xi \leftarrow \tau_k \xi + (1 - \tau_k) \theta, \quad \tau_k = \tau_0 + (\tau_1 - \tau_0) \frac{k}{K}, \quad (5.7)$$

where k is the optimizer step, K the total number of steps, $\tau_0 = 0.99995$ and $\tau_1 = 0.99999$.

5.5.3 Background Noise and Denoising

To build the noisy input, a window of the noise corpus is mixed with the music window in the power domain, after the mean log level of each corpus has been removed so that the two sources start at comparable levels. With the music log-mel window $x^{(m)}$, the noise window $x^{(b)}$, their corpus means μ_m and μ_b , and the mixing ratio η ,

$$x_{\text{mix}} = \log \left((1 - \eta) \exp(x^{(m)} - \mu_m) + \eta \exp(x^{(b)} - \mu_b) \right), \quad \tilde{x} = \frac{x_{\text{mix}}}{\sigma_{\text{AS}}}, \quad (5.8)$$

which is the mixing rule of M2D-X [5] followed by the same scale normalization as in Equation 5.2. Here μ_m is the AudioSet mean μ_{AS} of Table 5.3 and μ_b is the mean of the noise corpus. The noise windows are drawn without replacement by cycling through a shuffled list of the corpus, and short noise clips are repeated to fill the window. The ratio is fixed at $\eta = 0.3$ in the arms that use noise, the value of

the respiratory-sound recipe [5]. Without a second view, the noise would only be an augmentation; the denoising character arises because the offline losses below are computed against the *clean* signal, so the encoder is asked to produce features that describe the music underneath the noise.

5.5.4 Frozen-Teacher Regularization

Both offline losses act on frame-level features. Let $\hat{Z}$ be the grid of $N_F \times N_T$ token representations in which the visible positions hold the encoder outputs $z_{\mathcal{V}}$ and the masked positions hold the predictions $\hat{z}_{\mathcal{M}}$. The feature of time step τ is the concatenation over the N_F frequency positions,

$$\hat{h}_\tau = \text{concat}(\hat{Z}_{1,\tau}, \dots, \hat{Z}_{N_F,\tau}) \in \mathbb{R}^{N_FD}, \quad N_FD = 5 \times 768 = 3840, \quad (5.9)$$

which is the same layout as the 3,840-dimensional frame features used in the linear evaluation of M2D [46]. The frozen original encoder f_{orig} processes the *clean* window with all patches visible and gives the target features $y_\tau^{\text{off}} \in \mathbb{R}^{3840}$. The regularization loss compares them after a linear map,

$$\mathcal{L}_{\text{off}} = \frac{1}{N_T} \sum_{\tau=1}^{N_T} \left\| \frac{W_{\text{off}} \hat{h}_\tau + b_{\text{off}}}{\|W_{\text{off}} \hat{h}_\tau + b_{\text{off}}\|_2} - \frac{y_\tau^{\text{off}}}{\|y_\tau^{\text{off}}\|_2} \right\|_2^2. \quad (5.10)$$

This term anchors the features of the adapted model to those of the original model on clean inputs, in the manner of learning without forgetting [63], while the first term moves them towards the music distribution. In the speech variant the same mechanism distils a domain model; here the teacher is the model itself, so the term is a regularizer, as in the respiratory-sound experiments [5].

5.5.5 Musical Teacher

The musical teacher adds a second offline task on the same frame features. A linear mapper predicts the pooled CQT target of Equation 5.4,

$$\mathcal{L}_{\text{cqt}} = \frac{1}{N_T K_c} \sum_{\tau=1}^{N_T} \|W_{\text{cqt}} \hat{h}_\tau + b_{\text{cqt}} - \bar{c}_\tau\|_2^2, \quad K_c = 84. \quad (5.11)$$

Because $\hat{h}_\tau$ is built from encoder outputs at visible positions and from predictor outputs at masked positions, the term has two roles at once: it asks the encoder to recover the clean pitch content from noisy visible patches, and it asks the predictor to infer the pitch content of hidden regions. This differs from MERT, where the CQT is regressed on masked positions of a single network, and it keeps the M2D-X convention that offline losses act on all frames. Since the target is standardized, the initial value of the loss is of the order of one, comparable to the other two terms, and the weight $\lambda_{\text{cqt}} = 1$ follows MERT’s finding that a unit weight worked best among 1, 2 and 5 [17].

5.5.6 Joint Objective and Optimization

The full objective is the weighted sum

$$\mathcal{L} = \lambda_{\text{m2d}} \mathcal{L}_{\text{m2d}} + \lambda_{\text{off}} \mathcal{L}_{\text{off}} + \lambda_{\text{cqt}} \mathcal{L}_{\text{cqt}}. \quad (5.12)$$

The experimental arms of Section 5.6 differ only in η , λ_{off} and λ_{cqt} ; all other settings are identical and are listed in Table 5.4. They follow the official M2D-X recipe [46, 5] with AdamW [66]. The learning rate is scaled with the effective batch size B_{eff} by the rule of the code base,

$$\text{lr} = \text{blr} \cdot \frac{B_{\text{eff}}}{256} = 3 \times 10^{-4} \cdot \frac{128}{256} = 1.5 \times 10^{-4}. \quad (5.13)$$

The effective batch size is kept at 128 even though the micro-batch is smaller, because the EMA update is applied once per optimizer step, so the number of steps affects the target network; the guide of the authors stresses that the effective batch size matters [46]. Algorithm 2 gives the procedure.

Table 5.4: Fixed settings of the further pre-training runs. Settings in the upper part follow the released M2D model and the M2D-X recipe; the lower part are the choices made for this study.

Item	Setting
Initial weights	M2D/0.7 (m2d_vit_base-80x608p16x16-221006-mr7): online encoder, predictor and target encoder
Network	ViT-Base encoder, 768-dimensional tokens, about 86 M parameters; MAE-style predictor
Input and masking	80×608 log-mel window, 16 × 16 patches ($N = 190$), masking ratio 0.7 (57 visible, 133 masked)
Optimizer	AdamW with $\beta = (0.9, 0.95)$, weight decay 0.05, gradient-norm clipping at 3.0
Learning rate EMA decay	Base 3×10^{-4} scaled to 1.5×10^{-4} ; 24 warm-up epochs; cosine decay to zero Linear from 0.99995 to 0.99999 over training
Epochs and data per epoch	600 epochs of 10,000 virtual samples, that is, 6.0 M sample views and 46,875 optimizer steps
Batch	Micro-batch 32, gradient accumulation 4, effective batch 128
Noise	FSD50K development clips, $\eta = 0.3$ in arms with noise
Loss weights	$\lambda_{\text{m2d}} = 1$; $\lambda_{\text{off}} \in \{0, 1\}$; $\lambda_{\text{cqt}} \in \{0, 1\}$
Precision and hardware	Mixed precision on one RTX A4000 (16 GB)
Checkpoints	Saved every 100 epochs; epoch 600 is the reported checkpoint

5.5.7 Compute and Memory Budget

One run processes $600 \times 10,000 = 6.0$ million sample views, which is about 1 % of the roughly 600 million views of the original AudioSet pre-training (300 epochs of about two million clips). The respiratory-sound run of the authors, which had the same number of views, used one RTX 3090 Ti with 24 GB, 2-s inputs and 16×4 patches (250 patches per input), and aimed to finish pre-training within a day [5]. The

Data: initial weights (θ_0, ξ_0) ; frozen teacher f_{orig} ; music list $\mathcal{D}$ with log-mel and CQT arrays; noise list $\mathcal{B}$; η , λ_{m2d} , λ_{off} , λ_{cqt} ; epochs E ; epoch size n ; micro-batch b ; accumulation a

Result: further pre-trained encoder f_θ

$\theta \leftarrow \theta_0$; $\xi \leftarrow \xi_0$; initialize W_{off} , W_{cqt} ;

repeat $\mathcal{D}$ until the epoch list $\mathcal{D}_n$ has n entries;

for $e = 1$ **to** E **do**

 shuffle $\mathcal{D}_n$;

foreach *micro-batch of b clips* **do**

foreach *clip i in the micro-batch* **do**

 draw one start index; crop 608 frames of the log-mel array and of the CQT array at that index;

 take the next crop from the shuffled noise list $\mathcal{B}$ and form $\tilde{x}_i$ with Equation 5.8;

end

 draw random masks; compute $\mathcal{L}_{\text{m2d}}$ with Equations 5.5 and 5.6;

 build $\hat{h}$ (Equation 5.9); compute y^{off} from the clean window with f_{orig} without gradient;

 compute $\mathcal{L}_{\text{off}}$ and $\mathcal{L}_{\text{cqt}}$ (Equations 5.10 and 5.11); form $\mathcal{L}$ (Equation 5.12);

 backpropagate $\mathcal{L}/a$ into θ , W_{off} , W_{cqt} ;

if *a micro-batches have been accumulated* **then**

 clip the gradient norm; apply an AdamW step with the scheduled learning rate;

 update ξ with Equation 5.7;

end

end

if e is a multiple of 100 **then** save a checkpoint of f_θ ;

end

return f_θ ;

Algorithm 2: Further pre-training with the M2D-X-Music objective

window of this study has 190 patches, but the RTX A4000 has less memory and, to our knowledge, lower throughput, so the time per run is not known in advance: the budget assumes a run of the order of one day and is to be confirmed by a pilot of a few epochs before the main runs. The micro-batch of 32 is the setting that the authors give for smaller GPUs [46]. If it does not fit into 16 GB, the micro-batch is halved and the accumulation doubled, which leaves the effective batch size of 128 unchanged. If the throughput is lower than expected, the number of epochs is reduced for *all* arms alike, and the intermediate checkpoints saved every 100 epochs make the effect of the shorter schedule visible in the data.

5.6 Experimental Design

The experiments fall into three groups: reference models that are only evaluated, a small set of further pre-training *arms* that differ in one factor at a time, and runs that vary the number of FMA tracks. All pre-training runs are compute-matched, that is, each processes the same number of sample views.

5.6.1 Reference Models

Table 5.5 lists the models against which the further pre-trained checkpoints are compared. They are chosen to separate the effect of data volume from the effect of the objective. MERT-v1-95M and MERT-v0-public have a parameter count similar to that of the M2D encoder (95 M against 86 M) but were trained from scratch on about 20,000 h and about 900 h of music, which is roughly 240 and 11 times the 83 h of FMA used here, and MusicFM-FMA shows what training from scratch on the whole of FMA achieves. Reference models are never fine-tuned or further trained. For transformer references whose layers differ in quality, features of all layers are extracted and the best layer is chosen per task on the validation split; this is a favorable setting for the references and therefore a conservative one for the claims of this study. For the M2D family the final-layer features with the 3,840-dimensional frame layout of the original linear evaluation are used. Audio is resampled to the rate of each model and cut into windows of the context length of the model (5 s for MERT), and the segment embeddings are averaged over time.

Table 5.5: Reference models. Parameter counts and pre-training data are as reported in the cited sources; MATPAC++ values are from its preprint and the checkpoints are the ones published by its authors.

Model	Params	Pre-training data	Role
M2D/0.7 [5]	86 M	AudioSet, 5,569 h	Arm A0: starting point without FMA training
MERT-v1-95M [17, 20]	95 M	About 20,000 h of music; 24 kHz	Music-specific, same size, large data
MERT-v0-public [20]	95 M	About 900 h of open music (Music4All and part of FMA); 16 kHz	Music-specific, same size, small data
MusicFM (FMA) [21, 50]	About 330 M	FMA, about 8,000 h	Music-specific, same corpus family, larger model
MATPAC++ (music) [14, 49]	About 86 M	Million Song Dataset samples and web-radio samples, about 1.1 M in total	Music-specific model of the masked-latent-prediction family
MATPAC++ (general) [14, 49]	About 86 M	AudioSet	General counterpart of the previous row

5.6.2 Pre-training Arms

Table 5.6 defines the arms. The five arms A1 to A5 form a design in which each component is switched on or off, so that every claim about a component rests on a specific, pre-declared comparison rather than on a free search through results:

- A1 against A0: the effect of naive continuation of M2D on FMA (RQ1, RQ2).
- A2 against A1: the effect of background noise.

- A3 against A2: the effect of the frozen-teacher regularizer.
- A4 against A3: the effect of the musical teacher on top of M2D-X (RQ3).
- A5 against A2 and A4 against A5: the effect of the musical teacher without the regularizer, and of the regularizer when the teacher is present.
- S1 to S3 against the best of A3 and A4: the effect of the number of tracks (RQ5).

The best of A3 and A4, called A^* , is chosen by the mean music score on the *validation* splits only, so that the test splits take no part in any choice. If time is short, the arms are run in the order A3, A1, A4, A2, A5, then S1 to S3, then the optional runs.

Table 5.6: Pre-training arms. Dashes mark runs without FMA pre-training. All arms use the same track list, the same random seed for data order and masks, and 6.0 M sample views.

Arm	η	λ_{off}	λ_{cqt}	Tracks	Purpose
A0	—	—	—	—	Unmodified M2D/0.7 (reference point)
A1	0	0	0	10,000	Naive continuation with the M2D loss only
A2	0.3	0	0	10,000	Adds background noise
A3	0.3	1	0	10,000	M2D-X: noise and frozen-teacher regularizer
A4	0.3	1	1	10,000	M2D-X-Music: adds the CQT musical teacher
A5	0.3	0	1	10,000	Musical teacher without the regularizer
S1–S3	0.3	as A^*	as A^*	1,000; 2,500; 5,000	Data-size study with the best arm A^*

Optional runs, executed only if time remains, are: a noise pool without music-related FSD50K classes; a lower noise ratio $\eta = 0.1$; training with the FMA normalization statistics; a 20,000-track run with 300 epochs so that the number of sample views stays at 6.0 M; a second pre-training seed for A3 and A4; and distillation of frozen MERT features as a second domain teacher.

5.6.3 Controls and Reproducibility

Several measures keep the comparison honest. All arms share the track list of Algorithm 1 and the same seed for data order and masking. The reported checkpoint is the one at epoch 600, fixed before any test result is seen; checkpoints at epochs 100 to 500 are evaluated on the validation splits of the small tasks only (GTZAN, FMA-small, ESC-50, UrbanSound8K) to show the learning curve and to detect degradation or collapse, as in the checkpoint analysis of the authors [5]. The official training script also logs label-free feature statistics at fixed intervals, which provides an additional collapse indicator. The seeds, the configuration files, the track list and the manifest are stored with the code. If time permits, a second pre-training seed for A3 and A4 estimates the variance that comes from pre-training itself rather than from the probes.

5.7 Evaluation Protocol

Evaluation uses frozen features and a linear classifier, which isolates the quality of the representation from the capacity of the classifier and is the protocol used for the M2D family, MATPAC and the HEAR and MARBLE benchmarks [5, 12, 44, 43].

5.7.1 Downstream Tasks

Table 5.7 lists the tasks. The music tasks cover genre (GTZAN and FMA-small), instrument family (NSynth), pitch (NSynth) and tagging (MagnaTagATune). The pitch task is included because the CQT teacher injects exactly this kind of information, so it is where the benefit of the teacher should be most visible, as key and pitch detection were in MERT [17]. FMA-small is the only in-domain task; its test clips come from artists who are absent from pre-training because of the artist-filtered split. Two environmental-sound tasks, ESC-50 and UrbanSound8K, measure whether general-audio ability is retained. GTZAN is used with its fault-filtered split [40, 39]. The tasks GTZAN, NSynth (instrument and pitch), ESC-50 and UrbanSound8K are available in the EVAR toolkit of the M2D authors [45]; FMA-small and MagnaTagATune are added by writing task metadata in the same format, using the official FMA split and one fixed published split of MagnaTagATune that is recorded before any evaluation.

Table 5.7: Downstream tasks. Splits are given as training, validation and test sizes where a fixed split exists. The GTZAN and NSynth figures are those used in the M2D evaluation [5]; the FMA-small split follows the dataset paper [32].

Task	Role	Labels	Metric	Split
GTZAN [38]	Music, genre	10 genres	Accuracy	Fault-filtered: 443 / 197 / 290
NSynth, instrument [41]	Music, timbre	11 instrument families	Accuracy	289,205 / 12,678 / 4,096
NSynth, pitch [41]	Music, pitch	MIDI pitch	Accuracy	Same notes as above
FMA-small genre [32]	Music, in-domain	8 genres	Accuracy	Official, artist-filtered: 6,400 / 800 / 800
MagnaTagATune [37]	Music, tagging	50 tags	mAP	One fixed published split
ESC-50 [51]	Retention	50 sound classes	Accuracy	5-fold cross-validation
UrbanSound8K [52]	Retention	10 sound classes	Accuracy	10-fold cross-validation

5.7.2 Frozen-Feature Linear Evaluation

For every task, clips longer than the 6.08-s input of M2D are cut into consecutive windows, each window is encoded with all patches visible, and the frame features of the windows are concatenated along time, which is the first of the two options for variable-length audio described by the authors [5]. The clip

embedding is the temporal mean of the T' frame features h_τ of the encoder,

$$e = \frac{1}{T'} \sum_{\tau=1}^{T'} h_\tau \in \mathbb{R}^{3840}. \quad (5.14)$$

A single linear layer is trained on the standardized embeddings with the Adam optimizer, a learning rate of 3×10^{-5} for at most 200 epochs, and early stopping with a patience of 20 epochs on the validation split, following the evaluation of the M2D authors [5], and the model of the best validation epoch is evaluated on the test split. The softmax cross-entropy loss is used for single-label tasks and the sigmoid binary cross-entropy loss for tagging. Each probe is trained with six random seeds. Because the encoder is frozen, the embeddings of each model are computed once per task and cached, so the cost of evaluating a checkpoint is dominated by the forward pass over the data, and the large NSynth task is evaluated only for the final checkpoints.

5.7.3 Metrics, Aggregates and Statistics

Accuracy is used for single-label tasks and mean average precision (mAP) for tagging, all expressed in percent. Two aggregate scores summarize a model m over the music tasks $\mathcal{T}_{\text{mus}}$ and the retention tasks $\mathcal{T}_{\text{gen}}$, with $s_u(m)$ the score on task u :

$$S_{\text{mus}}(m) = \frac{1}{|\mathcal{T}_{\text{mus}}|} \sum_{u \in \mathcal{T}_{\text{mus}}} s_u(m), \quad S_{\text{gen}}(m) = \frac{1}{|\mathcal{T}_{\text{gen}}|} \sum_{u \in \mathcal{T}_{\text{gen}}} s_u(m), \quad (5.15)$$

and the forgetting score is $\Delta_{\text{gen}}(m) = S_{\text{gen}}(m) - S_{\text{gen}}(\text{A0})$, which is negative when general ability is lost. To express how far the cheap step goes towards a music-specific reference r , the gap-closure ratio on task u is

$$\text{GCR}_u(m; r) = \frac{s_u(m) - s_u(\text{A0})}{s_u(r) - s_u(\text{A0})}, \quad (5.16)$$

where a value of 0 means no change from the unmodified model, 1 means parity with the reference, and values above 1 mean that the adapted model surpasses it. The ratio is unstable when the reference is close to A0, so it is computed only on tasks where $s_u(r) - s_u(\text{A0}) \geq 1$ point, and the number of such tasks is reported next to the mean. In addition, the number of tasks on which a model is within the confidence interval of a reference, or above it, is reported as a parity count.

Uncertainty is handled at two levels. For one trained model, the mean over the $n = 6$ probe seeds is reported with the 95 % interval

$$\bar{s} \pm t_{n-1, 0.975} \frac{\text{sd}}{\sqrt{n}}. \quad (5.17)$$

For the difference between two models on a fixed test split, a paired bootstrap over the test clips with

10,000 resamples gives a 95 % percentile interval of the difference [67]; for the cross-validated tasks the per-fold differences are used instead. A difference between two arms is called *supported* only if the interval of the difference excludes zero and, where a second pre-training seed exists, the sign is the same for both seeds; otherwise it is reported as inconclusive. Only the contrasts listed in Section 5.6 are tested, so that the number of comparisons stays small and fixed in advance.

5.7.4 Threats to Validity

- **Small test sets.** The GTZAN test split has 290 clips, so one clip is worth about 0.34 points and its confidence intervals are several points wide. Conclusions will not rest on GTZAN alone, and the bootstrap intervals are reported.
- **Uncontrolled references.** The reference models differ in size, training data, sampling rate and context length (Table 5.5). Comparisons with them are indicative of what is possible, not controlled experiments; only A0 to A5 differ in one factor at a time.
- **In-domain advantage.** FMA-small shares its source with the pre-training data. Its result is therefore reported separately from the out-of-domain music tasks, and conclusions about general music understanding rely on the latter.
- **Limited tuning and seeds.** The noise ratio and loss weights are taken from the literature and not tuned, the best arm is selected on validation data only, and only one or two pre-training seeds are affordable. Differences smaller than the seed-to-seed gap will not be claimed.
- **Possible overlap with external data.** The AudioSet-pretrained starting point may already have seen music similar to the evaluation sets, and no audio fingerprinting is run between FMA and the external benchmarks, so a small undetected overlap cannot be excluded.
- **Computational nondeterminism.** Seeds are fixed, but GPU kernels are not bitwise deterministic, so repeating a run may give slightly different numbers.

5.8 Conclusion

This chapter presented the proposed approach. A resumable pipeline selects 10,000 tracks of the Free Music Archive from the training split of the medium subset with a seeded, artist-capped, nested sampling procedure, audits them, and converts them with the front-end of the original M2D model into log-mel arrays and into CQT targets for a musical teacher. The further pre-training framework M2D-X-Music extends the M2D-X recipe: the online and target networks receive music mixed with background noise, a frozen copy of the original model regularizes the features on clean input, and an additional linear head

regresses the CQT of the clean audio in the manner of the musical teacher of MERT. The experimental design turns these components on and off in a small number of compute-matched arms, adds runs with fewer tracks, and compares the results with public music-specific models. The evaluation uses a frozen-feature linear protocol on music tasks and on two retention tasks, with aggregate scores, a gap-closure ratio, bootstrap intervals and a pre-declared rule for calling a difference supported. The limits of the design, in particular the small test sets, the uncontrolled references and the single GPU, are stated so that the results can be interpreted correctly. The next chapter presents the high-level and low-level design of the system that implements this approach.

Chapter 6 High-Level and Low-Level Design

This chapter presents the design of the whole system: the research platform that carries out the method of Chapter 5 and the demonstration dashboard specified in Chapter 4. It first gives an overview and the considerations that shaped the design, then describes the architecture at system and subsystem level with two interaction scenarios. It then explains the main architectural strategies, the class diagram, and the policies and tactics that govern the implementation. All software is written in Python, and all models are trained and run with PyTorch [68]. Wherever possible, the design extends the public M2D code base [46] and the EVAR evaluation toolkit [45] rather than replacing them, and it borrows the conventions of the MERT [61] and MATPAC++ [49] code bases where they fit.

6.1 System Overview

The system consists of four subsystems. The data preparation pipeline turns the Free Music Archive and the noise corpus into the input format of M2D. The further pre-training framework trains one checkpoint per experimental arm. The evaluation and analysis subsystem measures every checkpoint and every reference model with the same protocol. The demonstration dashboard presents the results and runs the four demonstration tasks. The subsystems communicate only through files on disk: spectrogram arrays, CSV lists of training files, model checkpoints, score files and saved linear probes. Each subsystem can therefore be run, tested and replaced on its own, and every result can be traced to the files that produced it.

6.1.1 Data Preparation Pipeline

A command-line package, `fmaprep`, runs the steps of Section 5.4 as named stages: *select* draws the nested 1,000- to 10,000-track subsets, *audit* decodes and screens each track, *convert* writes the log-mel and CQT arrays, and *check* runs the integrity checks. Each stage reads the outputs of the earlier stages and writes to its own folder, so an interrupted run resumes where it stopped.

6.1.2 Further Pre-training Framework

The framework is the M2D code base with a small extension that adds the musical teacher, the CQT targets and fixed normalization statistics. One training run produces the checkpoints of one arm (Table 5.6); the arms differ only in the command-line options of the training script.

6.1.3 Evaluation and Analysis

The EVAR toolkit computes frozen embeddings, trains the linear probes and records the scores. Small wrapper classes make the reference models available through the same interface, and analysis scripts compute the aggregates, confidence intervals and gap-closure ratios of Section 5.7.

6.1.4 Demonstration Dashboard

A FastAPI backend serves a single-page HTML and JavaScript frontend, reads the imported results from a SQLite database and runs the encoders and probes of the evaluation for the four demonstration tasks (Chapter 4).

6.2 Design Considerations

The design must fit a single GPU and a three-month schedule, keep the comparison between arms fair, and produce results that can be reproduced and traced. The considerations below shaped it.

6.2.1 Assumptions and Dependencies

- **Software.** The M2D code base at a fixed commit, together with the MAE source files that it patches [53], the timm library and nnAudio [47]; the EVAR toolkit; PyTorch with CUDA; and the Hugging Face transformers library [69] for loading the MERT and MATPAC++ checkpoints. A change in the training interface of the M2D code would affect the extension built on it, which is why the commit is fixed.
- **Operating system.** Linux on the workstation, with the NVIDIA driver and CUDA runtime.
- **Hardware.** One NVIDIA RTX A4000 with 16 GB of memory for training, evaluation and the dashboard; the dashboard can also run on a CPU, more slowly.
- **Data.** The FMA medium subset and metadata, the FSD50K development set and the downstream datasets of Table 5.7 are available locally at configured paths, together with the released checkpoints of M2D/0.7 and of the reference models.
- **Users.** Researchers use the command line and configuration files; visitors use only the web interface of the dashboard.
- **Likely changes.** The best arm A^* is known only at the end of the experiments; optional runs (Section 5.6), the distillation of MERT features as a second teacher, and further dashboard tasks may be added later. The design keeps each of these to a new script, a new option or new database

rows.

6.2.2 General Constraints

- **Memory and compute.** The online, target and predictor networks, the frozen teacher and the two linear heads must fit into 16 GB during training. This fixes the micro-batch at 32 with four accumulation steps (Table 5.4), mixed precision, and a teacher that runs without gradients on the same GPU.
- **Input compatibility.** Spectrograms must be computed with exactly the front-end of the original M2D pre-training (Table 5.3); any other implementation would add a distribution shift that cannot be separated from the effect under study.
- **Storage.** About 10 GB of half-precision arrays for 10,000 tracks (Section 5.4), plus the checkpoints of every arm, which hold the online and target networks and the optimizer state and are saved every 100 epochs.
- **Licensing.** FMA audio is used under its Creative Commons licenses, and the MERT weights are licensed for non-commercial use [20]; no audio is redistributed except the openly licensed dashboard samples.
- **Library compatibility.** The MATPAC++ inference code pins an old version of timm that conflicts with the version needed by the M2D code [49], so the two cannot share one Python environment.
- **Reproducibility and verification.** All arms must use the same track list and seeds, every stage must verify its outputs, and the dashboard must meet the non-functional requirements of Section 4.4.

6.2.3 Goals and Guidelines

- **Reuse before rewrite.** The official recipe is only reproduced faithfully if its code is reused, so new code is added as subclasses and options of the existing code rather than as a re-implementation.
- **One factor per arm.** Arms differ only in command-line options, never in code paths that are switched by editing files, so that a comparison between arms is a comparison of exactly the intended factor.
- **One front-end everywhere.** Training, evaluation and the dashboard compute spectrograms and embeddings with the same functions, so that the dashboard demonstrates the models that were evaluated.
- **Keep it simple.** Plain files, command-line scripts and a file-based database are used instead of

services, because the data are small and must be easy to inspect.

- **Fail loudly.** Missing files, shape mismatches, non-finite losses and unknown codes stop the run with a clear message instead of being skipped silently.
- **Reproducible by default.** Every run stores its options, the code version and a hash of its track list next to its outputs.

6.2.4 Development Methods

The project follows an iterative, research-driven process. The data pipeline and its tests are built first; a pilot run of a few epochs then measures time and memory on the A4000 (Section 5.5); the arms are run in the order fixed in Chapter 5, with evaluation of each checkpoint as it appears; and the dashboard is built in small steps once the first probes exist, starting with the unmodified model A0 (Section 4.10). A strict waterfall process was rejected because the pilot run can change the schedule, for example by reducing the number of epochs for all arms. Parts whose correctness can be checked automatically, such as subset selection, shapes and losses, are developed with unit tests; the rest is checked with spectrogram galleries, training curves and the label-free feature statistics logged by the training script.

6.3 System Architecture

Figure 6.1 shows the layered architecture. The data layer holds the raw corpora, the downstream datasets and the released checkpoints. The preparation layer turns the corpora into arrays and file lists. The learning layer runs the further pre-training of each arm with the frozen original model as teacher. The evaluation layer measures checkpoints and reference models and computes the statistics. The presentation layer imports the results into the dashboard. Each layer depends only on the files written by the layers below it.

This decomposition was chosen because the three expensive steps, conversion, training and embedding extraction, have very different costs and must not be repeated when a later step changes. With files between the layers, a new arm reuses the converted data, and a new statistic reuses the cached embeddings. A single training script that reads MP3 files and evaluates at the end was rejected: it would mix data handling with model code, repeat the conversion for every arm, and make it hard to show that all arms saw the same data.

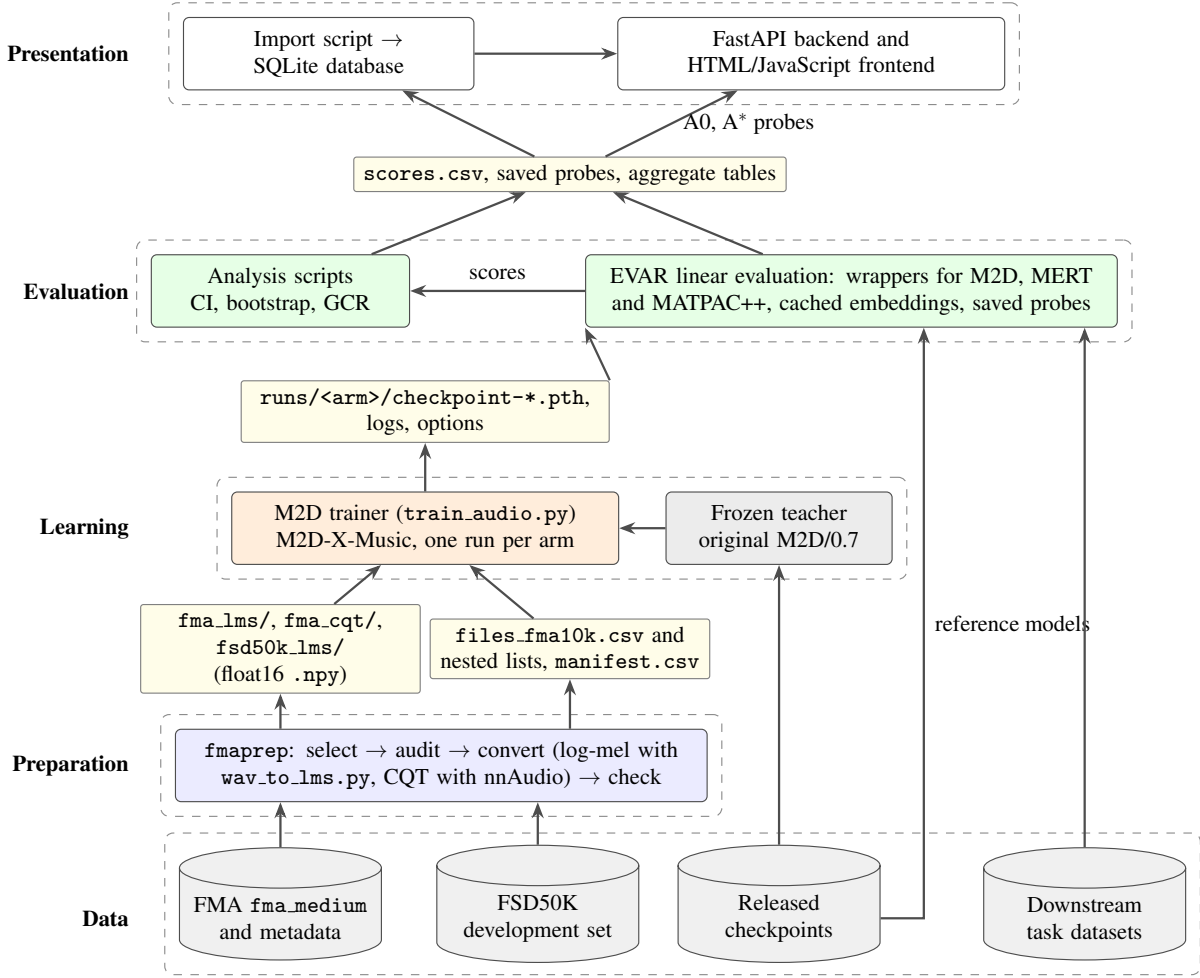


Figure 6.1: Layered system architecture, read from bottom to top. The data layer holds FMA, the FSD50K noise corpus, the released checkpoints and the downstream datasets. The preparation layer, the fmaprep pipeline, writes spectrogram and CQT arrays and the training file lists (yellow boxes are files). The learning layer trains one run per arm, with the frozen original M2D model as teacher, and writes checkpoints. The evaluation layer runs EVAR on the checkpoints and on the reference models, and the analysis scripts compute the statistics. At the top, the results and the probes of A0 and A* are imported into the dashboard.

6.3.1 Subsystem Architecture

Further pre-training. Figure 6.2 shows the modules of the training subsystem; the data flow of the objective itself is given in Figure 5.3. The training script `train_audio.py` of the M2D code base parses the options of a run, creates the run folder and builds three objects. The dataset `MusicMixedSpecDataset` extends the mixed-noise dataset of M2D-X: it reads the music and noise spectrograms through the existing `SpectrogramDataset`, mixes them with the existing log-domain mixing function, crops the CQT array at the same start frame, and applies the fixed `AudioSet` statistics. The model `M2D_X_Music_ViT` extends `M2D_X_ViT`, which already contains the online encoder, the predictor, the EMA target encoder and the offline linear head, with a second linear head for the CQT target. The teacher is the existing `RuntimeM2D` class loaded with the original M2D/0.7 weights and frozen. A training loop adapted from

`train_one_epoch_m2dx` passes the three losses to the optimizer, updates the target encoder and logs each loss separately. Checkpoints are written every 100 epochs by the existing `misc.save_model`, which also starts the evaluation hook `quick_eval.sh`.

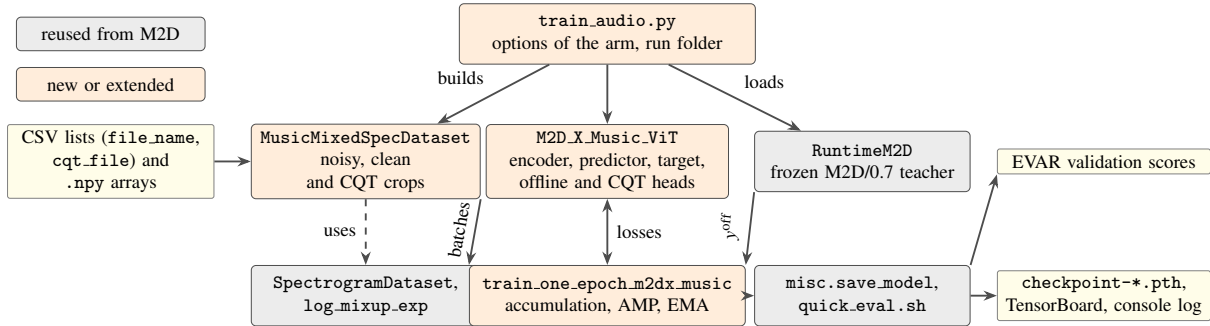


Figure 6.2: Modules of the further pre-training subsystem. Grey boxes are reused unchanged from the M2D code base, orange boxes are new or extended, and yellow boxes are files. The training script builds the music dataset, the model and the frozen teacher. The dataset reads the file lists and arrays through the existing spectrogram dataset and mixing function. The training loop takes batches from the dataset and teacher features from the frozen model, obtains the losses from the model, and passes checkpoints to the existing saving function, which also starts a quick evaluation with EVAR.

Data preparation. The stages of `fmaprep` follow Figure 5.2. Each stage is a class with a `run` method and writes a completion marker, so finished stages are skipped unless forced. All paths, seeds and thresholds are read from one YAML file. The convert stage calls the unchanged converter of `wav_to_lms.py` for log-mel spectrograms and an `nnAudio` CQT layer with the settings of the MERT musical teacher [61], and writes the training list in the CSV format of the M2D code, extended with a column for the CQT file.

Evaluation. Each model is described by an EVAR configuration file that names its wrapper class, weight file, sampling rate and normalization statistics. The M2D checkpoints use the existing wrapper `AR_M2D`; new wrappers `AR_MERT` and `AR_MATPAC` load the reference models from the Hugging Face hub and return the features of a chosen layer. Task metadata for FMA-small and MagnaTagATune are added as CSV files in the format of the existing tasks. EVAR caches the embeddings of each model and task, trains the probes with the protocol of Section 5.7, saves the trained linear layers on request, and appends every score to one results file, which the analysis scripts read.

Dashboard. The backend has four modules: an API module with the routes for results, tasks, samples and predictions; an inference module whose `InferenceService` holds the loaded encoders and probes and processes one request at a time; a repository module that maps the tables of Figure 4.8 with SQLAlchemy; and the import script of use case UC-06. The frontend consists of one HTML page, one script and one style sheet, and it obtains all data through the JSON API.

6.3.2 Sequence Diagrams

Figures 6.3 and 6.4 show the two central interactions: one step of further pre-training, and one prediction request of the dashboard.

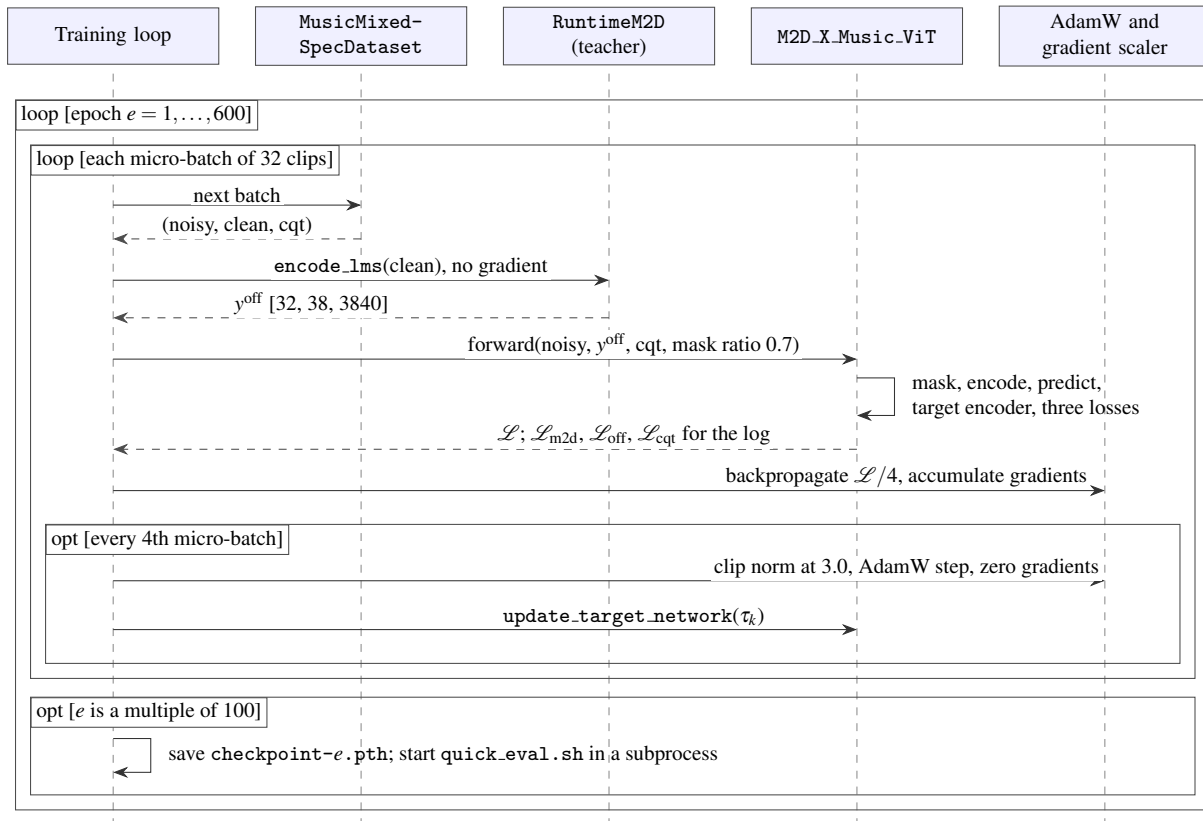


Figure 6.3: Sequence diagram of further pre-training with five participants: the training loop, the music dataset, the frozen teacher, the model and the optimizer with the gradient scaler. In a loop over epochs and an inner loop over micro-batches, the loop takes a batch of noisy, clean and CQT crops; the teacher encodes the clean crop without gradient; the model masks, encodes and predicts on the noisy crop and returns the total loss and its three terms. The scaled loss is backpropagated, and every fourth micro-batch the gradients are clipped, the AdamW step is taken and the target encoder is updated. Every 100 epochs a checkpoint is saved and a quick evaluation is started.

6.4 Architectural Strategies

The strategies below have the largest effect on the structure of the system. For each, the alternatives that were considered are given.

6.4.1 Extending the M2D Code Base

The further pre-training is implemented as a small extension of the M2D code base: a model subclass, a dataset subclass, an adapted training loop and three new options (the CQT loss weight, the CQT column of the file list, and the use of the fixed normalization statistics given by the existing `--norm_stats` op-

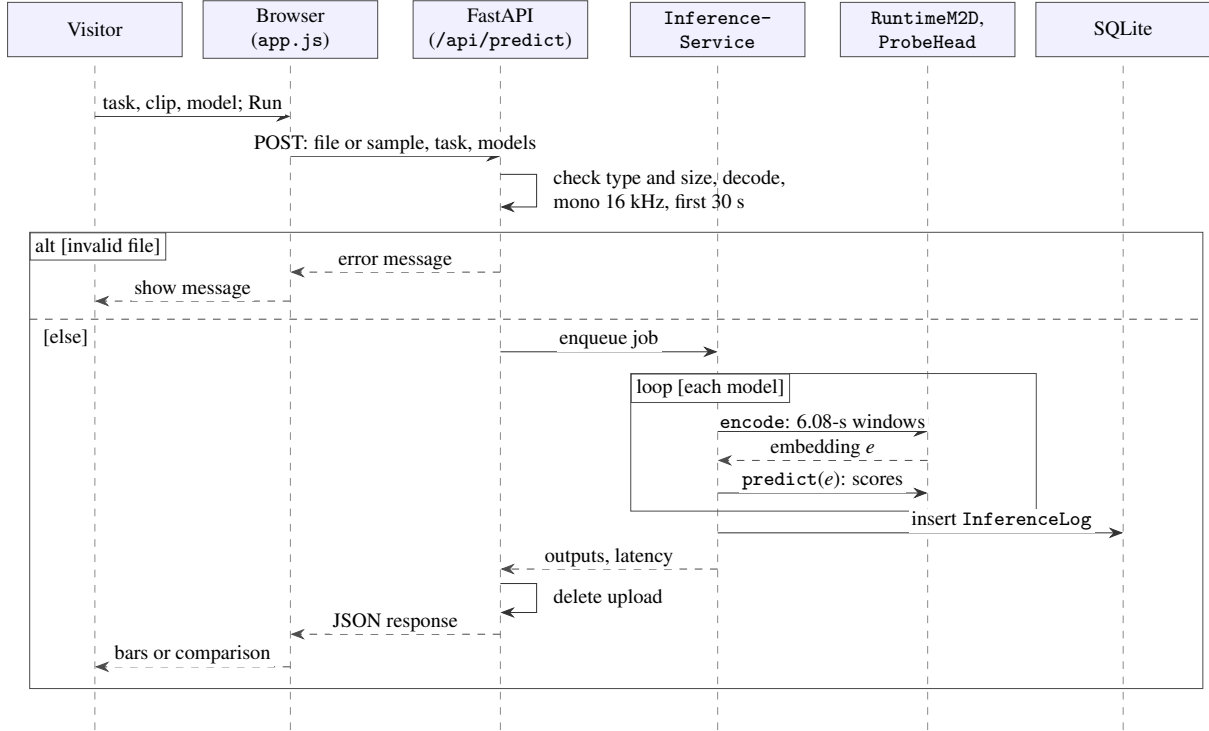


Figure 6.4: Sequence diagram of a dashboard prediction request (use cases UC-04 and UC-05) with six participants: the visitor, the browser, the FastAPI route, the inference service, the encoder with its probe, and the SQLite database. The browser posts the clip, task and models; the route checks and decodes the audio. If the file is invalid, an error message is returned and shown. Otherwise the job is queued, and for each selected model the encoder computes the clip embedding and the probe returns the scores. The request is logged, the uploaded file is deleted, and the browser shows the bars or the side-by-side comparison.

tion). Everything else, from masking to the EMA schedule and checkpointing, is the code that produced the published M2D-X results [5], and arms A1 to A3 run on it unchanged except for the normalization statistics. The musical-teacher head copies the structure of the CQT predictor of MERT, a single linear layer from the features to 84 CQT bins [61]. Three alternatives were rejected. Porting the method to the fairseq framework [70], on which the MERT and MATPAC++ training code is built, would require converting the M2D weights and re-implementing the recipe. Writing a new training loop from scratch would risk small deviations from the recipe that could not be told apart from the effects under study. A framework such as PyTorch Lightning would add a layer of indirection around a loop that already exists and works.

6.4.2 Offline Precomputation of Inputs and Targets

Log-mel spectrograms and CQT targets are computed once and stored as half-precision arrays, and the data loader only crops, mixes and normalizes them. This follows the M2D code, which also trains from precomputed spectrograms, and it keeps the GPU free for the networks. The alternative of the MERT

implementation, computing the CQT on the GPU at every step, was rejected: with 10,000 tracks the one-time conversion is cheap, whereas a per-step transform would cost memory and time on a 16-GB GPU for every one of the 600 epochs. Mixing the noise at load time, rather than storing noisy copies, keeps the noise samples different in every epoch, as in the official recipe.

6.4.3 Experiments as Command Lines and Run Folders

Each arm is a short shell script that calls the training script with its options, for example the noise ratio, the loss weights, the file list and the seed; the arms of Table 5.6 differ only in these options. The training script writes the options, the commit of the code and a hash of the track list into the run folder next to the checkpoints and logs, so that the folder is a complete record of the run. Configuration through Hydra YAML files, as in the fairseq-based code of MERT and MATPAC++, was considered but not adopted, because the M2D code is configured with command-line options and two configuration systems would have to be kept consistent.

6.4.4 One Evaluation Interface for All Models

All models, whether further pre-trained checkpoints or references, are evaluated through the model interface of EVAR, in which a wrapper class turns a batch of waveforms into frame features. The same probe code, splits and seeds therefore apply to every model, and the probes saved by EVAR are the ones loaded by the dashboard. The reference models are loaded from the Hugging Face hub with the transformers library [69] in a separate environment, because of the library conflict noted above; this environment writes only embeddings to the shared cache. Evaluating with the MARBLE toolkit, for which EVAR provides a plug-in, was rejected for this study because of its cost on one GPU and because scores of its revised versions are not comparable with each other (Chapter 3).

6.4.5 Files as Interfaces and Single-Worker Serving

The subsystems exchange only files, and the dashboard reads only the files exported by the evaluation and its own database. Training curves are kept in TensorBoard logs, which the M2D code already writes, rather than in an experiment-tracking server, which would add a service to maintain. In the dashboard, the encoders and probes are loaded once at start-up and all inference requests pass through one queue processed by a single worker, which bounds the GPU memory and satisfies the concurrency requirement of Section 4.4. A task queue with a separate broker, such as Celery with Redis, and loading the models for each request were both rejected: the first is unnecessary for a handful of visitors, and the second would make every prediction slow.

6.5 Domain Model/Class Diagram

Figure 6.5 shows the main classes. Grey classes are reused unchanged from the M2D code base and EVAR; white classes are new. In the data part, `MusicMixedSpecDataset` extends `MixedSpecDataset`, which is composed of two `SpectrogramDataset` objects for music and noise, and the preparation stages share the abstract class `Stage`. In the model part, `M2D_X_Music_ViT` extends `M2D_X_ViT` (itself derived from `M2DViT`) with the CQT head, and the training loop uses the dataset, the model and `RuntimeM2D` as teacher. In the evaluation part, the wrappers derive from `BaseAudioRepr`. In the dashboard part, `InferenceService` holds encoders and `ProbeHead` objects and writes through `ResultRepository`, which `ResultImporter` also uses. `RuntimeM2D` appears three times: as the teacher during training, inside `AR_M2D` during evaluation, and inside `InferenceService` in the dashboard. This single encoder path is what makes the dashboard demonstrate exactly the evaluated models.

6.6 Policies and Tactics

These policies and tactics affect the details of the implementation rather than its overall structure.

6.6.1 Choice of Tools

Python 3.10 or above and PyTorch [68] are used throughout, with the versions of PyTorch, timm and nnAudio pinned in a conda environment file. A second environment with the versions required by the reference models is used only for extracting their embeddings. TensorBoard records training curves, pytest runs the tests, and the dashboard uses FastAPI with Uvicorn, SQLAlchemy with SQLite, and Chart.js, as listed in Section 4.7. fairseq is not installed, because no fairseq-based model is trained; MERT and MATPAC++ are only used through their published checkpoints.

6.6.2 Coding Guidelines and Conventions

The code follows PEP 8 and is checked with a linter; functions carry type hints and docstrings. Extensions of the M2D code follow its naming, for example `M2D_X_Music_ViT` with the factory function `m2d_x_music_vit_base` and the option `--loss_cqt`, so that they read like the existing `M2D_X_ViT`, `m2d_x_vit_base` and `--loss_off`. Changes to upstream files are kept in a patch file, in the same way as the M2D code base distributes its own changes to the MAE files, and new classes are placed in new files. Paths are never hard-coded; they come from options or the configuration file. Every source of randomness takes an explicit seed.

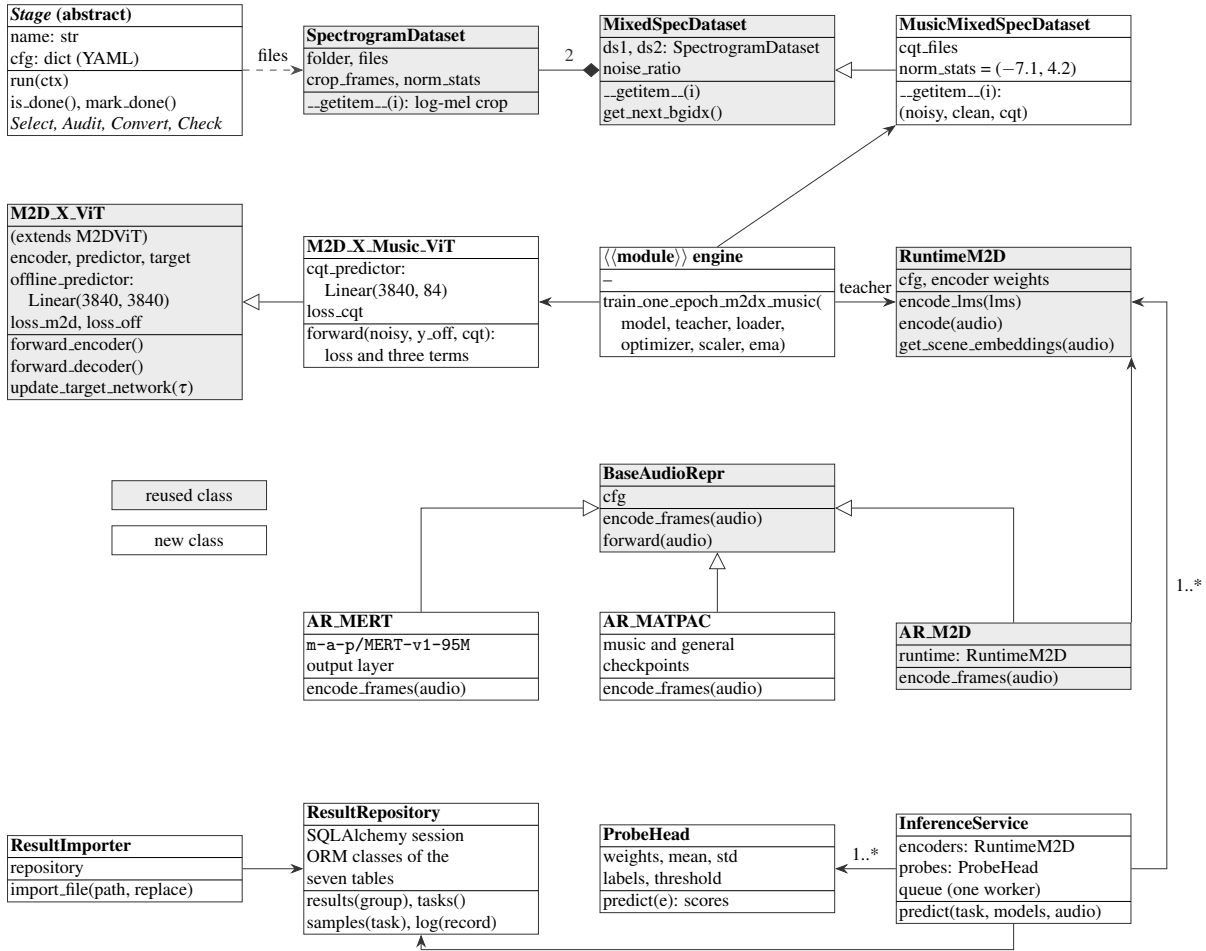


Figure 6.5: Class diagram. Grey classes are reused from the M2D code base and EVAR, white classes are new; open triangles mark inheritance, the diamond composition, solid arrows associations and the dashed arrow a dependency through files. Top row (data): the preparation stages derive from Stage and write the files read by SpectrogramDataset; MixedSpecDataset is composed of two spectrogram datasets and is extended by MusicMixedSpecDataset. Second row (model): M2D_X_Music_ViT extends M2D_X_ViT; the training-loop module uses the dataset, the model and RuntimeM2D as teacher. Middle (evaluation): AR_MERT, AR_MATPAC and AR_M2D derive from BaseAudioRepr, and AR_M2D wraps RuntimeM2D. Bottom row (dashboard): InferenceService uses RuntimeM2D encoders and ProbeHead objects and logs through ResultRepository, which ResultImporter also uses.

6.6.3 Testing

Unit tests with pytest cover the parts whose correctness can be checked automatically: the subset selection (nested lists, artist cap and identical output for the same seed); the audit rules on synthetic silent, short and duplicate files; the equal frame counts of log-mel and CQT arrays; the mixing function against Equation 5.8; the shapes of the CQT head output ($32 \times 38 \times 84$); and checkpoint loading, which must accept the M2D/0.7 weights with only the new heads missing. Two tests guard the comparison between arms. The first checks that, with the CQT weight set to zero, the loss of the new model equals that of M2D_X_ViT on the same batch and masks, so that the extension cannot change arms A1 to A3. The second checks which view each network receives: the dataset returns its views by name rather than by

position, and the test asserts that the online network gets the noisy view and the teacher the clean view, because swapping two tensors of the same shape would raise no error. A smoke test runs one epoch on a list of a few tracks, and the pilot run of Chapter 5 serves as the system test of training. For the dashboard, API tests use the FastAPI test client, and a consistency test checks that the dashboard reproduces the cached EVAR embeddings of a few test clips. The full test cases are given in the next chapter.

6.6.4 Requirements Traceability

Each research question is linked to the arms and contrasts that answer it (Section 5.6), each contrast to rows of the results file, and each row to its checkpoint, task, split and probe seed. The functional requirements FR-01 to FR-18 of the dashboard (Table 4.1) map to the modules of Section 6.3: results views to the API and repository, FR-06 to FR-15 to the API and inference modules, FR-16 to the repository, and FR-17 to the import script. The database keeps the source file of every imported score, so every number on the dashboard can be traced back to a run.

6.6.5 Source Organization and Build

The repository contains the following folders: `fmaprep/` (preparation stages and their YAML configuration); `m2d/` (a clone of the M2D code at a fixed commit, the downloaded MAE files and the patch `patch_music.diff`); `evan/` (a clone of EVAR with the new wrappers, configuration files and task metadata); `runs/` (one script per arm); `analysis/` (statistics and report tables); `dashboard/backend/` and `dashboard/frontend/`; and `tests/`. Data, arrays, checkpoints and caches are kept outside version control. The system is built by creating the conda environments, cloning the two upstream repositories, downloading the MAE files and applying the patches, as in the setup of the M2D code base [46]. Then `python -m fmaprep all` prepares the data, `bash runs/A4.sh` trains an arm, the EVAR command `lineareval.py` evaluates a checkpoint on a task, and `uvicorn` starts the dashboard.

6.6.6 Maintenance

A new arm is added as one script in `runs/`, a new reference model as one wrapper class and configuration file, and a new dashboard task as task metadata, a saved probe and database rows. If the upstream code changes, the patch is re-applied at the new commit and the unit tests show whether the extension still behaves as before.

6.7 Conclusion

This chapter presented the design of the system. A layered architecture separates data preparation, further pre-training, evaluation and presentation, which communicate only through files, so that each expensive step runs once and every result can be traced. The main strategies are a small extension of the M2D code base instead of a new framework, offline precomputation of spectrograms and CQT targets, experiments defined by command-line options and recorded in run folders, one evaluation interface for all models, and single-worker serving in the dashboard. The sequence and class diagrams show how the components collaborate, with one encoder path shared by training, evaluation and the dashboard, and the policies and tactics fix the tools, conventions, tests and traceability. Together with the method of Chapter 5 and the requirements of Chapter 4, this design is the basis for the implementation and the test cases in the next chapter.

Chapter 7 Implementation and Test Cases

For each chapter, provide a paragraph of introduction and at the end, a paragraph of conclusions. (Not the programming code but the algorithmic and procedural details especially related to the hidden/ backend algorithms that are not covered in the design)

7.1 Implementation

Whatever implementation that you have done so far, please elaborate here. Give clear details of the algorithms that were implemented along with the platform and the APIs which were used. **NOTE: For FYP-1, this chapter can be changed to description of prototype developed.**

7.1.1 Implementation of First Component/Algorithm

Write the implementation of the first component of your system here.

7.2 Test case Design and description

This section will be added in FYP-II. Summarize the common attributes of test cases. This may include input constraints that must be true for every input in the set of associated test cases, any shared environmental needs, any shared special procedural requirements, and any shared case dependencies. The following scheme is recommended for describing test cases in detail. The Sample test case is provided in table 7.1.

Table 7.1: Sample Test case No.1

<Software Component Name>			
<Reference>			
Test Case ID:	Reference Number	QA Test Engineer:	Name of personnel
Test case Version:	Version number	Reviewed By:	Testing Team lead
Test Date:	Date	Use Case Reference(s):	Relation to use cases
Revision History:	Refer to previous test case identity (if any)		
Objective:	Need and scope of the testing		
Product/Ver/Module:	Refer to overall system being built and the place of this test case in it.		
Environment:	Necessary and desired properties of the test environment.		
Assumptions:	Assumptions that might affect the testing process.		
Pre-Requisite:	Necessary condition that needs to be fulfilled before the test case.		
Step No.	Execution description	Procedure result	
	Events being tested.	Mention software response.	
Comments:			
Passed Failed Not Executed			

7.3 Test Metrics

Summarize here the common ground of attributes of test case metrics as explained in Table 7.2.

Table 7.2: Sample Test case Metric.No.1

Metric	Purpose
Number of Test Cases	Total number of test cases that you have developed for your system.
Number of Test Cases Passed	The number of test cases that successfully passed
Number of Test Cases Failed	The number of test cases that failed
Test Case Defect Density	(No of test cases failed * 100) No of test cases executed
Test Case Effectiveness	No of defects detected using test cases *100 Total number of defects detected
Traceability Matrix	Traceability is the ability to determine that each the feature has a source in requirements and each requirement has a corresponding implemented feature.

7.3.1 Sample Test case Metric.No.2

Similar to the above table, add here for test metrics 2,3,4.

Chapter 8 User Manual

This chapter will be added in FYP-II. Design a complete user manual of your application and add that in this chapter. **NOTE: This Chapter is optional for Research projects but compulsory for Development projects.**

Chapter 9 Experimental Results and Discussion

This chapter will be added in FYP-II. Give proper analysis and discussion of experimental results (in plain English text) along with tables of results. For each chapter provide a paragraph of introduction and in the end a paragraph of conclusions.

NOTE: This Chapter is optional for Development projects but compulsory for Research projects.

Chapter 10 Conclusions

10.1 aa

10.1.1 dfs

10.1.1.1 dsd

This chapter is mandatory. Give conclusions and summary of the work done. What were your findings and what were the results? Discuss in detail whether the scope of your project was entirely covered or not and whether the objectives of the project were met or not. What challenges did you face and what has been left out and why.

Sum up all the conclusions of all the chapters here to make a final conclusion chapter. Do not repeat any text, just summarize it in different words. Give recommendations for future work also.

For FYP-1 it is mandatory to list down a plan of the work to be done for FYP-2.

Bibliography

- [1] P.-Y. Huang, H. Xu, J. Li, A. Baevski, M. Auli, W. Galuba, F. Metze, and C. Feichtenhofer, “Masked autoencoders that listen,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [2] S. Chen, Y. Wu, C. Wang, S. Liu, D. Tompkins, Z. Chen, W. Che, X. Yu, and F. Wei, “BEATs: Audio pre-training with acoustic tokenizers,” in *Proc. International Conference on Machine Learning (ICML)*, PMLR, vol. 202, pp. 5178–5193, 2023.
- [3] D. Niizumi, D. Takeuchi, Y. Ohishi, N. Harada, and K. Kashino, “Masked spectrogram modeling using masked autoencoders for learning general-purpose audio representation,” in *HEAR: Holistic Evaluation of Audio Representations (NeurIPS 2021 Competition)*, PMLR, vol. 166, pp. 1–24, 2022.
- [4] D. Niizumi, D. Takeuchi, Y. Ohishi, N. Harada, and K. Kashino, “Masked Modeling Duo: Learning representations by encouraging both networks to model the input,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1–5, 2023.
- [5] D. Niizumi, D. Takeuchi, Y. Ohishi, N. Harada, and K. Kashino, “Masked Modeling Duo: Towards a universal audio pre-training framework,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 32, pp. 2391–2406, 2024.
- [6] D. Niizumi, D. Takeuchi, Y. Ohishi, N. Harada, and K. Kashino, “Masked Modeling Duo for speech: Specializing general-purpose audio representation to speech using denoising distillation,” in *Proc. Interspeech*, pp. 1294–1298, 2023.
- [7] D. Niizumi, D. Takeuchi, Y. Ohishi, N. Harada, M. Yasuda, S. Tsubaki, and K. Imoto, “M2D-CLAP: Masked Modeling Duo meets CLAP for learning general-purpose audio-language representation,” in *Proc. Interspeech*, pp. 57–61, 2024.
- [8] D. Niizumi, D. Takeuchi, M. Yasuda, B. T. Nguyen, Y. Ohishi, and N. Harada, “M2D-

- CLAP: Exploring general-purpose audio-language representations beyond CLAP.” arXiv preprint arXiv:2503.22104, 2025.
- [9] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas, “Self-supervised learning from images with a joint-embedding predictive architecture,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 15619–15629, 2023.
- [10] A. Riou, S. Lattner, G. Hadjeres, and G. Peeters, “Investigating design choices in joint-embedding predictive architectures for general audio representation learning,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing Workshops (ICASSPW)*, pp. 680–684, 2024.
- [11] L. Tuncay, E. Labbé, E. Benetos, and T. Pellegrini, “Audio-JEPA: Joint-embedding predictive architecture for audio representation learning.” arXiv preprint arXiv:2507.02915, 2025.
- [12] A. Quelenec, P. Chouteau, G. Peeters, and S. Essid, “Masked latent prediction and classification for self-supervised audio representation learning,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1–5, 2025.
- [13] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, “Emerging properties in self-supervised vision transformers,” in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9630–9640, 2021.
- [14] A. Quelenec, P. Chouteau, G. Peeters, and S. Essid, “MATPAC++: Enhanced masked latent prediction for self-supervised audio representation learning.” arXiv preprint arXiv:2508.12709, 2025. Preprint, under review.
- [15] Y. Gu, J. Zhang, J. Li, Z. Wu, and L. Juvela, “Frequency-aware self-supervised music representation learning.” arXiv preprint arXiv:2606.25713, 2026. Preprint; the model was first named PupuJEPA and renamed PupuM2D in version 3.
- [16] R. Castellon, C. Donahue, and P. Liang, “Codified audio language modeling learns useful representations for music information retrieval,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, pp. 88–96, 2021.
- [17] Y. Li, R. Yuan, G. Zhang, Y. Ma, X. Chen, H. Yin, C. Xiao, C. Lin, A. Ragni, E. Benetos, N. Gyenge, R. Dannenberg, R. Liu, W. Chen, G. Xia, Y. Shi, W. Huang, Z. Wang, Y. Guo, and J. Fu, “MERT: Acoustic music understanding model with large-scale self-supervised training,” in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

- [18] J. Spijkervet and J. A. Burgoyne, “Contrastive learning of musical representations,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, pp. 673–681, 2021.
- [19] M. C. McCallum, F. Korzeniowski, S. Oramas, F. Gouyon, and A. F. Ehmann, “Supervised and unsupervised learning of audio representations for music understanding,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, pp. 256–263, 2022.
- [20] m-a-p, “MERT-v0-public and MERT-v1-95m model cards.” <https://huggingface.co/m-a-p/MERT-v0-public>. Accessed: October 2026.
- [21] M. Won, Y.-N. Hung, and D. Le, “A foundation model for music informatics,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024. arXiv:2311.03318.
- [22] C.-C. Chiu, J. Qin, Y. Zhang, J. Yu, and Y. Wu, “Self-supervised learning with random-projection quantizer for speech recognition,” in *Proc. International Conference on Machine Learning (ICML)*, *PMLR*, vol. 162, pp. 3915–3924, 2022.
- [23] H. Zhu, Y. Zhou, H. Chen, J. Yu, Z. Ma, R. Gu, Y. Luo, W. Tan, and X. Chen, “MuQ: Self-supervised music representation learning with Mel residual vector quantization.” arXiv preprint arXiv:2501.01108, 2025.
- [24] S. Gururangan, A. Marasović, S. Swayamdipta, K. Lo, I. Beltagy, D. Downey, and N. A. Smith, “Don’t stop pretraining: Adapt language models to domains and tasks,” in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 8342–8360, 2020.
- [25] W.-N. Hsu, A. Sriram, A. Baevski, T. Likhomanenko, Q. Xu, V. Pratap, J. Kahn, A. Lee, R. Collobert, G. Synnaeve, and M. Auli, “Robust wav2vec 2.0: Analyzing domain shift in self-supervised pre-training,” in *Proc. Interspeech*, pp. 721–725, 2021.
- [26] S. Lee, M. Kang, J. Lee, S. J. Hwang, and K. Kawaguchi, “Self-distillation for further pre-training of transformers,” in *Proc. International Conference on Learning Representations (ICLR)*, 2023.
- [27] Y. Zhang, Y. Gao, W. Zhou, Z. Yuan, K. Imoto, and T. Kawahara, “SONAR: Self-distilled continual pre-training for domain adaptive audio representation.” arXiv preprint arXiv:2509.15703, 2025.
- [28] D. Niizumi, D. Takeuchi, M. Yasuda, B. T. Nguyen, Y. Ohishi, and N. Harada, “Towards pre-training an effective respiratory audio foundation model,” in *Proc. Interspeech*, 2025. arXiv:2505.15307.
- [29] Y. Ding and A. Lerch, “Parameter-efficient transfer learning for music foundation models.” arXiv preprint arXiv:2411.19371, 2024.

- [30] J. F. Gemmeke, D. P. W. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, “Audio Set: An ontology and human-labeled dataset for audio events,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 776–780, 2017.
- [31] E. Fonseca, X. Favory, J. Pons, F. Font, and X. Serra, “FSD50K: An open dataset of human-labeled sound events,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 829–852, 2022.
- [32] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson, “FMA: A dataset for music analysis,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, 2017.
- [33] M. Defferrard *et al.*, “FMA: A dataset for music analysis, repository and download page.” <https://github.com/mdeff/fma>. Accessed: October 2026.
- [34] T. Bertin-Mahieux, D. P. W. Ellis, B. Whitman, and P. Lamere, “The Million Song Dataset,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, pp. 591–596, 2011.
- [35] I. A. P. Santana *et al.*, “Music4All: A new music database and its applications,” in *Proc. International Conference on Systems, Signals and Image Processing (IWSSIP)*, 2020.
- [36] D. Bogdanov, M. Won, P. Tovstogan, A. Porter, and X. Serra, “The MTG-Jamendo dataset for automatic music tagging,” in *Machine Learning for Music Discovery Workshop, International Conference on Machine Learning (ICML)*, 2019.
- [37] E. Law, K. West, M. Mandel, M. Bay, and J. S. Downie, “Evaluation of algorithms using games: The case of music tagging,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, pp. 387–392, 2009.
- [38] G. Tzanetakis and P. Cook, “Musical genre classification of audio signals,” *IEEE Transactions on Speech and Audio Processing*, vol. 10, no. 5, pp. 293–302, 2002.
- [39] B. L. Sturm, “The GTZAN dataset: Its contents, its faults, their effects on evaluation, and its future use.” arXiv preprint arXiv:1306.1461, 2013.
- [40] C. Kereliuk, B. L. Sturm, and J. Larsen, “Deep learning and music adversaries,” *IEEE Transactions on Multimedia*, vol. 17, no. 11, pp. 2059–2071, 2015.
- [41] J. Engel, C. Resnick, A. Roberts, S. Dieleman, M. Norouzi, D. Eck, and K. Simonyan, “Neural audio synthesis of musical notes with WaveNet autoencoders,” in *Proc. International Conference on Machine Learning (ICML), PMLR*, vol. 70, pp. 1068–1077, 2017.

- [42] E. J. Humphrey, S. Durand, and B. McFee, “OpenMIC-2018: An open dataset for multiple instrument recognition,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, pp. 438–444, 2018.
- [43] R. Yuan, Y. Ma, Y. Li, G. Zhang, X. Chen, H. Yin, *et al.*, “MARBLE: Music audio representation benchmark for universal evaluation,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023.
- [44] J. Turian, J. Shier, H. R. Khan, B. Raj, B. W. Schuller, *et al.*, “HEAR: Holistic evaluation of audio representations,” in *Proc. NeurIPS 2021 Competitions and Demonstrations Track, PMLR*, vol. 176, pp. 125–145, 2022.
- [45] NTT Communication Science Laboratories, “EVAR: Evaluation of audio representations.” <https://github.com/nttcslab/eval-audio-repr>. Accessed: October 2026.
- [46] NTT Communication Science Laboratories, “M2D: Masked modeling duo, official implementation and pre-trained weights.” <https://github.com/nttcslab/m2d>. Accessed: October 2026.
- [47] K. W. Cheuk, H. Anderson, K. Agres, and D. Herremans, “nnAudio: An on-the-fly GPU audio to spectrogram conversion toolbox using 1D convolutional neural networks,” *IEEE Access*, vol. 8, pp. 161981–162003, 2020.
- [48] X-ARES developers, “X-ARES: Extensive audio representation and evaluation suite.” <https://github.com/jimbozhang/xares>. Accessed: October 2026.
- [49] A. Quellenec, “MATPAC: Masked latent prediction and classification, code and checkpoints.” <https://github.com/aurianworld/matpac>. Accessed: October 2026.
- [50] M. Won, “MusicFM: code and pre-trained checkpoints.” <https://github.com/minzwon/musicfm>. Accessed: October 2026.
- [51] K. J. Piczak, “ESC: Dataset for environmental sound classification,” in *Proc. ACM International Conference on Multimedia*, pp. 1015–1018, 2015.
- [52] J. Salamon, C. Jacoby, and J. P. Bello, “A dataset and taxonomy for urban sound research,” in *Proc. ACM International Conference on Multimedia*, pp. 1041–1044, 2014.
- [53] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked autoencoders are scalable vision learners,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16000–16009, 2022.
- [54] J.-B. Grill, F. Strub, F. Altché, C. Tallec, P. H. Richemond, E. Buchatskaya, C. Doersch, B. A.

- Pires, Z. D. Guo, M. G. Azar, B. Piot, K. Kavukcuoglu, R. Munos, and M. Valko, “Bootstrap your own latent: A new approach to self-supervised learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [55] A. Baevski, W.-N. Hsu, Q. Xu, A. Babu, J. Gu, and M. Auli, “data2vec: A general framework for self-supervised learning in speech, vision and language,” in *Proc. International Conference on Machine Learning (ICML)*, PMLR, vol. 162, pp. 1298–1312, 2022.
- [56] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *Proc. International Conference on Learning Representations (ICLR)*, 2021.
- [57] Z. Xie, Z. Zhang, Y. Cao, Y. Lin, Y. Wei, Q. Dai, and H. Hu, “On data scaling in masked image modeling,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10365–10374, 2023.
- [58] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, “HuBERT: Self-supervised speech representation learning by masked prediction of hidden units,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3451–3460, 2021.
- [59] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, “High fidelity neural audio compression,” *Transactions on Machine Learning Research*, 2023.
- [60] J. C. Brown, “Calculation of a constant Q spectral transform,” *The Journal of the Acoustical Society of America*, vol. 89, no. 1, pp. 425–434, 1991.
- [61] Y. Li *et al.*, “MERT: official implementation (fairseq plug-in) and training scripts.” <https://github.com/yizhilll/MERT>. Accessed: October 2026.
- [62] D. Perera, V. Letzelter, T. Mariotte, A. Cortés, M. Chen, S. Essid, and G. Richard, “Annealed multiple choice learning: Overcoming limitations of winner-takes-all with annealing,” in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 10541–10574, 2024.
- [63] Z. Li and D. Hoiem, “Learning without forgetting,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 12, pp. 2935–2947, 2018.
- [64] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network.” arXiv preprint arXiv:1503.02531, 2015.
- [65] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, “Over-

- coming catastrophic forgetting in neural networks,” *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [66] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. International Conference on Learning Representations (ICLR)*, 2019.
- [67] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.
- [68] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 8024–8035, 2019.
- [69] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, “Transformers: State-of-the-art natural language processing,” in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, pp. 38–45, 2020.
- [70] M. Ott, S. Edunov, A. Baevski, A. Fan, S. Gross, N. Ng, D. Grangier, and M. Auli, “fairseq: A fast, extensible toolkit for sequence modeling,” in *Proc. Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT): Demonstrations*, pp. 48–53, 2019.
- [71] R. O. Duda, P. E. Hart, and D. G. Stork, *Pattern Classification*. John Wiley and Sons, 2000.
- [72] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [73] C.-C. Chang and C.-J. Lin, *LIBSVM: A Library for Support Vector Machines*, 2001. Available: <http://www.csie.ntu.edu.tw/~cjlin/libsvm>.
- [74] I. Guyon, A. Saffari, G. Dror, and G. Cawley, “Agnostic learning vs. prior knowledge challenge,” in *Proceedings of International Joint Conference on Neural Networks*, August 2007.
- [75] “Agnostic Learning Vs. Prior Knowledge Challenge by International Joint Conference on Neural Networks (IJCNN).” Available: <http://www.agnostic.inf.ethz.ch> [Accessed November 02, 2018].
- [76] I. Guyon, A. Saffari, G. Dror, and G. Cawley, “Analysis of the IJCNN 2007 agnostic learning vs. prior knowledge challenge,” *Neural Networks*, vol. 21, pp. 544–550, December 2008.

Chapter A First Appendix if Required

If you want to add appendices then make sure you provide a proper title. Add different appendices by using the chapter command as shown for this one. They will automatically be added to the table of contents.

A.1 References

See how .bib file is prepared. The bibliography would be automatically added by Latex. So here is how you cite papers (you'll need to view the .bib file to understand). [71] is the correct way to make one citation and [72, 73] to make multiple citations. You can see various reference styles for manuals [73], conferences [74], websites [75] and journal articles [76]. Also, note that all citations appear in the order in which they appear within text. They are not sorted according to author names.

A.2 Equations

Make sure you use the math environment for writing equations. Again, make sure you do not hard code equation numbers. Use label and ref commands of Latex to number them and reference them. Here is an example of (A.1)

$$E = mc^2 \tag{A.1}$$

Note, how the equation is referenced by its equation number within brackets.

A.3 Figures

To insert a figure, you can use the following code as template. Here the figure is referenced using ref command of Latex. So Figure A.1 is the FAST logo. DO NOT HARD CODE FIGURE NUMBERS. Let Latex manage them for you.



Figure A.1: Fast Logo is the Caption.

A.4 Tables

Note how tables can be added in Latex. Also, make sure you use label and ref commands to define and use table numbers. You can use Table A.1 as a template to make tables, add captions to them and reference them. DO NOT HARD CODE TABLE NUMBERS. Let Latex manage their numbering for you.

If you are having problems in managing Latex tables, you may use Lyx <https://www.lyx.org/>.

Table A.1: Give a Caption to the Table.

No.	Column 1	Column 2
1.	Some data	More data
2.	Some values	More values

A.5 Pseudo Code

When you need to explain an existing algorithm of an algorithm you devised, then you should use pseudo code notation. Given below, algorithm 3, is an example. Note how the numbering is managed by the Latex itself.

Data: this text

Result: how to write algorithm with L^AT_EX2_ε

initialization;

while *not at end of this document* **do**

 read current;

if *understand* **then**

 go to next section;

 current section becomes this one;

else

 go back to the beginning of current section;

end

end

Algorithm 3: How to write algorithms

A.6 Code of Programming Languages

If you need to add C++, Java or any other programming language's code, then you can use "listings" package. Following is an example. Normally, the code is only added in the appendices to avoid clutter

in the document.

```
#include <stdio.h>
#define N 10
/* Block
   * comment */

int main()
{
    int i;

    // Line comment.
    puts("Hello world!");

    for (i = 0; i < N; i++)
    {
        puts("LaTeX is also great for programmers!");
    }

    return 0;
}
```

A.7 Recommendations for L^AT_EX

Here we are providing you with only two Latex source files. main.tex and fypbib.bib file. You can also start a project at overleaf.com and upload the zip file of this template there and share the Latex source between your group and supervisor. This way all of you can work on the Latex files together online.